



Revisión de conceptos asociados al cálculo de componentes principales

Brayan Alejandro Romero Castro

Universidad Nacional de Colombia
Facultad de Ciencias, Departamento de Matemáticas
Bogotá D.C, Colombia
2024

Revisión de conceptos asociados al cálculo de componentes principales

Brayan Alejandro Romero Castro

Trabajo de grado presentado como requisito parcial para optar por el título de:
Matemático

Supervisor:
Ph.D., Juan Galvis

Linea de trabajo :
Análisis numérico y aprendizaje de máquina - Matemática aplicada

Universidad Nacional de Colombia
Facultad de Ciencias, Departamento de Matemáticas
Bogotá D.C, Colombia
2024

Resumen

El análisis de componentes principales (PCA por sus siglas en inglés) es una técnica utilizada para describir un conjunto de datos en términos de nuevas variables (“componentes”) no correlacionadas. Las componentes se ordenan por la cantidad de varianza original que describen, por lo que la técnica es útil para reducir la dimensionalidad de un conjunto de datos. Estas componentes principales son descritas en álgebra lineal como los vectores de la matriz U en la descomposición en valores singulares (SVD por sus siglas en inglés) de la matriz de datos (o como los vectores propios de la matriz de covarianza asociada a los datos). En el presente proyecto estudiaremos algoritmos clásicos para el cálculo de estas componentes cómo lo son el Método de potencias y el método de Jacobi expuesto en [1] y los compararemos con algoritmos recientes como lo son las rutinas actuales que utiliza Intel en [2], 1980 (LAPACK routines), con adelantos recientes en el uso de modelos generativos (Redes Neuronales), que usan su rango para aplicar el método de potencias que utilizan Z. Li, J. Liu, S. Ghosh, J. Han, y J. Scarlett en [3].

Palabras clave: PCA, Álgebra lineal, Aprendizaje de máquina, Componentes principales, Modelos generativos, Rutinas.

Índice

1	Introducción	2
2	Preliminares: Redes neuronales y probabilidad	3
2.1	Redes neuronales	3
2.1.1	El método del gradiente en descenso	10
2.1.2	Diferenciación automática	13
2.1.3	Retropropagación	18
2.2	Algunos conceptos de probabilidad	21
2.2.1	Probabilidad básica.	21
2.2.2	Entropía	25
2.2.3	Inferencia variacional	34
2.2.4	Truco de reparametrización	39
2.2.5	Codificador automático variacional	41
3	Métodos Clásicos	46
3.1	Jacobi	46
3.2	Iteración de potencias	48
3.2.1	Deflación de matrices	50
3.2.2	Método por iteración ortogonal	51
4	Algoritmo bidiagonal de Golub-Kahan para la descomposición SVD.	55
4.1	Primera parte (bidiagonalización)	56
4.2	Segunda parte (SVD de la parte bidiagonal)	57
4.2.1	Algoritmo	59
5	Componentes principales, utilizando el rango de un operador	63
5.1	Cuando el rango es el de una matriz.	63
5.2	Cuando el rango es el de un VAE	66
6	Comparación	71
	References	79

Lista de Figuras

2-1	Arquitectura de Red Neuronal no totalmente conectada, con 7 neuronas y 3 capas, done la primera capa L_1 tiene 2 neuronas, L_2 3 neuronas y L_3 las restantes 2 neuronas. Idea propia.	4
2-2	Gráfica de la función sigmoide, lleva a todos los valores entre 0 y 1, como vemos es una función creciente que para $-\infty$ tiende a 0 y para ∞ tiende 1, la idea de esto es estandarizar los valores que entran a las neuronas, donde valores cercanos a 0 indican poca activación y valores cercanos a 1 indican mucha activación.	5
2-3	Gráfica de la función tangente hiperbólica, lleva a todos los valores entre -1 y 1, como vemos es una función creciente que para $-\infty$ tiende a -1 y para ∞ tiende 1, la idea de esto es estandarizar los valores que entran a las neuronas, donde valores cercanos a -1 indican poca activación y valores cercanos a 1 indican mucha activación.	5
2-4	Gráfica de la función ReLU, lleva a todos los valores entre 0 y ∞ , como vemos es una función creciente que para valores negativos es 0 y para valores positivos da el mismo numero, a diferencia de las demás esta es la única no diferenciable en todo su dominio, la ventaja es su fácil representación y manejo pues es casi lineal.	6
2-5	Ejemplo de como actúan los pesos, sesgos y función de activación en una arquitectura de red neuronal. La operación se puede representar como un aplicación no lineal de multiplicación de matrices, pues cada entrada se multiplica por sus pesos y se le suma algún sesgo. [4], en esta figura se ve a la izquierda la interacción entre las neuronas de la capa 0 con la primera neurona de la capa siguiente, en la formula de la derecha esto solo vendría a representar la primera componente que resulta de hacer la multiplicaciones de la primer fila, por el vector de neuronas mas el primer sesgo, pasado por la función de activación. Figura inspirada en [4]	7
2-6	Estructura de una CNN, la primera capa solo recibe la entrada, las siguientes hacen convolución con algún filtro que extrae ciertas características, luego se comprimen en las capas de agrupación, y por último se pasa por una red totalmente conectada para agregar no linealidades [4]. Figura inspirada en [4].	8

- 2-7** Forma en la que actúa un filtro convolucional (ver [4]). En cada ítem se ve como actúa el filtro aplicado a una sección de la imagen. En a) actúa sobre una parte central de la imagen haciendo la respectiva convolución de (2-2), en b) actúa en una esquina, sin salirse de la imagen, en c) y d) se sale luego para este caso se agregan 0's en la multiplicación de (2-2). 9
- 2-8** a) Esquema de la convergencia del descenso del gradiente [5], se ve como dado pequeños pasos (flechas y puntos rojos) se llega a un mínimo o máximo local. Los círculos representan curvas de nivel de alguna función de 2 variables w_1 y w_2 . b) Esquema de la posible convergencia del descenso del gradiente estocástico [5], como se ve los pasos (flechas y puntos rojos) zigzaguean hasta encontrar el mínimo o máximo de la función de dos variables, esto es debido a que es una aproximación de la dirección de mayor descenso (o ascenso). Figura inspirada en [5] 11
- 2-9** Grafo asociado a la función $f(x_1, x_2) = \ln(x_1) + x_1x_2 - \sin(x_2)$, cada nodo representa una operación como por ejemplo una multiplicación, una suma, un logaritmo etc. Las flechas representan la composición de estas operaciones [6]. Para este grafo tenemos que $v_{-1} = x_1$, $v_0 = x_2$, $v_1 = \ln(v_{-1})$, $v_2 = v_{-1} \cdot v_0$, $v_3 = \sin(v_0)$, $v_4 = v_1 + v_2$, $v_5 = v_4 - v_3$. Figura inspirada en [6] 14
- 2-10** Tabla asociada al proceso directo en la diferenciación automática de la función $f(x_1, x_2) = \ln(x_1) + x_1x_2 - \sin(x_2)$ (tomada de [6]), las flechas indican el orden en que se van hacer las composiciones en la parte de forward prime trace y las respectivas derivaciones en la parte de forward tangent (derivate) trace. . 14
- 2-11** Tala asociada a la función $f(x_1, x_2) = \ln(x_1) + x_1x_2 - \sin(x_2)$, utilizando el modo reverso, a diferencia del modo anterior, en este las derivadas se calculan de abajo hacia arriba (o de derecha hasta izquierda), hasta llegar el nodo de índice menor en la Figura **2-9**, todo esto con uso de la regla de la cadena expuesta en la ecuación (2-4) 16
- 2-12** Grafo para la ejecución del *reverse mode* asociado a las ultimas dos capas de una red neuronal, los grafos en azul, son operaciones intermedias, cuando se pasa la información de una capa a la otra, en este caso representan la función de activación. 18
- 2-13** Gráfica de $-x \log_2 x$, como se ve cerca al centro es máximo y en los extremos es 0, lo cual concuerda con la noción expuesta de entropía 27
- 2-14** Aproximaciones de las distribución p , por medio de la divergencia KL , en la de la izquierda vemos que cuando q se coloca a la derecha y empezamos a minimizar $D_{KL}(p||q)$ la aproximación que tenderemos es una distribución que busca la media, en cambio si aproximamos por la izquierda vemos que la aproximación busca algún modo de la distribución como se ve en la figura de la derecha. (Figura tomada de [7]). 32

- 2-15** Histograma de frecuencias asociada a la medición de alturas de una muestra de una población. Como se ve la mayoría de las personas cae el rango de altura entre 165 y 175 cm, y datos mas extremos como personas de 160 o mas de 180 tienen frecuencias muy bajas. Imagen tomada de [8] 34
- 2-16** Cuando intentamos aproximar una distribución bimodal (gráfica azul) por medio de un conjunto de distribuciones unimodales (gráfica verde punteada), por medio de la divergencia KL a la izquierda, es decir, haciendo $\arg \min_q D_{KL}(q||p)$ lo que encontramos es una distribución que aproxime alguno de los dos modos [7]. Imagen tomada de [7] 36
- 2-17** Direct Acyclic Graph asociado a una distribución exponencial con parámetro $\lambda = 1$ y una distribución normal con media $\sigma = z$, la flecha entre en nodo de la variable aleatoria con distribución exponencial y el nodo de la variable aleatoria con distribución normal indica una relación de condicional entre ella (Imagen tomada de [8]). 37
- 2-18** Esquema de un codificador variacional, la primera parte (encoder) trata de reducir la dimensionalidad de la entrada (input) y la segunda parte (decoder) trata de reconstruir lo que se redujo en el encoder. Los círculos y flechas que se encuentran en el encoder y decoder hace referencia a ellos son como tal redes neuronales. Imagen inspirada de [8]. 41
- 2-19** Esquema de un VAE, en este caso el encoder mapea un dato X , a los parámetros de una distribución normal multivariada ($\mu_\phi(X)$ y $\sigma_\phi^2(X)$), y el decoder a partir de un muestreo de esta distribución intenta decodificar el dato. Imagen inspirada de [9] 42
- 2-20** Representaciones latentes de los datos de Fashion-MNIST [10], los colores representan una prenda en particular, por ejemplo el color amarillo son las representaciones latentes de imágenes de botas, Así es como se verían distribuidos los datos de todas las fotos de las prendas (imágenes de tamaño 784) luego de haberlas pasado por el encoder a un espacio latente de dimensión 2. Como se ve los datos parecen estar distribuidos cerca a (0,0) y para ciertas direcciones se acomodan representaciones de los mismos tipos de datos, Imagen inspirada de [10] 43
- 2-21** Espacio latente (de dimención 2) de un codificador automático variacional, entrenado con un imágenes de caras de la universidad de Chicago [11], se asigno a algunos puntos cercanos a (0,0) su respectiva decodificación por medio del decodificador y se graficaron sobre ellos. En la imagen se aprecia como la red neuronal aprende determinadas características, por ejemplo en la imagen se ve que subiendo en el eje y las personas tiene cabello y bajando no. Imagen propia [12] 45

4-1	Reflexión de Householder en $\mathbb{R}^2$, escogiendo un vector v apropiado de tal manera que la reflexión de x respecto al normal de v sea un múltiplo escalar del vector canónico $(1, 0)$ (Idea de [1]).	56
5-1	Proyección en el rango de G una matriz. Imagen propia	64
6-1	Tensor de datos M , en donde la columna j representa la información contenida en una imagen j del data set que hallamos tomado (en nuestro caso es el de las caras de la Universidad de Chicago [11]), y la fila i -ésima representa la información contenida en el pixel i -ésimo de todas las imagenes. Imagen inspirada en [11]	71
6-2	A la izquierda se encuentra la primer cara propia del conjunto de datos de caras de chicao tomado de [11], esta cara es la representación del vector propio de mayor magnitud asociado a MM^T , a la derecha se encuentra la cara promedia de este conjunto de datos, que se calcula sumando todas la caras y dividiendo por el total en cada canal. Imagen propia	72
6-3	a) En esta gráfica se ve el promedio del error de reconstrucción para diferentes valores de tasa de aprendizaje, para una imagen en el rango del decoder (la imagen correspondiente a la decodificación del 0 en dimensión 30) normalizado fija, se ve como alrededor de 0.6 obtenemos un mínimo de 0.625. b) En esta parte se ve la proyección de una imagen que pertenece al decoder sobre el mismo y sobre su imagen normalizada, también se visualiza la proyección de un imagen aleatoria tanto en la imagen del decoder, como en la imagen normalizada de este. Recordar que las imágenes son de caras y fueron tomadas de [11]. Imagen propia	75
6-4	a) En la parte superior izquierda se ve el vector propio dominante de Σ^T (superponiendo los 3 canales) que aproxima Ppower con 10 iteraciones (se multiplico por un factor de 30 para poder visualizarla y no fuera totalmente oscura). En la parte superior derecha se ve el vector propio dominante de Σ^T real calculado con numpy. b) La gráfica de la parte inferior izquierda muestra la diferencia entre los valores propios dominantes en cada canal, en azul los calculados por Ppower y en naranja los calculados por la librería numpy de python. La gráfica de la parte inferior derecha muestra el error en norma 2 de los vectores propios dominantes en cada uno de los 3 canales. Puede consultar el código donde se hace esta implementación en [12]. Y del cual se inspiraron las imagenes de este documento	76

1 Introducción

En diversos problemas actuales como en el reconocimiento facial, se requiere calcular ciertas direcciones en un espacio de datos gigante, en las cuales estos tengan poca varianza, o de manera mas coloquial direcciones en las cuales sea mas fácil representar los datos esto nos sirve no solamente para hallar características relevantes de estos, si no también para ahorrar el tiempo y costo computacional que gastamos en representar todos los datos. Para todo esto se necesita calcular los valores y vectores propios de matrices de cientos o miles de entradas, desde el punto del álgebra lineal, esto es bastante costoso. Es por ello que a lo largo de la historia se han desarrollado métodos eficientes para este calculo, y que arrojan aproximaciones bastante certeras de los valores y vectores propios de estas matrices (G.H. Golub [1] cuarta e.d, año 2013).

En este proyecto expondremos diferentes métodos para el cálculo de valores propios de forma computacional, como lo son el método de Jacobi (G.H. Golub [1] cuarta e.d, Pag 476-477, año 2013), el método por iteración potencias (G.H. Golub [1] cuarta e.d, Pag 450-454, año 2013) expuestos en el Capítulo 3, el algoritmo SVD de Golub-Kahan (G.H. Golub [1] cuarta e.d, Pag 488-491, año 2013) expuesto en el Capítulo 4 y el método proyectado de potencias (Z. Li, J. Liu, S. Ghosh, J. Han, y J. Scarlett, [3],p. 33, año 2021) expuesto en el Capítulo 5. El propósito de nuestro trabajo es comparar la eficiencia computacional de cada uno de los métodos, en lenguajes como Python y Matlab (para una matriz en particular) lo cual se hace en el Capítulo 6, además de indagar en la teoría que los sustentan. Se ha destinado un capitulo de preliminares para obtener los prerrequisitos en aprendizaje de máquina y probabilidad que exige el Capítulo 5. Se asumen ciertos conocimientos básicos en álgebra lineal, para los Capítulos 3 y 4. Para obtener el código correspondiente a cada capitulo, puede consultar el repositorio de B. A. R. Castro y C. E. N. Guzman en [12].

2 Preliminares: Redes neuronales y probabilidad

En este capítulo, introduciremos algunos conceptos fundamentales en redes neuronales (Sección 2.1) y probabilidad (Sección 2.2) que utilizaremos, a lo largo de este trabajo, principalmente en el Capítulo 5, en donde presentaremos una técnica utilizando modelos generativos, que son un tipo especial de redes neuronales, profundizaremos mucho mas en esto en la Sección 2.2.5. Si se quiere profundizar mas en estos temas se recomienda leer [4] y [8], en donde se detallan a profundidad las definiciones y teoremas que damos en este capítulo.

2.1. Redes neuronales

Las *redes neuronales*, son un tipo especial de funciones que introducen no linealidades en el procesamiento de información (Capítulo 3 de [4]), en general la intención de esto es aprender ciertas características intrínsecas de los datos. En la presente sección se expondrá en detalle lo que son estas funciones, cómo funcionan, cómo aprenden y algunas variaciones recientes. Para profundizar aún más en este tema puede consultar el Capítulo 3 de [4].

Definición 2.1.1. Una **Arquitectura de Red Neuronal** es un grafo dirigido $G = (L, E)$ tal que $V = \{L_1, L_2, \dots, L_n\}$ es una partición de los nodos, con las siguientes propiedades:

- i) Para todo $i = 1, 2, \dots, n - 1$, existe (a, b) , tal que $a \in L_i$ y $b \in L_{i+1}$.
- ii) Si $(a, b) \in E$, entonces existe $i \in \{1, 2, \dots, n - 1\}$, tal que $a \in L_i$ y $b \in L_{i+1}$.

A L_i se le conoce como la capa i de la red neuronal (para $i = 1, 2, \dots, n$), a los elementos de V se le conocen como neuronas, y a los elementos de E se les conoce como conexiones [7].

Un ejemplo de arquitectura de red neuronal es ilustrada en la Figura 2-1.

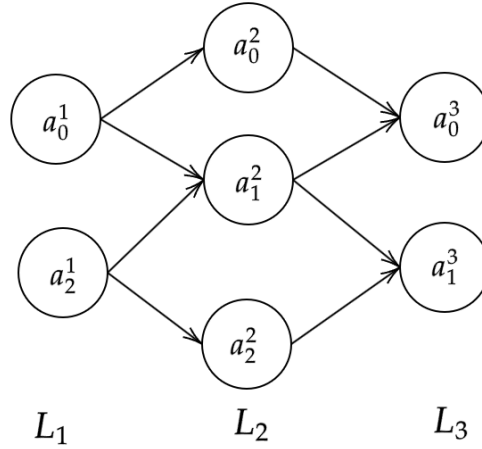


Figura 2-1: Arquitectura de Red Neuronal no totalmente conectada, con 7 neuronas y 3 capas, donde la primera capa L_1 tiene 2 neuronas, L_2 3 neuronas y L_3 las restantes 2 neuronas. Idea propia.

Notemos que en la arquitectura que se ve en la Figura 2-1 no necesariamente todas las neuronas de una capa se unen con todas las neuronas de la siguiente, en ese caso se dice que la red no está totalmente conectada, en caso contrario se dice que está totalmente conectada [4].

Definición 2.1.2. Dada una arquitectura de red neuronal con L capas a esta se le puede asociar una función (**Red Neuronal**) de la siguiente manera

$$f(x) = f_{l_L}(f_{l_{L-1}}(\dots f_{l_1}(x; \theta_1) \dots; \theta_{L-1}); \theta_L) \quad (2-1)$$

donde

- i) El vector x es el vector de entrada.
- ii) La función f_{l_i} es la función de activación de la i -ésima capa.
- iii) Los parámetros θ_i son los parámetros (pesos y sesgos) asociados con la función de activación de la i -ésima capa.

Cada capa l_i realiza una transformación generalmente no lineal de la entrada x ($x \in \mathbb{R}^n$ para algún natural n), esta transformación es asociada a las uniones entre las neuronas y se les conoce como funciones de activación [4]. La salida de cada capa es utilizada como entrada para la siguiente capa.

Las funciones de activación f_{l_i} que se ven en la ecuación (2-1), introducen no linealidades en la red, permitiendo así que la red aprenda representaciones más complejas y no lineales

de los datos. Ejemplos comunes de funciones de activación son la función sigmoide (Figura 2-2), tangente hiperbólica (Figura 2-3), y la ReLU (Figura 2-4)(esta última es la mas usada [7]).

i) *Función sigmoide*, $f(x) = \frac{1}{1+e^{-x}}$.

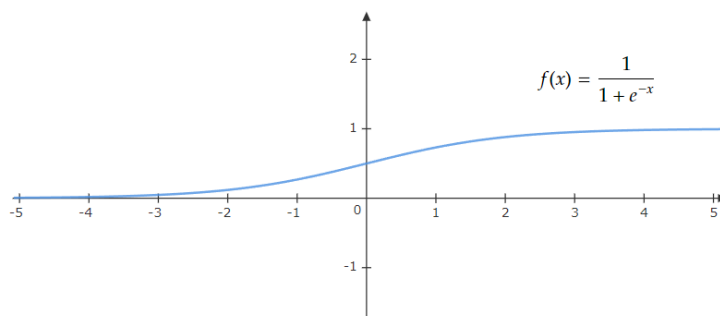


Figura 2-2: Gráfica de la función sigmoide, lleva a todos los valores entre 0 y 1, como vemos es una función creciente que para $-\infty$ tiende a 0 y para ∞ tiende a 1, la idea de esto es estandarizar los valores que entran a las neuronas, donde valores cercanos a 0 indican poca activación y valores cercanos a 1 indican mucha activación.

ii) *Función tangente hiperbólica*, $f(x) = \tanh(x)$

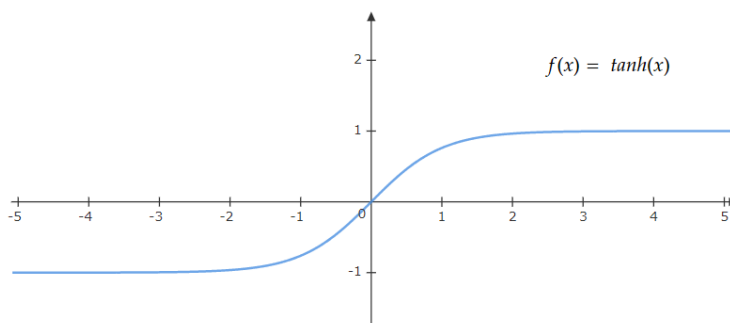


Figura 2-3: Gráfica de la función tangente hiperbólica, lleva a todos los valores entre -1 y 1, como vemos es una función creciente que para $-\infty$ tiende a -1 y para ∞ tiende a 1, la idea de esto es estandarizar los valores que entran a las neuronas, donde valores cercanos a -1 indican poca activación y valores cercanos a 1 indican mucha activación.

iii) *Función ReLU*, $f(x) = xI_{[0,\infty)}(x)$

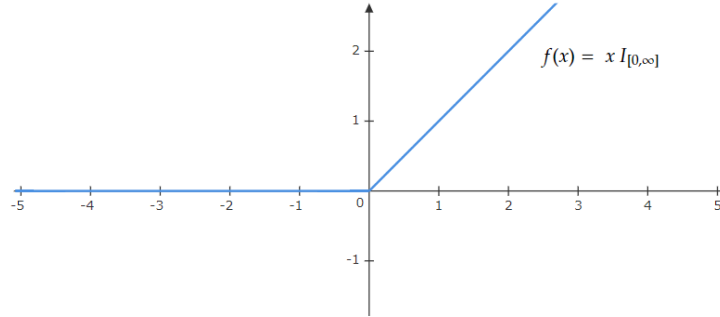


Figura 2-4: Gráfica de la función *ReLU*, lleva a todos los valores entre 0 y ∞ , como vemos es una función creciente que para valores negativos es 0 y para valores positivos da el mismo número, a diferencia de las demás esta es la única no diferenciable en todo su dominio, la ventaja es su fácil representación y manejo pues es casi lineal.

Notemos que las anteriores funciones son escalares pero para nuestro problema necesitamos que las entradas y las salidas a las capas sean vectores [3], es por esto que las funciones actuarán componente a componente [4],

$$f \left(\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \right) = \begin{bmatrix} f_1(x_1) \\ f_2(x_2) \\ \vdots \\ f_n(x_n) \end{bmatrix}.$$

Donde los f_j son funciones de activación, generalmente para cada capa todos los f_j se toman como la misma función de activación [4], en ese caso simplemente se reduce la notación colocando $f_j = f$ para todo $j = 1, 2, \dots, n$.

Los pesos que antes nombramos van a determinar que tanto una neurona influye sobre otra esto se hace como una combinación lineal de las entradas multiplicadas por estos pesos, los sesgos nos dirán que tan fácil es que una neurona de la capa siguiente se active, entre más grande en valor absoluto sea este valor mayor dificultad habrá para activarla. En la Figura 2-5, se muestra como actuara la función de activación sigmoide de una capa a la otra, en este caso cada fila representa el valor numérico que entrara en la siguiente neurona.

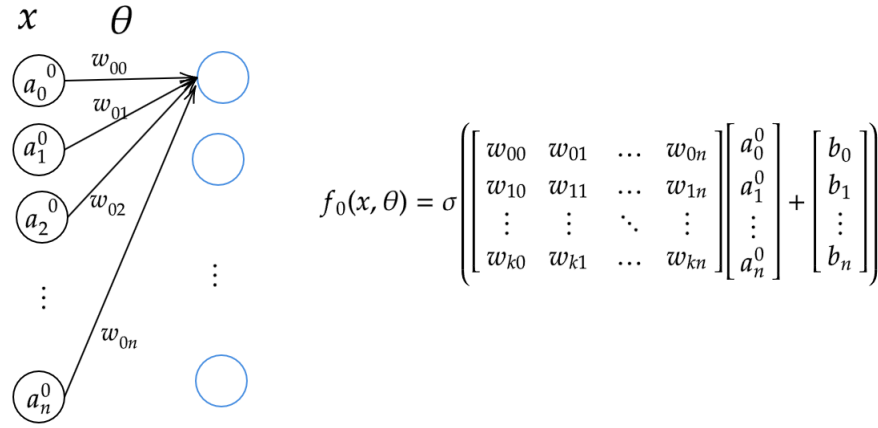


Figura 2-5: Ejemplo de como actúan los pesos, sesgos y función de activación en una arquitectura de red neuronal. La operación se puede representar como un aplicación no lineal de multiplicación de matrices, pues cada entrada se multiplica por sus pesos y se le suma algún sesgo. [4], en esta figura se ve a la izquierda la interacción entre las neuronas de la capa 0 con la primera neurona de la capa siguiente, en la formula de la derecha esto solo vendría a representar la primera componente que resulta de hacer la multiplicaciones de la primer fila, por el vector de neuronas mas el primer sesgo, pasado por la función de activación. Figura inspirada en [4]

También existen redes neuronales convoluciones, Las CNN por sus siglas en inglés (Convolutional Neural Networks), son un tipo de modelo de aprendizaje profundo especialmente poderoso para el procesamiento de datos que tiene una estructura espacial, como imágenes o señales de audio (Capítulo 10 de [4]). Están diseñadas para reconocer patrones espaciales y de correlación en datos de entrada.

Las CNN se componen principalmente de capas convolucionales, capas de agrupación (pooling), y capas completamente conectadas al final para la clasificación [4]. La capa convolucional aplica filtros a la entrada para extraer características relevantes, mientras que la capa de agrupación reduce la dimensionalidad de las características, preservando la información importante. Estas capas se suelen alternar varias veces antes de las capas completamente conectadas [4] (ver Figura 2-6).

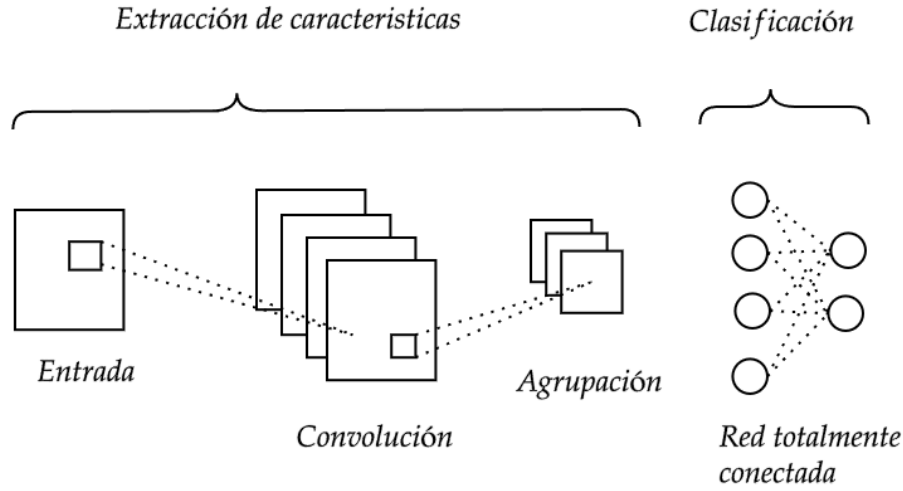


Figura 2-6: Estructura de una CNN, la primera capa solo recibe la entrada, las siguientes hacen convolución con algún filtro que extrae ciertas características, luego se comprimen en las capas de agrupación, y por último se pasa por una red totalmente conectada para agregar no linealidades [4]. Figura inspirada en [4].

Explicaremos esto un poco mas en detalle. Para ello primero necesitamos saber que es un filtro, y a que hacemos referencia cuando decimos que hacemos convolución y agrupación.

Suponga que tenemos una imagen de entrada I , con altura H , anchura W , y D canales de color, un filtro K simplemente va ser una matriz $F \times F$ de tamaño menor a las dimensiones de la imagen. La operación de convolución entre la imagen de entrada y el filtro se denota como $I \otimes K$ [4], y produce un mapa de características, que va ser una matriz, las entradas de esta matriz están dadas por,

$$(I \otimes K)_{ij} = \sum_{m=0}^{F-1} \sum_{n=0}^{F-1} \sum_{d=0}^{D-1} I_{(i+m)(j+n)d} \cdot K_{mn}. \quad (2-2)$$

Gráficamente lo que nos dice (2-2) es pasar una lamina por la imagen tensor e ir combinando con sumas y productos los elementos de esta lamina y la imagen. A continuación mostramos en la Figura 2-7 (tomada de [4]) como se esta pensando la suma anterior.

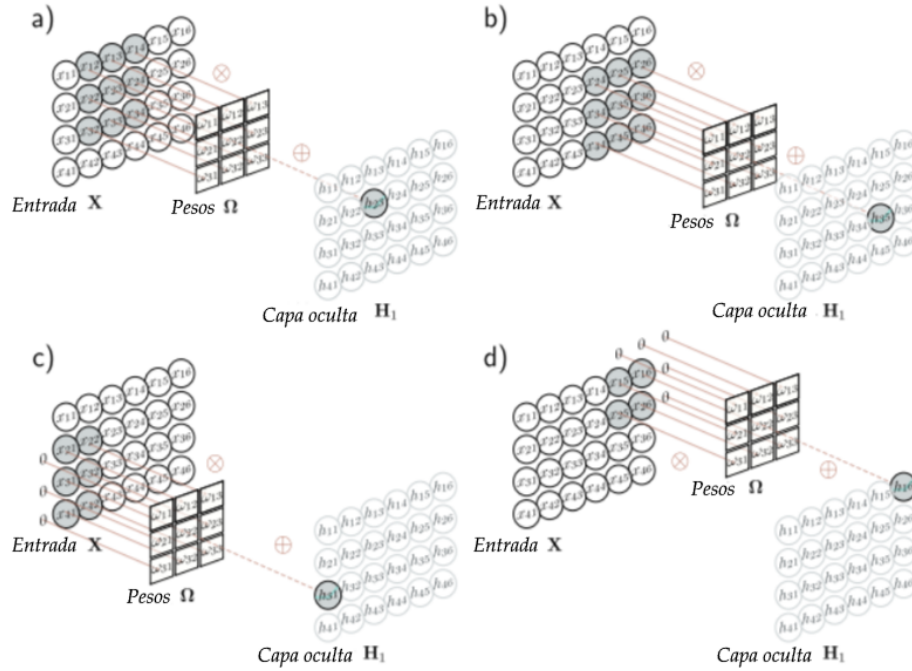


Figura 2-7: Forma en la que actúa un filtro convolucional (ver [4]). En cada ítem se ve como actúa el filtro aplicado a una sección de la imagen. En a) actúa sobre una parte central de la imagen haciendo la respectiva convolución de (2-2), en b) actúa en una esquina, sin salirse de la imagen, en c) y d) se sale luego para este caso se agregan 0's en la multiplicación de (2-2).

Después de la convolución, se aplica una función de activación no lineal como por ejemplo un ReLU, para introducir no linealidad en la Red, y por último se puede pasar por capas de agrupación que son básicamente capas que convierten una región de la imagen en una única salida, que generalmente es el valor máximo (max-pooling) o el promedio (average-pooling) de esa región (ver [4]).

Por ejemplo, en el max-pooling, para cada región de la entrada, se selecciona el valor máximo y se asigna como la salida (ver [4]). Esto ayuda a conservar las características más relevantes mientras se reduce la dimensionalidad.

Generalmente el orden en el que se aplican estas capas es (ver [4]): Convolución, activación, agrupación y por ultimo capas neuronales completamente conectadas. Note que las capas de convolución, y agrupación no son capas completamente conectadas pues la operación se aplica de manera sectorizada, haciendo que la entrada de la siguiente capa solo depende de las neuronas que lean píxeles cercanos [7].

Ahora que ya sabemos que es una red Neuronal, nos disponemos a aprender como hacer que

esta *aprenda*, Eso simplemente consiste en una optimización sobre los parámetros de la red neuronal para que este reconstruya lo que sea que queramos (ver [5] y [13]), este proceso de optimización se realiza por medio de lo que se conoce como descenso del gradiente.

2.1.1. El método del gradiente en descenso

Suponga que tenemos una función $J : \mathbb{R}^n \rightarrow \mathbb{R}$ que es diferenciable y convexa en un cierto abierto U y suponga que $\theta_0 \in U$, entonces existe $\alpha \in \mathbb{R}$, tal que la sucesión de puntos

$$\theta_{n+1} = \theta_n - \alpha \nabla J(\theta_n),$$

converge a un mínimo local de la función J en U (ver [5]). Intuitivamente lo que estamos haciendo es utilizar el hecho de que el gradiente apunta en la dirección de máximo ascenso, luego, menos el gradiente apunta en la dirección de máximo decrecimiento, de esta forma, si damos pasos en esta dirección, encontraremos un mínimo local como se muestra en la Figura 2-8 (tomada de [5]), el pseudocódigo, de este descenso se muestra en 1, y se quiere consultar la implementación de este en Python puede consultar [14].

Algorithm 1 Descenso del gradiente

Require: $\varepsilon > 0$, $N > 0$, f convexa y diferenciable y θ_0

- 1: $k = 0$
 - 2: **while** $\|\nabla f(\theta_0)\| \geq \varepsilon$ or $k < N$ **do**
 - 3: $\theta_0 = \theta_0 - \alpha \nabla f(\theta_0)$
 - 4: $k = k + 1$
 - 5: **end while**
 - 6: Devolver θ_0
-

A α se le conoce como la tasa de apredizaje, si es demasiado pequeña puede hacer que el algoritmo converja lentamente, si es demasiado grade puede que el algoritmo diverja. Generalmente en el campo practico se escogen valores entre 0 y 1.

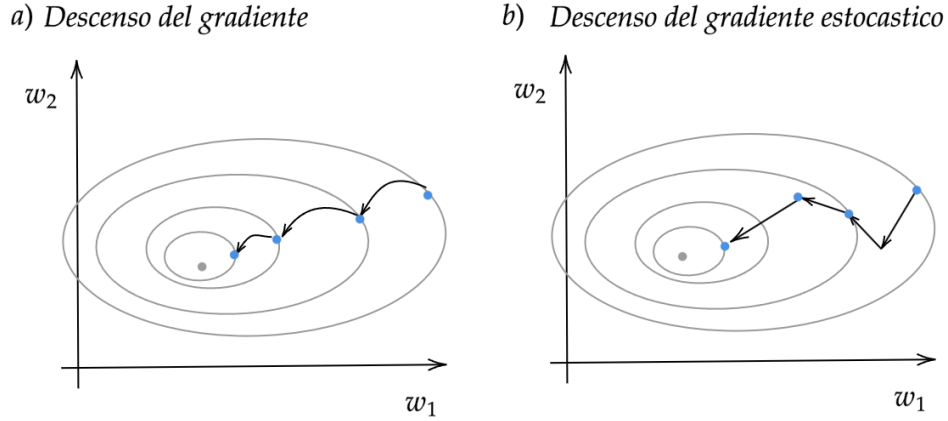


Figura 2-8: a) Esquema de la convergencia del descenso del gradiente [5], se ve como dado pequeños pasos (flechas y puntos rojos) se llega a un mínimo o máximo local. Los círculos representan curvas de nivel de alguna función de 2 variables w_1 y w_2 . b) Esquema de la posible convergencia del descenso del gradiente estocástico [5], como se ve los pasos (flechas y puntos rojos) zigzaguean hasta encontrar el mínimo o máximo de la función de dos variables, esto es debido a que es una aproximación de la dirección de mayor descenso (o ascenso). Figura inspirada en [5]

Existen algunas variantes que se pueden usar como actualizar en cada paso la tasa de aprendizaje (ver [5]) el cual se conoce gradiente en descenso con sondeo lineal. Este en cada paso quiere encontrar el α_k de tal manera que el movimiento sea lo mas efectivo posible, es decir busca.

$$\alpha_k = \arg \min_{\alpha \in \mathbb{R}} J(\theta_k - \alpha \nabla J(\theta_k)).$$

El pseudocódigo de la variante del descenso del gradiente con sondeo lineal se muestra en 2 (para mas detalles de la implementación en Python puede consultar), [14].

Algorithm 2 Descenso del gradiente con sondeo lineal

Require: $\varepsilon > 0$, $N > 0$, f convexa y diferenciable y θ_0

- 1: $k = 0$
 - 2: **while** $\|\nabla f(\theta_0)\| \geq \varepsilon$ or $k < N$ **do**
 - 3: $\alpha_k = \arg \min_{\alpha \in \mathbb{R}} f(\theta_k - \alpha \nabla f(\theta_k))$
 - 4: $\theta_0 = \theta_0 - \alpha_k \nabla f(\theta_0)$
 - 5: $k = k + 1$
 - 6: **end while**
 - 7: Devolver θ_0
-

Otra variante es el descenso de gradiente estocástico (SGD por sus siglas en inglés) [5]. Este método difiere del descenso del gradiente clásico en que no utiliza el gradiente completo

de la función de costo en cada iteración para actualizar el θ , si no tan solo algunos datos de entrenamiento (incluso solo uno). Para ver esto mas en detalle considere la función de perdida de un red neuronal de manera general como

$$J(\theta) = \frac{1}{N} \sum_{l=1}^N J_l(\theta),$$

donde J_l suele estar relacionado con la observación l -ésima del conjunto de datos, por ejemplo en el caso de error cuadrático medio

$$J_l(\theta) = \frac{1}{2} \|f_{\theta}(x_l) - y_l\|^2.$$

El algoritmo consiste en barajar aleatoriamente las muestras del conjunto de entrenamiento y calcular el gradiente utilizando solo esta muestra, es decir para la iteración 1 calculamos

$$\theta_1 = \theta_0 - \frac{\alpha}{n_m} \sum_{k=1}^{n_m} \nabla J_{l_k}(\theta_0),$$

donde n_m es el número de muestras que tomamos para calcular el gradiente (generalmente a este número se le llama *tamaño del minibatch*), luego volvemos a repetir, escogiendo otro lote (minibatch) y repetimos el proceso anterior hasta un número de iteraciones dadas o un error escogido. El pseudocódigo del anterior procedimiento se muestra en 3, si se quiere una implementación de este se puede consultar

El pseudocódigo del anterior procedimiento se muestra en 3, si se quiere una implementación en Python de este se puede consultar [14], muchas de las aplicaciones actuales suelen utilizar este método, junto con algunas pequeñas modificaciones [4].

Algorithm 3 Descenso del gradiente estocastico (SGD)

Require: $\varepsilon > 0$, $N > 0$, $f, f_1, f_2, \dots, f_n$, con $\frac{1}{n} \sum_i f_i = f$ θ_0 , m , α

- 1: $k = 0$
 - 2: **while** $\|\nabla f(\theta_0)\| \geq \varepsilon$ or $k < N$ **do**
 - 3: Genere una muestra aleatoria de tamaño m de los índices de las funciones f_i
 - 4: Calcule una aproximación g del gradiente de f , mediante las funciones del item anterior, haciendo un
 - 5: promedio de los gradientes de estas funciones.
 - 6: Haga la actualización
 - 7: $\theta_0 = \theta_0 - \alpha g(\theta)$
 - 8: $k = k + 1$
 - 9: **end while**
 - 10: Devolver θ_0
-

2.1.2. Diferenciación automática

Ahora nos disponemos a entender, como es el cálculo computacional del gradiente. en el anterior algoritmo. Para ello tenemos que saber que existen diferentes formas de hacer diferenciación en una maquina, como lo es la *diferenciación simbólica*, la *diferenciación numérica* y la *diferenciación automática* (ver [6]). La diferenciación simbólica enfrenta la dificultad de convertir un programa de computadora en una única expresión matemática y puede conducir a un código ineficiente [6]. La diferenciación numérica (el método de diferencias finitas) puede introducir errores de redondeo en el proceso de discretización y cancelación. Ambos métodos clásicos tienen problemas para calcular derivadas de orden mas alto, donde aumentan los errores. Finalmente, ambos métodos clásicos son lentos a la hora de calcular las derivadas parciales de una función con respecto a muchas entradas [6].

La diferenciación automática aprovecha el hecho de que cada cálculo informático, por complicado que sea, ejecuta una secuencia de operaciones aritméticas elementales (suma, resta multiplicación, división, etc.) y funciones elementales (e^x , $\log(x)$, $\sin(x)$, $\cos(x)$, etc) [6]. Al aplicar la regla de la cadena repetidamente a estas operaciones las derivadas parciales de orden arbitrario se pueden calcular automáticamente con precisión, utilizando como máximo un pequeño factor constante de mas operaciones aritméticas que el programa original [6]. Existen dos formas bastante conocidas de hacer diferenciación automática, una que es que es hacia adelante (*forward mode*) y otra que es hacia atrás (*reverse mode*) [6].

Comencemos introduciendo el *forward mode*, para ello lo que haremos es escribir nuestra función como un conjunto de operaciones básicas entre composiciones de funciones. Para hacer de esto mas claro suponga que tenemos la función

$$f(x_1, x_2) = \ln(x_1) + x_1x_2 - \sin(x_2) \quad (2-3)$$

y que queremos calcular $\frac{\partial f}{\partial x_1}$. Notemos que f la podemos escribir de diferentes formas como suma o producto de funciones (que a su vez son composiciones de funciones), una de estas podría ser como se ve en la Figura **2-9** (tomada de [6]).

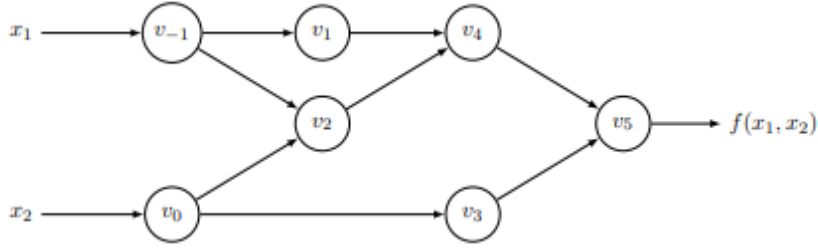


Figura 2-9: Grafo asociado a la función $f(x_1, x_2) = \ln(x_1) + x_1x_2 - \sin(x_2)$, cada nodo representa una operación como por ejemplo una multiplicación, una suma, un logaritmo etc. Las flechas representan la composición de estas operaciones [6]. Para este grafo tenemos que $v_{-1} = x_1$, $v_0 = x_2$, $v_1 = \ln(v_{-1})$, $v_2 = v_{-1} \cdot v_0$, $v_3 = \sin(v_0)$, $v_4 = v_1 + v_2$, $v_5 = v_4 - v_3$. Figura inspirada en [6]

Ahora, con esta información es fácil calcular la derivada parcial $\frac{\partial f}{\partial x_1}$ en cualquier punto, basta ir sucesivamente derivando y evaluando desde el índice -1 , hasta el índice 5. Por ejemplo suponga que queremos $\frac{\partial f}{\partial x_1}(2, 5)$, entonces

Modo primal hacia adelante			Forward Tangent (Derivative) Trace		
$v_{-1} = x_1$	$= 2$		$\dot{v}_{-1} = \dot{x}_1$	$= 1$	
$v_0 = x_2$	$= 5$		$\dot{v}_0 = \dot{x}_2$	$= 0$	
$v_1 = \ln v_{-1}$	$= \ln 2$		$\dot{v}_1 = \dot{v}_{-1}/v_{-1}$	$= 1/2$	
$v_2 = v_{-1} \times v_0$	$= 2 \times 5$		$\dot{v}_2 = \dot{v}_{-1} \times v_0 + \dot{v}_0 \times v_{-1}$	$= 1 \times 5 + 0 \times 2$	
$v_3 = \sin v_0$	$= \sin 5$		$\dot{v}_3 = \dot{v}_0 \times \cos v_0$	$= 0 \times \cos 5$	
$v_4 = v_1 + v_2$	$= 0.693 + 10$		$\dot{v}_4 = \dot{v}_1 + \dot{v}_2$	$= 0.5 + 5$	
$v_5 = v_4 - v_3$	$= 10.693 + 0.959$		$\dot{v}_5 = \dot{v}_4 - \dot{v}_3$	$= 5.5 - 0$	
$y = v_5$	$= 11.652$		$\dot{y} = \dot{v}_5$	$= 5.5$	

Figura 2-10: Tabla asociada al proceso directo en la diferenciación automática de la función $f(x_1, x_2) = \ln(x_1) + x_1x_2 - \sin(x_2)$ (tomada de [6]), la flechas indican el orden en que se van hacer las composiciones en la parte de forward prime trace y las respectivas derivaciones en la parte de forward tangent (derivate) trace.

Note que podríamos haber colocada otra función que dependa de algunos v_i , y tendimos un algoritmo también para calcular, la deriva parcial de una función vectorial respecto de una variable, esto mismo se puede extender para funciones de $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ [6], y por cada vez que hagamos lo de la anterior tabla estaríamos calculado una columna de la matriz Jacobina

$$J_f|_x = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}(x) & \frac{\partial f_1}{\partial x_2}(x) & \dots & \frac{\partial f_1}{\partial x_n}(x) \\ \frac{\partial f_2}{\partial x_1}(x) & \frac{\partial f_2}{\partial x_2}(x) & \dots & \frac{\partial f_2}{\partial x_n}(x) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1}(x) & \frac{\partial f_m}{\partial x_2}(x) & \dots & \frac{\partial f_m}{\partial x_n}(x) \end{bmatrix}.$$

Generalmente se adoptan las siguientes convenciones,

- i) $v_{i-n} = x_i$, $i = 1, 2, \dots, n$ son las variables de entrada,
- ii) v_i , $i = 1, \dots, l$ son las variables intermedias,
- iii) $y_{m-i} = v_{l-i}$, $i = m - 1, \dots, 0$ son las variables de salida.

En la Figura **2-10**, a cada uno de las variables, se le asigna un valor (Primal) y una derivada (Tangent) [6].

Ahora pasemos a entender el *reverse mode* (ver [6]). En este caso lo que haremos es primero hacer un paso hacia adelante calculado sucesivamente los v_i y luego ir calculando la derivada parcial de una función, pero esta vez utilizaremos la regla de la cadena para ir de adelante hacia atrás (derecha a izquierda) en el grafo de la Figura **2-9**. Además para hacer este calculo mas eficiente y recursivamente cómodo llamaremos a variables auxiliares (adjuntos) [6], como sigue

$$\overline{v}_i = \frac{\partial f}{\partial v_i} = \sum_{j: i \rightarrow j} \overline{v}_j \frac{\partial v_j}{\partial v_i} \quad (2-4)$$

donde $i \rightarrow j$ significa que en el grafo existe una flecha del nodo de v_i al nodo de v_j . Para entender mejor esto, hagamos un ejemplo con el grafo de la Figura **2-9**.

Primero calcula v_i para $i = -1, 0, 1, 2, 3, 4, 5$, luego nuestra primera derivada de adelante hacia atrás es $\overline{v}_5 = \frac{\partial f}{\partial v_5}$, que en este caso es 1, luego calcula $\overline{v}_4 = \frac{\partial f}{\partial v_4}$ que por la regla de la cadena es.

$$\frac{\partial f}{\partial v_4} = \frac{\partial f}{\partial v_5} \frac{\partial v_5}{\partial v_4} = \overline{v}_5 \frac{\partial v_5}{\partial v_4} = 1 \cdot 1.$$

Esto mismo lo podemos hacer con cada una de las variables como se muestra, notemos que los índices inferiores dependen de los cálculos que hagamos con las variables que tengan los índices

$$\overline{v}_5 = \frac{\partial f}{\partial v_5} = 1,$$

$$\overline{v}_4 = \frac{\partial f}{\partial v_4} = \frac{\partial f}{\partial v_5} \frac{\partial v_5}{\partial v_4} = \overline{v}_5 \frac{\partial v_5}{\partial v_4} = 1 \cdot 1,$$

$$\bar{v}_3 = \frac{\partial f}{\partial v_3} = \frac{\partial f}{\partial v_5} \frac{\partial v_5}{\partial v_3} = \bar{v}_5 \frac{\partial v_5}{\partial v_3} = 1 \cdot (-1),$$

$$\bar{v}_2 = \frac{\partial f}{\partial v_2} = \frac{\partial f}{\partial v_5} \frac{\partial v_5}{\partial v_4} \frac{\partial v_4}{\partial v_2} = \bar{v}_4 \frac{\partial v_4}{\partial v_2} = 1 \cdot 1,$$

$$\bar{v}_1 = \frac{\partial f}{\partial v_1} = \frac{\partial f}{\partial v_5} \frac{\partial v_5}{\partial v_4} \frac{\partial v_4}{\partial v_1} = \bar{v}_4 \frac{\partial v_4}{\partial v_1} = 1 \cdot 1,$$

por lo tanto,

$$\bar{v}_0 = \frac{\partial f}{\partial v_0} = \frac{\partial f}{\partial v_5} \frac{\partial v_5}{\partial v_4} \frac{\partial v_4}{\partial v_2} \frac{\partial v_2}{\partial v_0} + \frac{\partial f}{\partial v_5} \frac{\partial v_5}{\partial v_3} \frac{\partial v_3}{\partial v_0} = \bar{v}_2 \frac{\partial v_2}{\partial v_0} + \bar{v}_3 \frac{\partial v_3}{\partial v_0} = 1 \cdot v_{-1} + (-1) \cos(v_0) = x_1 - \cos(x_2),$$

$$\bar{v}_{-1} = \frac{\partial f}{\partial v_{-1}} = \frac{\partial f}{\partial v_5} \frac{\partial v_5}{\partial v_4} \frac{\partial v_4}{\partial v_1} \frac{\partial v_1}{\partial v_{-1}} + \frac{\partial f}{\partial v_5} \frac{\partial v_5}{\partial v_4} \frac{\partial v_4}{\partial v_2} \frac{\partial v_2}{\partial v_{-1}} = \bar{v}_1 \frac{\partial v_1}{\partial v_{-1}} + \bar{v}_2 \frac{\partial v_2}{\partial v_{-1}} = 1 \cdot \frac{1}{v_{-1}} + 1 \cdot v_0 = \frac{1}{x_1} + x_2.$$

Reemplazando valores para $x_1 = 2$ y $x_2 = 5$ como lo hicimos para el ejemplo del caso directo nos queda la Figura 2-11.

<i>Rastro primario hacia adelante</i>	Reverse Adjoint (Derivative) Trace
$v_{-1} = x_1 = 2$	$\bar{x}_1 = \bar{v}_{-1} = 5.5$
$v_0 = x_2 = 5$	$\bar{x}_2 = \bar{v}_0 = 1.716$
$v_1 = \ln v_{-1} = \ln 2$	$\bar{v}_{-1} = \bar{v}_{-1} + \bar{v}_1 \frac{\partial v_1}{\partial v_{-1}} = \bar{v}_{-1} + \bar{v}_1 / v_{-1} = 5.5$
$v_2 = v_{-1} \times v_0 = 2 \times 5$	$\bar{v}_0 = \bar{v}_0 + \bar{v}_2 \frac{\partial v_2}{\partial v_0} = \bar{v}_0 + \bar{v}_2 \times v_{-1} = 1.716$
$v_3 = \sin v_0 = \sin 5$	$\bar{v}_{-1} = \bar{v}_2 \frac{\partial v_2}{\partial v_{-1}} = \bar{v}_2 \times v_0 = 5$
$v_4 = v_1 + v_2 = 0.693 + 10$	$\bar{v}_0 = \bar{v}_3 \frac{\partial v_3}{\partial v_0} = \bar{v}_3 \times \cos v_0 = -0.284$
$v_5 = v_4 - v_3 = 10.693 + 0.959$	$\bar{v}_2 = \bar{v}_4 \frac{\partial v_4}{\partial v_2} = \bar{v}_4 \times 1 = 1$
$y = v_5 = 11.652$	$\bar{v}_1 = \bar{v}_4 \frac{\partial v_4}{\partial v_1} = \bar{v}_4 \times 1 = 1$
	$\bar{v}_3 = \bar{v}_5 \frac{\partial v_5}{\partial v_3} = \bar{v}_5 \times (-1) = -1$
	$\bar{v}_4 = \bar{v}_5 \frac{\partial v_5}{\partial v_4} = \bar{v}_5 \times 1 = 1$
	$\bar{v}_5 = \bar{y} = 1$

Figura 2-11: Tala asociada a la función $f(x_1, x_2) = \ln(x_1) + x_1 x_2 - \sin(x_2)$, utilizando el modo reverso, a diferencia del modo anterior, en este las derivadas se calculan de abajo hacia arriba (o de derecha hasta izquierda), hasta llegar el nodo de índice menor en la Figura 2-9, todo esto con uso de la regla de la cadena expuesta en la ecuación (2-4)

Para una función $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$. Si nosotros denotamos el numero de operaciones que se hacen al evaluar la función por $ops(f)$ (ver [6]). Entonces calcular el Jacobiano de tamaño $m \times n$ por el procesos directo, se hace en

$$n \cdot c \cdot ops(f)$$

operaciones, donde c es una constante positiva menor a 6, tipicamente $c \sim [2, 3]$ (ver [6]). Para el modo inverso se necesitan alrededor de

$$m \cdot c \cdot ops(f)$$

operaciones. Notemos además que un solo paso del Reverse mode calcula una sola fila de la matriz jacobiana, lo que hace que esto sea mas eficiente cuando $n \gg m$. Ahora expongamos el método de retro-propagación y como se relaciona con con la diferenciación automática, principalmente con el modo en reversa [13].

2.1.3. Retropropagación

Ahora nos interesaremos en como se calcula computacionalmente de manera efectiva el descenso del gradiente. La idea general es ir devolviendo la información desde la ultima capa hasta la primera, lo que se conoce como retro-propagación (Back-propagation) [13]. Para saber que es exactamente esto necesitaremos algo de notación, Suponga que tenemos L -capas de neuronas, entonces:

- i) Denotamos por w^i , b^i serán los pesos y sesgos respectivamente que van de la capa $i - 1$ a la capa i , para $i = 2, 3, \dots L$.
- ii) Denotamos por a^i es la función de activación de la capa i .
- iii) Denotamos por $z^i = w^i a^{i-1} + b^i$, es una variable intermedia (que se utiliza para reducir la notación en lo que sigue).
- iv) La función C es la función de costo.

Notemos que la salida de la red neuronal es $a^L(z^L)$, y que la función de costo para un solo ejemplo es $C(a^L(z^L)) = \frac{1}{2} \|a^L(z^L) - Y\|^2$, donde Y es la etiqueta del ejemplo que introducimos en la red.

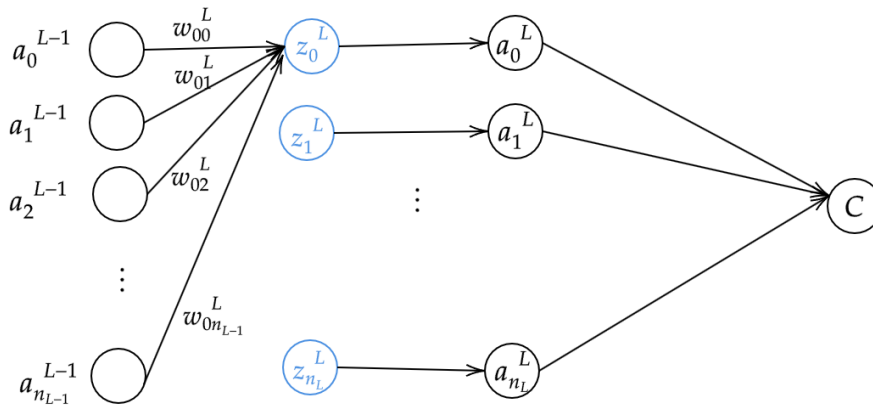


Figura 2-12: Grafo para la ejecución del reverse mode asociado a las ultimas dos capas de una red neuronal, los grafos en azul, son operaciones intermedias, cuando se pasa la información de una capa a la otra, en este caso representan la función de activación.

Ahora note que por la regla de la cadena, si queremos calcular la derivada parcial de $C(a^L(z^L(w^L, a^{L-1}, b^L)))$ respecto a los parámetros de de la última capa, tenemos que ejecutar la regla de la cadena, de la siguiente forma: Primero supongamos que queremos calcular

la derivada parcial de C respecto del primer peso de la capa w^L , es decir, queremos calcular $\frac{\partial C}{\partial w_{00}^L}$, guiándonos por la Figura **2-12** y por la regla de la cadena

$$\frac{\partial C}{\partial w_{00}^L} = \frac{\partial C}{\partial a_0^L} \frac{\partial a_0^L}{\partial z_0^L} \frac{\partial z_0^L}{\partial w_{00}^L},$$

donde $z_0^L = w_{00}^L a_0^{L-1} + w_{01}^L a_1^{L-1} + \dots + w_{0n_{L-1}}^L a_{n_{L-1}}^{L-1} + b_0$, a_0^L es la función de activación para la primera neurona de la capa L , si hiciéramos esto mismo para cada peso que llega a la primera neurona de la capa L , lo siguiente.

$$\frac{\partial C}{\partial w_0^L} = \left[\frac{\partial C}{\partial w_{00}^L}, \frac{\partial C}{\partial w_{01}^L}, \dots, \frac{\partial C}{\partial w_{0n_{L-1}}^L} \right] = \frac{\partial C}{\partial a_0^L} \frac{\partial a_0^L}{\partial z_0^L} \left[\frac{\partial z_0^L}{\partial w_{00}^L}, \frac{\partial z_0^L}{\partial w_{01}^L}, \dots, \frac{\partial z_0^L}{\partial w_{0n_{L-1}}^L} \right].$$

De esta manera, si hacemos la convención de que $\frac{\partial C}{\partial w^L} = \left[\frac{\partial C}{\partial w_0^L}, \dots, \frac{\partial C}{\partial w_{n_L}^L} \right]^T$, y que $*$ es el producto entre matrices pero componente a componente (conocido como producto de Hadamard [6]), llegamos a que

$$\frac{\partial C}{\partial w^L} = \begin{bmatrix} \frac{\partial C}{\partial w_0^L} \\ \vdots \\ \frac{\partial C}{\partial w_{n_L}^L} \end{bmatrix} = \begin{bmatrix} \frac{\partial C}{\partial a_0^L} \\ \vdots \\ \frac{\partial C}{\partial a_{n_L}^L} \end{bmatrix} * \begin{bmatrix} \frac{\partial a_0^L}{\partial z_0^L} \\ \vdots \\ \frac{\partial a_{n_L}^L}{\partial z_{n_L}^L} \end{bmatrix} * \begin{bmatrix} \frac{\partial z_0^L}{\partial w_{00}^L} & \dots & \frac{\partial z_0^L}{\partial w_{n_{L-1}}^L} \\ \vdots & \ddots & \vdots \\ \frac{\partial z_{n_L}^L}{\partial w_{n_L 0}^L} & \dots & \frac{\partial z_{n_L}^L}{\partial w_{n_L n_{L-1}}^L} \end{bmatrix},$$

quedándonos la regla de la cadena usual con el producto de Hadamard [6], de forma análoga podemos obtener la derivada parcial de C con respecto a b^L , es decir

$$\frac{\partial C}{\partial w^L} = \frac{\partial C}{\partial a^L} * \frac{\partial a^L}{\partial z^L} * \frac{\partial z^L}{\partial w^L}$$

y

$$\frac{\partial C}{\partial b^L} = \frac{\partial C}{\partial a^L} * \frac{\partial a^L}{\partial z^L} \frac{\partial z^L}{\partial b^L}.$$

Ahora, notemos que $\frac{\partial z^L}{\partial b^L} = 1$, y que $\frac{\partial w^L}{\partial b^L}$ es la matriz que en cada fila es a^{L-1} , (realmente esta operación es análoga a cuando derivamos una función afín en $\mathbb{R}$), además, llamaremos

$$\delta^L = \frac{\partial C}{\partial a^L} * \frac{\partial a^L}{\partial z^L}.$$

Que nos diga como se comporta el error en la función de costo dependiendo de los parámetros de la ultima capa, de esta forma nos queda la ecuación

$$\frac{\partial C}{\partial w^L} = \delta^L (a^{L-1}) \quad y \quad \frac{\partial C}{\partial b^L} = \delta^L.$$

Ahora hagamos lo mismo pero con la capa $L - 1$, para ello note que nuestra función de costo se ve de la forma

$$C(a^L(w^L a^{L-1}(w^{L-1} a^{L-2} + b^{L-1}) + b^L)),$$

por lo tanto si aplicamos nuevamente la regla de la cadena, obtenemos lo siguiente

$$\begin{aligned}\frac{\partial C}{\partial w^{L-1}} &= \frac{\partial C}{\partial a^L} \frac{\partial a^L}{\partial z^L} \frac{\partial z^L}{\partial a^{L-1}} \frac{\partial a^{L-1}}{\partial z^{L-1}} \frac{\partial z^{L-1}}{\partial w^{L-1}}, \\ \frac{\partial C}{\partial b^{L-1}} &= \frac{\partial C}{\partial a^L} \frac{\partial a^L}{\partial z^L} \frac{\partial z^L}{\partial a^{L-1}} \frac{\partial a^{L-1}}{\partial z^{L-1}} \frac{\partial z^{L-1}}{\partial b^{L-1}}.\end{aligned}$$

Notemos que las dos primeras derivadas de la primera linea corresponden a δ^L que ya habíamos calculado, la tercera derivada corresponde a w^L (como cuando derivamos una recta), la cuarta derivada corresponder a la derivada de la función de activación de la capa $L - 1$, y la ultima derivada corresponde a a^{L-2} en la primera linea y en la segunda linea a la matriz identidad. De esta manera si llamamos

$$\delta^{L-1} = (w^L)^T \delta^L * \frac{\partial a^{L-1}}{\partial z^{L-1}},$$

nos quedan las siguientes ecuaciones

$$\frac{\partial C}{\partial w^{L-1}} = \delta^{L-1} (a^{L-2})^T \quad y \quad \frac{\partial C}{\partial b^{L-1}} = \delta^{L-1}. \quad (2-5)$$

Y esto mismo lo podemos extrapolar a las capas $L - 2, L - 3, \dots, 2$. En 4 mostramos un pseudocódigo de como se podría aplicar este algoritmo. Para mas detalles podemos consultar [13], en donde también se da una implementación en Python.

Algorithm 4 Retropropagación

Require: la función de costo C con todos los parámetros expuestos anteriormente

- 1: $\delta^L = \frac{\partial C}{\partial a^L} * \frac{\partial a^L}{\partial z^L}$
 - 2: $\frac{\partial C}{\partial w^L} = \delta^L (a^{L-1})^T$
 - 3: $\frac{\partial C}{\partial b^L} = \delta^L$
 - 4:
 - 5: **for** $i = 1, 2, \dots, L - 2$ **do**
 - 6: Compute $\delta^{L-i} = w^{L-(i-1)} \delta^{L-(i-1)} * \frac{\partial a^{L-i}}{\partial z^{L-i}}$
 - 7: Compute $\frac{\partial C}{\partial w^{L-i}} = \delta^{L-i} (a^{L-(i+1)})^T$
 - 8: Compute $\frac{\partial C}{\partial w^{L-i}} = \delta^{L-i}$
 - 9: **end for**
 - 10: Devolver todas las derivadas parciales de C con respecto a w^i y b^i para $i = 2, 3, \dots, L$
-

2.2. Algunos conceptos de probabilidad

En gran parte de los modelos generativos actuales se utilizan herramientas de probabilidad [4, 7], dado que en nuestro interés esta estudiarlos pues hace parte del método Ppower expuesto en el Capitulo 4, introducimos esta sección dedicada a repasar los conceptos básicos y necesarios para poder llegar a entender lo que son y como funcionan estos modelos, principalmente esta Sección esta basada en el libro de probabilidad de Liliana Blanco Castañeda [15] y en la notas de clase de Stefano Ermon [9].

2.2.1. Probabilidad básica.

A continuación daremos unas definiciones que nos ayudaran a tener una base solida sobre probabilidad, si se quiere tener una visión mas detallada de estos puede consultar [15], de donde se sacaron gran parte de estas definiciones y teoremas, también se puede consultar [9] para contrastar.

Definición 2.2.1. *Diremos que un experimento es aleatorio si su resultado no puede ser determinado de antemano. Al conjunto Ω de todos los posibles resultados de un experimento aleatorio los llamaremos espacio muestral y a los elementos $\omega \in \Omega$ los llamaremos puntos muestrales. Si Ω es finito o numerable lo llamaremos un espacio muestral discreto.*

Por ejemplo suponga que tenemos el experimento de lanzar una moneda corriente y ver que lado de la moneda queda en la parte superior, cuando esta caiga al suelo. Los posibles resultados, en este caso son cara c , y sello s . Esto es $\Omega = \{c, s\}$ es el espacio muestral (en este caso discreto) y c y s son los puntos muestrales de este.

La idea ahora es definir cierto tipo de conjuntos a los cuales se les pueda asignar un “chance” de aparecer, pues estamos dirigiéndonos hacia el concepto probabilidad y queremos tener objetos a los cuales podemos asignarles un valor numérico [4], podríamos pensar simplemente en tomarlo como un subconjunto del espacio muestral Ω , pero resulta, que en algunos casos esto no siempre se puede, es decir, no siempre todos los subconjuntos de un espacio muestral tienen el chance de aparecer como se ve en [9]. La clase de subconjuntos del espacio muestral para los que esta definido este chance, es llamado σ -álgebra sobre Ω , y lo definimos a continuación.

Definición 2.2.2. *Sea $\Omega \neq \emptyset$. Diremos que una colección F de subconjuntos de Ω es una σ -álgebra, sobre Ω , sí:*

- i) $\Omega \in F$.
- ii) Si $A \in F$ entonces $A^c \in F$.
- iii) Si $A_1, A_2, \dots \in F$, entonces $\bigcup_{i=1}^{\infty} A_i \in F$.

A los elementos de F los llamaremos eventos, y a la pareja (Ω, F) le llamaremos espacio medible.

Un ejemplo reconocido de la definición anterior, es llamada la σ -álgebra de borel $\mathcal{B}$ sobre $\mathbb{R}$, que es el conjunto mínimo que se puede formar con los intervalos de la forma $(-\infty, a]$ con $a \in \mathbb{R}$, utilizando las reglas anteriores, como se ve en [15], este espacio no contiene a todos los subconjuntos de $\mathbb{R}$, como se podría llegar a pensar, por lo que no todos los conjuntos de $\mathbb{R}$ van a tener un “chance” de aparecer. De forma análoga se puede definir la σ -álgebra de borel $\mathcal{B}$ de $\mathbb{R}^n$ Pero esta vez utilizando cubos n dimensionales [15]. Ahora formalicemos esta idea de “chance” que hemos tratado anteriormente.

Definición 2.2.3. Sea (Ω, F) un espacio medible. Una función P definida sobre F y de valor real que satisface las siguientes propiedades

- i) $P(A) \geq 0$ para todo $A \in F$.
- ii) $P(\Omega) = 1$.
- iii) Si $A_1, A_2, \dots$ son elementos de F mutuamente excluyentes, entonces

$$P\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P(A_i).$$

la llamaremos medida de probabilidad sobre (Ω, F) , y a la tripla (Ω, F, P) la llamaremos espacio de probabilidad.

Por ejemplo suponga que se lanza una moneda 100 veces, en este caso el espacio muestral es $\Omega = \{c, s\}^{100}$, y la sigma álgebra es $F = \mathcal{P}(\Omega)$. En este caso la probabilidad de cualquier evento unitario (subconjunto con un solo elemento de F) de F puede ser definida como

$$P(E) = \frac{1}{2^{100}}.$$

Y dado que F es un conjunto finito se puede ver por la definición anterior que a todos los eventos se les puede asignar una probabilidad con esta definición sobre los elementos unitarios de F . Así (Ω, F, P) es un espacio de probabilidad.

Definición 2.2.4. Sea (Ω, F, P) un espacio de probabilidad. Si $A, B \in F$, con $P(A) > 0$, entonces definimos la probabilidad del evento B bajo la condición del evento A como sigue

$$P(B|A) := \frac{P(A \cap B)}{P(A)}.$$

Leemos $P(B|A)$ como la probabilidad de que suceda B dado que ha ocurrido A .

Teorema 2.2.1. (Regla de Bayes) Sean $A_1, A_2, \dots$ una partición finita o numerable de Ω , con $P(A_i) > 0$ para todo i , entonces se satisface para todo $B \in F$ con $P(B) > 0$ lo siguiente

$$P(A_i|B) = \frac{P(A_i)P(B|A_i)}{P(B)} = \frac{P(A_i)P(B|A_i)}{\sum_j P(B|A_j)P(A_j)}. \quad (2-6)$$

Para dar una interpretación a la regla de Bayes, supóngase que los eventos $A_1, A_2, \dots$ son todas las posibles causas, mutuamente excluyentes, de un evento B . Bajo el supuesto que el evento B ha sido observado, la fórmula de Bayes permite conocer cuál de estas causas es la más probable de haber producido el evento B . Para mas profundización acerca de la regla de Bayes puede consultar [15, 16].

Definición 2.2.5. Sea $A_1, A_2, \dots$ una partición finita o numerable de Ω , con $P(A_i) > 0$ para todo i . Si B es un elemento de F , con $P(B) > 0$, entonces a $\{P(B|A_n)\}_n$ le llamaremos *distribución a priori*, esto es antes que suceda B . y a $\{P(A_n|B)\}_n$ le llamaremos *distribución a posteriori*, es decir después de que suceda B .

En un experimento aleatorio hay frecuentemente más interés por ciertos valores numéricos que se pueden deducir de los resultados de un experimento aleatorio que por los resultados en sí [15]. Supóngase por ejemplo, que se lanza una moneda corriente seis veces consecutivas y que interesa conocer el número de caras obtenidas. En este caso se tiene que el espacio muestral es igual a.

$$\Omega = \{(a_1, a_2, a_3, a_4, a_5, a_6) : a_i \in \{C, S\}\}$$

Si se define $X :=$ "Número de caras obtenidas", entonces se tiene que X es una aplicación de Ω , en Ω^* , con $\Omega^* = \{1, 2, \dots, 6\}$. Así por ejemplo, se tiene que $X((C, C, S, C, C, S)) = 4$. La aplicación X es un ejemplo de variable aleatoria, concepto que se precisa a continuación y que puede ser buscado también en [15].

Definición 2.2.6. Sean (Ω, F, P) un espacio de probabilidad, y (Ω^*, F^*) , un espacio medible. Una $F - F^*$ - variable aleatoria es una aplicación $X : \Omega \rightarrow \Omega^*$ tal que para todo $A \in F^*$ se tiene que $X^{-1}(A) \in F$. Si $(\Omega^*, F^*) = (\mathbb{R}, \mathcal{B})$, entonces diremos que X es una variable aleatoria real.

Para lo que sigue se trabajara con variables aleatorias reales, o generalizaciones de esta a $\mathbb{R}^n$, es decir con F^* , la σ -álgebra de Borel en $\mathbb{R}^n$.

Definición 2.2.7. Sea X una variable aleatoria real. La función F_X definida sobre $\mathbb{R}$ por medio de

$$F_X(x) := P_X((-\infty, x)) = P(X \leq x)$$

la llamaremos *distribución de la variable aleatoria X* [15].

Ahora para una variable aleatoria discreta podemos definir la *función de probabilidad* como $f_X(x) = P(X = x)$ [15], Análogamente cuando una variable aleatoria cumple que existe una función no negativa tal que

$$P(X \in A) = \int_A f(x)dx.$$

Para todo evento A , se dice que esta tiene una distribución continua, y la función f se denomina *función de densidad de probabilidad* (f.d.p) [15]. Una función de densidad de probabilidad muy conocida y que vamos a trabajar bastante es la distribución normal, cuya f.d.p, esta dada por

$$f_X(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \quad (2-7)$$

para el caso real, donde σ es un parámetro que mas tarde conoceremos como desviación estándar y μ la media [15]. Para $\mathbb{R}^n$, toma la forma

$$f_X(x) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)} \quad (2-8)$$

Donde $\mu = [\mu_1 \ \dots \ \mu_n]^T$, y Σ es una matriz $n \times n$, que harían el papel de μ , y σ^2 respectivamente [15].

Sea X una variable aleatoria real. Se sabe que sus propiedades de tipo probabilístico están determinadas por su función de distribución $F_X(x)$ [15]. Sin embargo, interesa conocer un par de valores que resuman.^{en} cierta forma dicha información. Por ejemplo, se quiere definir un "promedio" de los valores asumidos por X y una cantidad que "mida" que tanto varían los valores de X con respecto a ese valor [15].

Definición 2.2.8. Sea X una variable aleatoria real definida sobre el espacio de probabilidad (Ω, F, P)

i) Si X es una variable aleatoria discreta con valores $x_1, x_2, \dots$ se dice que X posee un valor esperado si

$$\sum_{k=1}^{\infty} |x_k| P(X = x_k) < \infty,$$

en tal caso se define el valor esperado $\mathbb{E}(X)$ como

$$\mathbb{E}(X) = \sum_{k=1}^{\infty} x_k P(X = x_k).$$

ii) Si X es una variable aleatoria continua con función de densidad f_X , se dice que X posee un valor esperado si

$$\int_{-\infty}^{\infty} |x| f_X(x) dx < \infty,$$

en tal caso se define el valor esperado $\mathbb{E}(X)$ como

$$\mathbb{E}(X) = \int_{-\infty}^{\infty} x f_X(x) dx.$$

Ahora podemos combinar dos distribuciones [16], de la siguiente forma, si X , tiene como f.d.p $Q(x)$, se puede calcular la esperanza con respecto a la distribución $P(x)$ en el caso continuo como

$$\mathbb{E}_{\sim P}[Q(x)] = \int_{-\infty}^{\infty} P(x)Q(x)dx, \quad (2-9)$$

análogamente se puede calcular esto en el caso discreto [16] pero tomando la suma sobre todos los valores en vez de la integral.

Otro concepto que ya hemos nombrado en las ecuaciones (2-7) y (2-11) y no hemos ahondado es el concepto de Varianza y desviación estándar [15]. Suponga que X es una variable aleatoria con esperanza μ , la varianza de esta variable, se denota por $Var(X)$, y se calcula por medio de la formula

$$Var(X) = \mathbb{E}[(X - \mu)^2],$$

generalmente esta varianza se denota por σ^2 , y la desviación estándar se denota por σ , pues para el caso de que X caiga en $\mathbb{R}$, la desviación estándar se define como la raíz cuadrada de la varianza [15].

También podemos relacionar dos o mas variables aleatorias ,por medio de los conceptos de *covarianza* y *matriz de covarianza* [15]. Si X y Y son dos variables aleatorias, entonces su covarianza se define mediante

$$Cov(X, Y) = \mathbb{E}((X - \mu_X)(Y - \mu_Y)). \quad (2-10)$$

A partir de la ecuación (2-12), podemos obtener una matriz que nos de la relación de diversas variables. para ello suponga que $X_1, X_2, \dots, X_n$, son variables aleatorias. La matriz $\Sigma = [\xi_{ij}]$, donde $\xi_{ij} = Cov(X_i, X_j)$, es llamada matriz de covarianza de la variable aleatoria $X = (X_1, \dots, X_n)$ [15].

2.2.2. Entropía

La teoría de la información estudia cuantitativamente la transmisión, almacenamiento y procesamiento de la información [17]. Uno de los conceptos clave en la teoría de la información es la *Entropía*, que mide la cantidad de incertidumbre asociada con una distribución de probabilidad, y cuanta sorpresa hay en la ocurrencia de eventos [17]. se suele utilizar porque proporciona una medida cuantitativa de la información contenida en un conjunto de datos.

Definición 2.2.9. Sea P una distribución sobre el conjunto de datos A , entonces definimos la entropía, mediante la ecuación

$$H(P) = - \sum_{x \in A} P(x) \log_2(P(x)) \quad (2-11)$$

o para el caso continuo

$$H(P) = - \int_A P(x) \log_2(p(x)) dx. \quad (2-12)$$

Ahora expliquemos de donde sale cada termino de las ecuaciones (2-11), (2-12) de la Definición 2.2.9 y que significan. Primero es claro que estamos tratando de sacar un promedio ponderado al multiplicar por la probabilidad de cada evento, por otro lado notemos que el segundo termino se puede ver como

$$-\log_2(P(x_i)) = \log_2(P(x_i)^{-1}).$$

Viéndolo de esta manera es mas fácil de interpretar. Para ello nos basaremos en la hipótesis de teoría de la información que dicta que eventos mas probables se representan con menos bits que eventos menos probables [17]. De esta forma $\log_2(P(x_i)^{-1})$ modela la cantidad de bits necesarios para representar un vector x_i dada su probabilidad de aparecer. Si x_i es un evento muy probable $P(x_i)^{-1}$ sera pequeño y por tanto su logaritmo sera también pequeño, y por el contrario si x_i es un evento poco probable $P(x_i)^{-1}$ será grande y por tanto su logaritmo también será grande. La introducción del logaritmo en base 2 se hace por dos cuestiones, la primera es para reescalar la información para que sea mas manejable, algo que es común en esta rama [17] y la segunda es que en computación los bits que se necesitan para representar determinado cantidad se encuentran en el exponente de un 2 (recordemos que si un numero entero es mas grande, en base 2 necesita generalmente de mas bits) [17], note además que esta definición de entropía se puede variar un poco al cambiarle la base en la que se expresa la información si es base 10 se mediría en dígitos decimales, y si se medie en base e la entropía se mediría en nats [17] (unidades naturales de información).

Uniéndolo todo lo que nos dice la definición de entropía (2-11), (2-12) es simplemente un promedio de la cantidad de bits necesarios para representar eficientemente los eventos de una distribución [17]. Ahora notemos que la función de entropía se ve como la Figura **2-13** para un solo dato con probabilidad x variando entre 0 y 1.

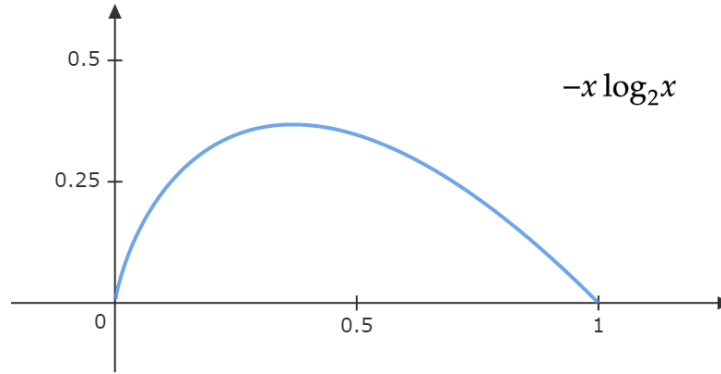


Figura 2-13: Gráfica de $-x \log_2 x$, como se ve cerca al centro es máximo y en los extremos es 0, lo cual concuerda con la noción expuesta de entropía

Notemos entonces que cuando la probabilidad de un dato esta cerca de 0.5, la entropía es mas grande que cuando esta cerca de 0 o 1 (esto tambien se tiene para varios datos [17, 7]), luego si hacemos una suma sobre estas probabilidades, cuando tengamos probabilidades muy altas para todos los datos, la entropía sera alta en comparación cuando las probabilidades de los datos sean mas cercanas a 0 o 1. De esta forma la entropía también se puede interpretar como la cantidad de sorpresa o incertidumbre en una distribución, ya que cuando los eventos de una distribución son equiprobables (es decir no sabemos cual de todos va a pasar) los datos tendrán una probabilidad alta en comparación a otras distribuciones y por tanto su entropía sera mayor. Ilustremos esto con un ejemplo.

Supongamos que tenemos la distribución uniforme P_1 , con eventos x_1, x_2, x_3, x_4 , tal que $P_1(x_i) = \frac{1}{4}$, por otro lado suponga que tenemos otra distribución P_2 que a los mismos eventos les asigna las probabilidades 0,7, 0,1, 0,1, 0,1 respectivamente. Si nosotros escogemos un evento siguiendo la distribución P_1 , tenderíamos bastante incertidumbre del evento que saldrá, en cambio si nosotros escogemos un evento de estos cuatro siguiente de la distribución P_2 , tendríamos poca incertidumbre del evento que saldrá, pues es muy probable que salga x_1 . Esto es lo que intenta codificar la entropía, y se ve de la siguiente manera

$$H(P_1) = -4 \cdot \frac{1}{4} \cdot \log_2(0,25) = 2$$

y para P_2 ,

$$H(P_2) = -(0,7 \cdot \log_2(0,7) + 0,1 \cdot \log_2(0,1) + 0,1 \cdot \log_2(0,1) + 0,1 \cdot \log_2(0,1)) = 1,356.$$

De esta forma la distribución uniforme que mostramos al principio tiene mas incertidumbre o sorpresa.

Ahora veamos como se pueden mezclar dos distribuciones y obtener una especie de relación o “distancia” entre ella utilizando las ideas de entropía que vimos anteriormente (ver [7, 17]).

Definición 2.2.10. *La entropía cruzada entre dos distribuciones P y Q sobre el mismo conjunto de datos se define mediante la fórmula*

$$H(P, Q) = -\mathbb{E}_{\sim P}[\log_2 Q] \quad (2-13)$$

La anterior fórmula se convierte en

$$H(P, Q) = -\mathbb{E}_{\sim P}[\log_2 Q] = -\sum_{x \in X} P(x) \log_2 Q(x) \quad (2-14)$$

y para el caso continuo se convierte en

$$H(P, Q) = -\mathbb{E}_{\sim P}[\log_2 Q] = -\int_{x \in X} P(x) \log_2 Q(x) dx \quad (2-15)$$

Haciendo un análisis análogo, al que se hizo con la entropía, se puede deducir una interpretación de esta fórmula [17]. La entropía cruzada mide la cantidad promedio de bits necesarios para representar eventos de P , cuando usamos un código basado en Q [17]. Generalmente esta medida se utiliza cuando P es la distribución real de los datos y Q es una probabilidad estimada de los datos [17].

Con la información que adquirimos en cuanto a entropía de la Subsección 2.2.2, podemos ahora sí, introducir lo que es la divergencia KL (ver [7]), entre dos distribuciones P y Q .

Definición 2.2.11. *La entropía relativa o la divergencia Kullback-Leibler (D_{KL}), entre dos distribuciones P y Q definidas sobre el mismo conjunto de datos, esta definida mediante la fórmula*

$$D_{KL}(P||Q) = H(P, Q) - H(P), \quad (2-16)$$

donde $H(P, Q)$ y $H(P)$ son la entropía cruzada y la entropía definidas respectivamente por la ecuación 2-11 y la Definición 2.2.9.

De esta manera la entropía relativa D_{KL} puede interpretarse como la longitud adicional esperada del mensaje por dato que debe comunicarse si se usa un código (conjunto de números que representa un mensaje) que es óptimo para una distribución Q dada, en comparación con el uso de un código basado en la distribución P [7], para el caso discreto tenemos lo siguiente

$$D_{KL}(P||Q) = H(P, Q) - H(P) \quad (2-17)$$

$$= -\sum_{x \in X} P(x) \cdot \log_2(Q(x)) + \sum_{x \in X} P(x) \cdot \log_2(P(x)) \quad (2-18)$$

$$= \sum_{x \in X} P(x) \cdot \log_2 \left(\frac{P(x)}{Q(x)} \right). \quad (2-19)$$

Para el caso continuo es análogo y se llega a

$$D_{KL}(P||Q) = \int_A P(x) \log_2 \left(\frac{P(x)}{Q(x)} \right) dx. \quad (2-20)$$

En general esta definición puede variar dependiendo del espacio medible que estemos utilizando y su medida μ , pero para los casos en los que nos interesaremos, utilizaremos la medida estándar (medida de lebesgue) de $\mathbb{R}^n$, además muchas veces en la definición se puede utilizar otro base en el logaritmo, pero la esencia es la misma [7]. Para lo que sigue utilizaremos la definición dada en la ecuación (2-20) pero con el logaritmo natural en vez del logaritmo en base 2, eso por algunas simplificaciones que nos dará con las exponenciales de las distribuciones de probabilidad Gaussianas.

Ahora presentamos algunas propiedades que tiene esta medida, estas propiedades están principalmente enfocadas en lo que necesitaremos en adelante acerca de esta medida, desde ahora se adelanta que D_{KL} , Es casi una métrica, en el espacio de las distribuciones, salvo que esta es asimétrica, a esta tipo de medidas se les conoce como divergencia (ver [17, 7]).

Teorema 2.2.2. *Suponga que P y Q son dos distribuciones cualesquiera sobre $\mathbb{R}^k$, entonces se cumplen las siguientes afirmaciones.*

i) *Siempre se tiene que $D_{KL}(P||Q) \geq 0$.*

ii) *La entropía relativa D_{KL} es aditiva para distribuciones independientes, es decir, si $P(x, y) = P_1(x)P_2(y)$, $Q(x, y) = Q_1(x)Q_2(y)$ para todo x, y , entonces*

$$D_{KL}(P||Q) = D_{KL}(P_1||Q_1) + D_{KL}(P_2||Q_2). \quad (2-21)$$

iii) *Supongamos que tenemos dos distribuciones normales multivariadas, con medias μ_0, μ_1 y con matrices de covarianza Σ_0, Σ_1 , si las dos distribuciones tiene la misma dimensión k , entonces la entropía relativa entre las dos distribuciones es la siguiente*

$$D_{KL}(P_0||P_1) = \frac{1}{2}(tr(\Sigma_1^{-1}\Sigma_0)) - k + (\mu_1 - \mu_0)^T \Sigma_1^{-1}(\mu_1 - \mu_0) + \ln \left(\frac{|\Sigma_1|}{|\Sigma_0|} \right). \quad (2-22)$$

Otras ecuaciones que se deducen de la ecuación (2-22) son las siguientes

$$D_{KL}(\mathcal{N}((\mu_1, \dots, \mu_k)^T, diag(\sigma_1^2, \dots, \sigma_k^2))||\mathcal{N}(0, I)) = \frac{1}{2} \sum_{i=1}^k (\sigma_i^2 + \mu_i^2 - 1 - \ln(\sigma_i^2)) \quad (2-23)$$

y si p_0 y p_1 son distribuciones normales univariadas

$$D_{KL}(p_0||p_1) = \log \left(\frac{\sigma_1}{\sigma_0} \right) + \frac{\sigma_0^2 + (\mu_0 - \mu_1)^2}{2\sigma_1^2} - \frac{1}{2} \quad (2-24)$$

Demostración. i) Para demostrar esta propiedad utilizaremos el hecho de que $\ln x \leq x - 1$, luego tenemos lo siguiente

$$\begin{aligned}
-D_{KL}(P||Q) &= - \int_{\mathbb{R}^n} P(x) \ln \left(\frac{P(x)}{Q(x)} \right) dx \\
&= \int_{\mathbb{R}^n} P(x) \ln \left(\frac{Q(x)}{P(x)} \right) dx \\
&\leq \int_{\mathbb{R}^n} P(x) \left(\frac{Q(x)}{P(x)} - 1 \right) dx \\
&= \int_{\mathbb{R}^n} Q(x) dx - \int_A P(x) dx \\
&= 1 - 1 \\
&= 0.
\end{aligned}$$

en la primera desigualdad utilizamos el hecho que nombramos anteriormente, y en la tercera igualdad utilizamos el hecho de que Q y P son distribuciones sobre $\mathbb{R}^n$ y por tanto su integral sobre este conjunto es 1. Este mismo resultado lo podemos demostrar de forma análoga para el caso discreto. Además en esta propiedad se puede ver que $D_{KL}(P||Q)$ si y solo si $P = Q$ en casi toda parte, pero esta demostración es un poco mas compleja (la implicación de izquierda a derecha) y se puede consultar en [17]

ii) Para este item tomaremos la convención de que $x \in \mathbb{R}^l$ y $y \in \mathbb{R}^{n-l}$, luego

$$\begin{aligned}
D_{KL}(P||Q) &= \int_{\mathbb{R}^n} P(x, y) \ln \left(\frac{P(x, y)}{Q(x, y)} \right) dx dy \\
&= \int_{\mathbb{R}^n} P_1(x) P_2(y) \ln \left(\frac{P_1(x) P_2(y)}{Q_1(x) Q_2(y)} \right) dx dy
\end{aligned}$$

utilizando la propiedad aditiva del logaritmo y distribuyendo a $P_1(x) P_2(y)$, obtenemos que lo anterior es

$$\begin{aligned}
&= \int_{\mathbb{R}^n} P_1(x) P_2(y) \ln \left(\frac{P_1(x)}{Q_1(x)} \right) dx dy + \int_{\mathbb{R}^n} P_1(x) P_2(y) \ln \left(\frac{P_2(y)}{Q_2(y)} \right) dx dy \\
&= \int_{\mathbb{R}^{n-l}} P_2(y) dy \int_{\mathbb{R}^l} P_1(x) \ln \left(\frac{P_1(x)}{Q_1(x)} \right) dx + \int_{\mathbb{R}^n} P_1(x) dx \int_{\mathbb{R}^l} P_2(y) \ln \left(\frac{P_2(y)}{Q_2(y)} \right) dy \\
&= D_{KL}(P_1||Q_1) + D_{KL}(P_2||Q_2).
\end{aligned}$$

En la segunda igualdad utilizamos que $P_2(y)$ es constante respecto a la variable x , al igual que $P_1(x)$ es constante respecto a la variable y , además en la tercera igualdad utilizamos que P_2 y P_1 son distribuciones sobre $\mathbb{R}^l$ y $\mathbb{R}^{n-l}$ respectivamente y por tanto su integral es 1.

iii) Para tenerlo un poco mas claro en este caso estamos suponiendo que

$$P_0(x) = \frac{1}{(2\pi)^{n/2}|\Sigma_0|^{1/2}} e^{-\frac{1}{2}(x-\mu_0)^T \Sigma_0^{-1}(x-\mu_0)} \quad P_1(x) = \frac{1}{(2\pi)^{n/2}|\Sigma_1|^{1/2}} e^{-\frac{1}{2}(x-\mu_1)^T \Sigma_1^{-1}(x-\mu_1)}$$

luego cuando pongamos estas distribuciones en el logaritmo natural que aparece en la definición de la divercia KL, veremos que la exponencial desaparece y se dan algunas cancelaciones como pasa a continuación

$$\begin{aligned} D_{KL}(P_0||P_1) &= \int_{\mathbb{R}^n} P_0(x) \cdot \ln \left(\frac{|\Sigma_1|^{1/2}}{|\Sigma_0|^{1/2}} e^{-\frac{1}{2}(x-\mu_0)^T \Sigma_0^{-1}(x-\mu_0) + \frac{1}{2}(x-\mu_1)^T \Sigma_1^{-1}(x-\mu_1)} \right) dx \\ &= \int_{\mathbb{R}^n} \frac{1}{2} \ln \left(\frac{|\Sigma_1|}{|\Sigma_0|} \right) - \frac{1}{2}(x-\mu_0)^T \Sigma_0^{-1}(x-\mu_0) + \\ &\quad + \frac{1}{2}(x-\mu_1)^T \Sigma_1^{-1}(x-\mu_1) \cdot P_0(x) dx \end{aligned}$$

Ahora utilizando la notación de la (2-9), el hecho de que $\text{tr}(ABC) = \text{tr}(BCA)$ y que la esperanza es aditiva llegamos a las siguientes igualdades

$$\begin{aligned} &= \frac{1}{2} \mathbb{E}_{\sim P_0(x)} \left[\ln \left(\frac{|\Sigma_1|}{|\Sigma_0|} \right) - (x-\mu_0)^T \Sigma_0^{-1}(x-\mu_0) + (x-\mu_1)^T \Sigma_1^{-1}(x-\mu_1) \right] \\ &= \frac{1}{2} \mathbb{E}_{\sim P_0(x)} \left[\ln \left(\frac{|\Sigma_1|}{|\Sigma_0|} \right) - \text{tr} [\Sigma_0^{-1}(x-\mu_0)(x-\mu_0)^T] + \text{tr} [\Sigma_1^{-1}(x-\mu_1)(x-\mu_1)^T] \right] \\ &= \frac{1}{2} \ln \left(\frac{|\Sigma_1|}{|\Sigma_0|} \right) - \text{tr} [\Sigma_0^{-1} \mathbb{E}_{\sim P_0(x)} [(x-\mu_0)(x-\mu_0)^T]] + \\ &\quad + \text{tr} [\Sigma_1^{-1} \mathbb{E}_{\sim P_0(x)} [(x-\mu_1)(x-\mu_1)^T]] , \end{aligned}$$

ahora podemos expandir $(x-\mu_1)(x-\mu_1)^T$, quedándonos $xx^T + 2\mu_1 x^T + \mu_1 \mu_1^T$, luego repartiendo la esperanza en cada uno de estos términos y utilizando el hecho de que $\mathbb{E}_{\sim P(x)}[Ax] = A\mu$ y $\mathbb{E}_{\sim P(x)}[x^T Ax] = \mu^T A\mu + \text{tr}(A\Sigma)$, cuando P es una distribución normal con media μ y matriz de covarianza Σ (ver [18]), nos queda

$$\begin{aligned} D_{KL}(P_0||P_1) &= \frac{1}{2} \left(\ln \left(\frac{|\Sigma_1|}{|\Sigma_0|} \right) - \text{tr}[\Sigma_0^{-1}\Sigma_0] + \text{tr}[\Sigma_1^{-1}(\Sigma_0 + \mu_0\mu_0^T - 2\mu_1\mu_0^T + \mu_1\mu_1^T)] \right) \\ &= \frac{1}{2} \left(\ln \left(\frac{|\Sigma_1|}{|\Sigma_0|} \right) - \text{tr}[I_n] + \text{tr}[\Sigma_1^{-1}\Sigma_0] + \text{tr}[\Sigma_1^{-1}(\mu_0\mu_0^T - 2\mu_1\mu_0^T + \mu_1\mu_1^T)] \right) \\ &= \frac{1}{2} \left(\ln \left(\frac{|\Sigma_1|}{|\Sigma_0|} \right) - n + \text{tr}[\Sigma_1^{-1}\Sigma_0] + \text{tr}[\mu_0\Sigma_1^{-1}\mu_0^T - 2\mu_1\Sigma_1^{-1}\mu_0^T + \right. \\ &\quad \left. + \mu_1\Sigma_1^{-1}\mu_1^T] \right) \\ &= \frac{1}{2} \left(\ln \left(\frac{|\Sigma_1|}{|\Sigma_0|} \right) - n + \text{tr}[\Sigma_1^{-1}\Sigma_0] + (\mu_1 - \mu_0)^T \Sigma_1^{-1}(\mu_1 - \mu_0) \right) \end{aligned}$$

Note que en la primera igualdad se utilizaron las propiedades nombradas en el anterior párrafo para el segundo y tercer termino, además en la cuarta igualdad se utilizo que la traza de una constante es ella misma y se factorizó la expreción como $(\mu_1 - \mu_0)^T \Sigma_1^{-1} (\mu_1 - \mu_0)$, de esta manera reorganizando los términos de la ultima igualdad nos queda demostrada la ecuación (2-22), note además que las ecuaciones (2-23) y (2-24) se obtienen de esta misma reemplazando las correspondientes distribuciones. $\square$

Ahora un propiedad que no tenemos es la simetría, es decir, no siempre tenemos que $D_{KL}(P||Q) = D_{KL}(Q||P)$, para ver esto ilustremos el siguiente ejemplo. Suponga que tenemos dos distribuciones P y Q con probabilidades para los eventos $\{0, 1, 2\}$ $9/25, 12/25, 4/25$ y $1/3, 1/3, 1/3$ respectivamente, entonces al calcular la entropía relativa D_{KL} ,

$$D_{KL}(P||Q) = \frac{1}{25}(42 \ln(2) + 55 \ln(3) - 50 \ln(5)) \approx 0,082599,$$

por otro lado

$$D_{KL}(Q||P) = \frac{1}{3}(-4 \ln(2) - 6 \ln(3) + 6 \ln(5)) \approx 0,097455.$$

Y por tanto no tenemos para este caso particular. Otro ejemplo es el que aparece en [7], suponga que tenemos una distribución normal bimodal p , y buscamos en las distribuciones normales unimodales, alguna distribución q^* tal que $D_{KL}(p||q^*)$ y $D_{KL}(q^*||p)$ sean mínimos. lo que se encuentra es lo siguiente:

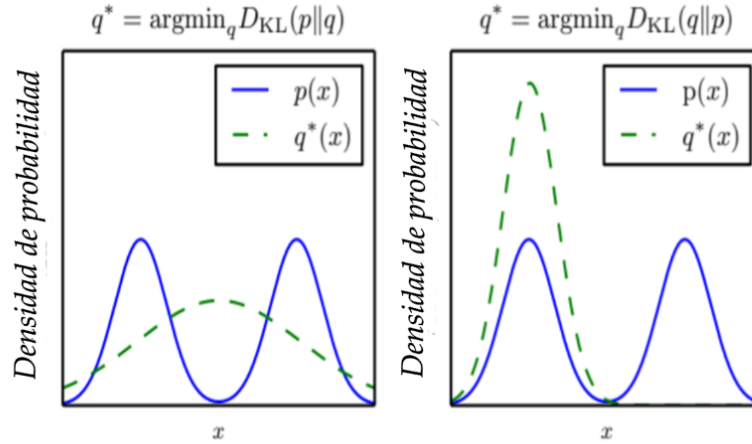


Figura 2-14: Aproximaciones de la distribución p , por medio de la divergencia KL , en la de la izquierda vemos que cuando q se coloca a la derecha y empezamos a minimizar $D_{KL}(p||q)$ la aproximación que tenderemos es una distribución que busca la media, en cambio si aproximamos por la izquierda vemos que la aproximación busca algún modo de la distribución como se ve en la figura de la derecha. (Figura tomada de [7]).

De esta manera se nota la asimetría de D_{KL} . Como vemos en el primer gráfico de **2-14** se ve como que cuando la q esta a la derecha y se minimiza D_{KL} , esta priorizara encontrar la media de los datos. en cambio cuando esta a la izquierda, esta minimización busca encontrar el modo de la distribución, es decir encontrar en donde están mayormente concentrados los datos. En la siguiente sección utilizaremos la divergencia KL para optimizar la búsqueda de parámetros, para ciertas distribuciones, la divergencia jugara un papel muy importante en los que sigue dado que es la forma en la que mediremos el error de ahora en adelante en los modelos que entrenaremos.

2.2.3. Inferencia variacional

Ahora nos disponemos a entender que es la inferencia variacional, para ello primero necesitamos saber que es la inferencia estadística, recordemos que la inferencia estadística, hace referencia hacer deducciones, afirmaciones o conclusiones acerca de una población basándose en la información recopilada de una muestra aleatoria de la misma [9, 15]. Por ejemplo suponga que queremos saber cual es la distribución de las alturas de las personas en cierto país, lo que se podría hacer es tomar una *muestra aleatoria* de un tamaño considerable de la población y hacer un diagrama de frecuencias, como el mostrado en **2-15**.

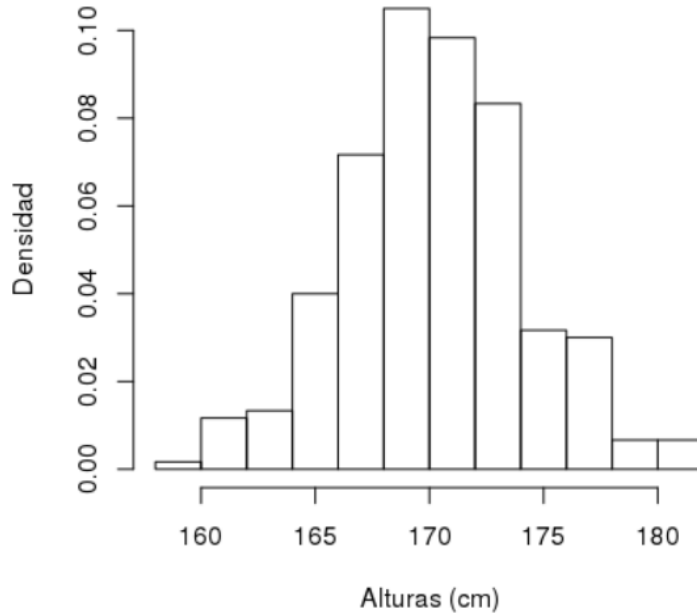


Figura 2-15: *Histograma de frecuencias asociada a la medición de alturas de una muestra de una población. Como se ve la mayoría de las personas cae el rango de altura entre 165 y 175 cm, y datos mas extremos como personas de 160 o mas de 180 tienen frecuencias muy bajas. Imagen tomada de [8]*

Mediante este histograma de frecuencias podremos ajustar alguna distribución que conozcamos (es decir una forma matemáticamente cerrada para poder tratar con ella), por ejemplo para este caso parece ser que las alturas se distribuyen de manera normal, por lo que se infiere que las parámetros de la distribución son alguna media μ , y alguna desviación estándar σ (en este caso $\mu \approx 170$ cm, y $\sigma \approx 5$ cm), y la forma matemática de su distribución es (ver [4])

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

a partir de esta función podemos hacer predicciones o *inferir* ciertas cualidades de la población [4, 9], como por ejemplo que porcentaje de personas en ese país miden entre 180 cm y

190 cm, aun si saber exactamente cuantas personas hay de esa estatura. En general este es el trabajo de la inferencia, es decir, inferir a partir de muestras, parámetros que den lugar a distribuciones conocidas, que se acerquen de manera aproximada a la realidad. y a partir de esto hacer suposiciones/inferir cualidades de esta.

Ahora que ya tenemos una noción de lo que es la *inferencia estadística*, procedemos a entender que es la *inferencia variacional* (mas detalles en [9, 8]), para ello nos basaremos en un ejemplo. Suponga que Tenemos un conjunto de imágenes, y X representa la variable aleatoria que cada imagen le asigna un vector en $\mathbb{R}^N$, donde N representa la cantidad de píxeles de esta imagen, y suponga que además tenemos otra variable aleatoria Z en $\mathbb{R}^k$ (donde k va a ser la cantidad de característica que vamos a considerar) que codifica ciertas cualidades de estas imágenes, por ejemplo la iluminación, ángulo, objetos en ciertas posiciones, etc, Suponga que nuestra tarea ahora es conocer $P(Z|X = D)$, es decir, cual es la probabilidad de tener ciertas características, dado que tenemos un conjunto de datos D , suponga además que tenemos dada la distribución conjunta entre estas dos variables (esto generalmente es conocido.)

Sabemos por la regla de Bayes ([4]) que

$$P(Z|X = D) = \frac{P(X = D|Z)P(Z)}{P(X = D)} = \frac{P(X = D, Z)}{P(X = D)}.$$

Despues veremos que este problema de encontrar $P(Z|X = D)$ es común cuando estamos en aprendizaje automático en los VAE [19], pues queremos encontrar una manera de identificar patrones en nuestro conjunto de datos [19]. Por las condiciones del problema conocemos $P(X = D, Z)$, pero nos falta conocer como se comparta $P(X = D)$, En general esto no es fácil, pues nos toca calcular

$$P(X = D) = \int_{z_1} \int_{z_2} \cdots \int_{z_k} P(X = D, Z) dz_k \cdots dz_2 dz_1,$$

que resulta computacionalmente costosa e intratable en algunos casos como veremos más adelante [8]. Es por esto que la inferencia variacional hace el uso de aproximar a la distribución $P(Z|X = D)$ por medio de la divergencia KL definida en (2-16). Es decir, supone que la distribución $P(Z|X = D)$ esta en un conjunto de distribuciones $\mathcal{H}$, y en este conjunto trata de encontrar la distribución $q^*(Z)$ que mejor se aproxime a $P(Z|X = D)$ en el sentido de D_{KL} [9], esto se puede escribir matemáticamente como

$$q^*(Z) = \arg \min_{q(Z) \in \mathcal{H}} D_{KL}(q(Z)||P(Z|X = D)),$$

como habíamos visto antes esta aproximación resultara en una distribución que mejor aproxime el modo de la distribución, es decir si por ejemplo $P(Z|X = D)$ es una distribución multimodal, y buscamos en

$$\mathcal{H} = \{N|N = \mathcal{N}(\mu, \Sigma), \text{ Para algunos } \mu \text{ y } \Sigma\}$$

a q , lo que vamos a encontrar es una distribución que aproxime el modo, como se muestra en la Figura 2-16. Ahora podemos hacer esto aún más intuitivo de la siguiente manera

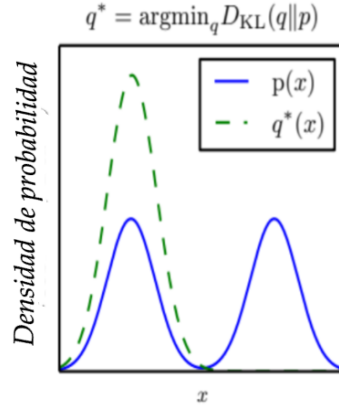


Figura 2-16: Cuando intentamos aproximar una distribución bimodal (gráfica azul) por medio de un conjunto de distribuciones unimodales (gráfica verde punteada), por medio de la divergencia KL a la izquierda, es decir, haciendo $\arg \min_q D_{KL}(q||p)$ lo que encontramos es una distribución que aproxime alguno de los dos modos [7]. Imagen tomada de [7]

$$D_{KL}(q(Z)||P(Z|D)) = \int_{\mathbb{R}^K} q(Z) \log \left(\frac{q(Z)}{P(Z|D)} \right) dz \quad (2-25)$$

$$= \int_{\mathbb{R}^K} q(Z) \log \left(\frac{q(Z)P(D)}{P(Z, D)} \right) dz \quad (2-26)$$

$$= \int_{\mathbb{R}^K} q(Z) \log \left(\frac{q(Z)}{P(Z, D)} \right) dz + \int_{\mathbb{R}^K} q(Z) \log(P(D)) dz \quad (2-27)$$

$$= \mathbb{E}_{z \sim q(Z)} \left[\log \left(\frac{q(Z)}{P(Z, D)} \right) \right] + \mathbb{E}_{z \sim q(Z)} [\log P(D)] \quad (2-28)$$

$$= -\mathbb{E}_{z \sim q(Z)} \left[\log \left(\frac{P(Z, D)}{q(Z)} \right) \right] + \mathbb{E}_{z \sim q(Z)} [\log P(D)] \quad (2-29)$$

$$= -\mathcal{L}(q) + \log P(D). \quad (2-30)$$

Para la ecuación (2-26) se utilizó el hecho de que $P(Z, D) = P(Z|D)P(D)$, para la ecuación (2-27) se utilizó la aditividad del logaritmo y de la integral, para la ecuación (2-28) se utilizó la notación que introdujimos en la ecuación (2-9) y ya por último para la ecuación (2-30) estamos utilizando el hecho de que la esperanza de una constante es ella misma. Ahora $\log(p(D))$ es llamado la evidencia (pues es simplemente el logaritmo de los datos, es decir, lo que se sabe), $\mathcal{L}(q)$ es llamado *ELBO* o Evidence Lower Bound [9, 8], puesto que por la propiedad de D_{KL} de ser siempre mayor o igual a 0, y que $\log P(D) \leq 0$, se debe tener que $\mathcal{L}(q) \leq \log P(D)$, es decir es una cota inferior de la evidencia, note además que esto mismo

implica lo siguiente

$$q^*(Z) = \arg \min_{q(Z) \in \mathcal{H}} D_{KL}(q(Z) || P(Z|X = D)) = \arg \max_{q(Z) \in \mathcal{H}} \mathcal{L}(q). \quad (2-31)$$

Es decir minimizar la divergencia Kl es equivalente a maximizar el ELBO. Uno de los problemas centrales de la inferencia variacional es justamente el estudio en la maximización del ELBO, eso muchas veces se convierte en un problema rutinario de optimización, en el siguiente enlace se puede ver como va variando el ELBO, la divergencia KL, el posterior verdadero y el posterior que generamos (ver [9]). Ilustremosnos con un ejemplo. Suponga que tenemos el gráfico dado en la figura **2-17** que relaciona la dependencia de dos variables aleatorias (este tipo de gráficos, son comúnmente llamados DAG, por sus siglas en inglés *Directed Acyclic Graph* [8]), En este caso vamos a suponer que $Z \in \mathbb{R}_+$ son características, $X \in \mathbb{R}$ son datos, y que además se tienen las ecuaciones (2-32) y (2-33)



Figura 2-17: *Direct Acyclic Graph asociado a una distribución exponencial con parámetro $\lambda = 1$ y una distribución normal con media $\sigma = z$, la flecha entre en nodo de la variable aleatoria con distribución exponencial y el nodo de la variable aleatoria con distribución normal indica una relación de condicional entre ella (Imagen tomada de [8]).*

$$Z \sim \text{Exp}(z; \lambda = 1) \quad (2-32)$$

y

$$X \sim \mathcal{N}(\mu = z, \sigma = 1). \quad (2-33)$$

En este caso vemos que la distribución de la variable aleatoria X tiene una dependencia de la variable aleatoria Z , podemos asociarle la condición

$$P(x|z) = \mathcal{N}(x; \mu = z, \sigma = 1) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}(x-z)^2}$$

es la distribución de los datos X dadas las características, generalmente a esta distribución se le llama likelihood [9, 8] o verosimilitud, pues mide que tan parecido es que un dato provenga de las características de el dataset, además, también podemos saber cual es la distribución conjunta, por medio de

$$P(x, z) = P(z)P(x|z) = e^{-z} I_{z \geq 0} \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}(x-z)^2}.$$

Como vimos anteriormente, la distribución marginal puede ser difícil de calcular, en este caso

$$P(x) = \int_0^\infty P(X, Z) dz = \int_0^\infty e^{-z} \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}(x-z)^2} dz,$$

es costoso de calcular, cuando las dimensiones de X y Z crecen [9, 8], es por esto que se decide utiliza el ELBO para encontrar la distribución posterior $P(Z|X = D)$. Para simplificar los cálculos suponga que tenemos un solo dato $D = \{x_0\}$. Además tenemos que escoger el conjunto $\mathcal{H}$ en donde se encontraran nuestras candidatas a representantes de la verdad posterior $P(Z|X = D)$. Para este ejemplo en particular escogeremos convenientemente el conjunto de las distribuciones exponenciales Q que consta de la distribuciones de la forma

$$q_\theta(z) = \text{Exp}(z; \lambda = \theta) = \theta e^{-\theta z} I_{z \geq 0} \quad \theta \in \mathbb{R}.$$

Esta decisión puede ser complicada y puede requerir un análisis extra para determinar esta familia, en este caso se sabe mas o menos que forma tiene la posterior, y por tanto se decide optar por este conjunto. De esta manera tenemos que el ELBO, tiene la siguiente forma

$$\begin{aligned} \mathcal{L}(q_\theta) &= \mathcal{L}(\theta) \\ &= \mathbb{E}_{z \sim q_\theta(z)} \left[\log \left(\frac{P(z, x = D)}{q_\theta(z)} \right) \right] \\ &= \mathbb{E}_{z \sim q_\theta} [\log P(z, x = D) - \log(q_\theta(z))] \\ &= \mathbb{E}_{z \sim q_\theta} \left[-z + 0 - \frac{1}{2} \log(2\pi) - \frac{1}{2}(x_0 - z)^2 - \log \theta - \theta z + 0 \right] \\ &= \mathbb{E}_{z \sim q_\theta} \left[-z - \frac{1}{2} \log(2\pi) - \frac{1}{2}x_0^2 + x_0z - \frac{1}{2}z^2 - \log \theta + \theta Z \right] \\ &= -\frac{1}{2} \log(2\pi) - \frac{1}{2}x_0^2 - \log \theta - \frac{1}{\theta} + \frac{x_0}{\theta} + \frac{\theta}{\theta} - \frac{1}{2} \frac{2}{\theta^2} \\ &= -\log(\theta) + \frac{x_0 - 1}{\theta} - \frac{1}{\theta^2}. \end{aligned}$$

Ahora dado que ya tenemos el ELBO de manera cerrada y además diferenciable en todo punto excepto en $\theta = 0$, podemos, encontrar el máximo haciendo:

$$\frac{dL}{d\theta} = \frac{1}{\theta} - \frac{x_0 - 1}{\theta^2} + \frac{2}{\theta^3} = 0,$$

despejando theta y descartando la opción que nos da un mínimo obtenemos que

$$\theta^* = -\frac{x_0 - 1}{2} + \sqrt{\frac{(x_0 - 1)^2}{4} + 2}$$

así la distribución que mejor se aproxima al posterior verdadero, en el conjunto de candidatos exponenciales

$$q_{\theta^*}(z) = \left(-\frac{x_0 - 1}{2} + \sqrt{\frac{(x_0 - 1)^2}{4} + 2} \right) e^{-\left(-\frac{x_0 - 1}{2} + \sqrt{\frac{(x_0 - 1)^2}{4} + 2} \right) z} I_{z \geq 0}.$$

2.2.4. Truco de reparametrización

A continuación introduciremos, una forma de muestrear de una manera algo distinta, que funcionara para reducir la variabilidad en la respuestas cuando se utilice el gradiente en descenso de la Sección 2.1.1 (ver [9]). Para ello veamos el siguiente problema, suponga que deseamos optimizar la siguiente función respecto de θ

$$\mathbb{E}_{z \sim q_\theta(z)} [\log p(x|z)p(z) - \log q_\theta(z)]. \quad (2-34)$$

calculando el gradiente respecto a θ , tenemos

$$\begin{aligned} \nabla_\theta \mathbb{E}_{z \sim q_\theta(z)} [\log p(x|z)p(z) - \log q_\theta(z)] &= \nabla_\theta \int q_\theta(z) [\log p(x|z)p(z) - \log q_\theta(z)] dz \\ &= \int \nabla_\theta [q_\theta(z) [\log p(x|z)p(z) - \log q_\theta(z)]] dz \\ &= \int \nabla_\theta q_\theta(z) [\log p(x|z)p(z) - \log q_\theta(z)] dz + \\ &\quad \int q_\theta(z) \nabla_\theta [\log p(x|z)p(z) - \log q_\theta(z)] dz \\ &= \int \nabla_\theta q_\theta(z) [\log p(x|z)p(z) - \log q_\theta(z)] dz - \\ &\quad \int \frac{1}{q_\theta(z)} \nabla_\theta q_\theta(z) q_\theta(z) dz \\ &= \int q_\theta(z) \nabla_\theta \log q_\theta(z) [\log p(x|z)p(z) - \log q_\theta(z)] dz \\ &= \mathbb{E}_{z \sim q_\theta(z)} \nabla_\theta \log q_\theta(z) [\log p(x|z)p(z) - \log q_\theta(z)] \\ &\approx \frac{1}{L} \sum_{l=1}^L \nabla_\theta \log q_\theta(z_l) [\log p(x|z_l)p(z_l) - \log q_\theta(z_l)] \end{aligned}$$

En la segunda igualdad utilizamos la regla de leibniz, en la quinta igualdad utilizamos que $\nabla_\theta q_\theta(z) = q_\theta(z) \nabla_\theta \log q_\theta(z)$, y en la ultima utilizamos una aproximación de la esperanza utilizando l muestras de la distribución $q_\theta(z)$, ahora aunque esta ultima función sea diferenciable, el inconveniente es que el descenso del gradiente bajo pequeñas perturbaciones de los datos converge a parámetros muy distintos, lo que hace que sea ineficiente para grandes conjunto de datos ver [19]. Una forma para que podamos solucionar esto es no muestrear directamente de los datos si no utilizar una distribución auxiliar [9], por ejemplo supongamos lo siguiente

i) $\varepsilon \sim p'(\varepsilon)$

ii) $z = g_\theta(\varepsilon, x)$ donde g es una transformación diferenciable

Luego el anterior problema de optimización que teníamos anteriormente se transforma en

$$\begin{aligned}
 \nabla_{\theta} ELBO &= \nabla_{\theta} \mathbb{E}_{q_{\theta}} [\log(p(x, z)) - \log q_{\theta}(z)] \\
 &= \nabla_{\theta} \mathbb{E}_{p'} [\log(p(x, g_{\theta}(\varepsilon, x))) - \log q_{\theta}(g_{\theta}(\varepsilon, x))] \\
 &\approx \nabla_{\theta} \frac{1}{L} \sum_{l=1}^L [\log(p(x, g_{\theta}(\varepsilon_l, x))) - \log q_{\theta}(g_{\theta}(\varepsilon_l, x))]
 \end{aligned}$$

Donde $\varepsilon_l \sim p'(\varepsilon)$. Un ejemplo de como podríamos utilizar este truco de reparametrización, es el siguiente; Supongamos que queremos muestrear l datos de la distribución $z \sim \mathcal{N}(\mu, \sigma)$, en vez de hacerlo directamente, podríamos hacerlo a partir de muestrear de la distribución $\varepsilon \sim \mathcal{N}(0, 1)$, y luego utilizar transformación $g(\varepsilon) = \sigma \cdot \varepsilon + \mu$, de esta manera los l datos que buscamos podrían estar dados en la forma $z_l = g(\varepsilon) = \sigma \cdot \varepsilon_l + \mu$.

2.2.5. Codificador automático variacional

Primero introduzcamos el concepto de *codificador automático*, como tal un codificador automático es un tipo de red neuronal utilizada para el aprendizaje no supervisado [7] (sin etiquetas) de representaciones eficientes de datos. La arquitectura del codificador automático consta de dos partes principales: el encoder (codificador) y el decoder (decodificador). La idea básica de un codificador automático es aprender una representación comprensiva y útil de los datos de entrada, reduciendo la dimensionalidad de la información mientras se intenta reconstruir los datos originales de manera precisa [7]. El *encoder* es un codificador, en esta etapa la red reduce la dimensionalidad de los datos de entrada. El encoder toma los datos originales y los transforma en una representación de menor dimensión que a menudo se llama *bottleneck* (cuello de botella). Después de la fase de codificación, el decoder toma la representación de menor dimensión generada por el encoder y la utiliza para intentar reconstruir los datos originales. El objetivo es que la reconstrucción sea lo más fiel posible a los datos de entrada originales.

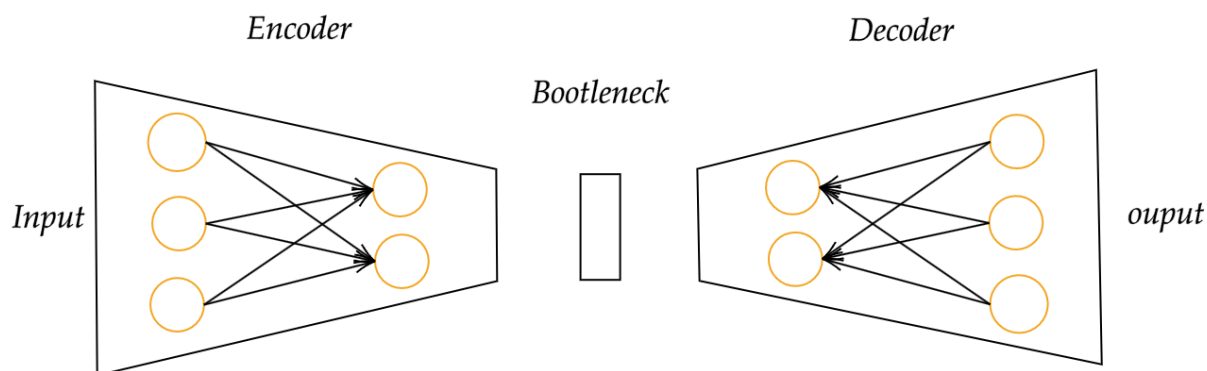


Figura 2-18: Esquema de un codificador variacional, la primera parte (encoder) trata de reducir la dimensionalidad de la entrada (input) y la segunda parte (decoder) trata de reconstruir lo que se redujo en el encoder. Los círculos y flechas que se encuentran en el encoder y decoder hacen referencia a ellos son como tal redes neuronales. Imagen inspirada de [8].

Como mostramos en la Figura 2-18 un autoencoder toma un dato y su salida trata de ser el mismo dato, así la dimensión del dato de salida, es igual a la dimensión a la del dato de entrada. De manera formal si $E : \mathbb{R}^n \rightarrow \mathbb{R}^k$, es el encoder y $D : \mathbb{R}^k \rightarrow \mathbb{R}^n$ es el decoder, donde n es la dimensión del dato de entrada y k es la dimensión del espacio latente o número

de neuronas del bottleneck, entonces el autoencoder total se puede ver como

$$V = D \circ E.$$

Ahora un *VAE* (*variational autoencoder* o *codificador automático variacional* en español), es un codificador automático pero en este caso a un X de entrada se le asocian la media y la matriz de covarianza (diagonalizada) de una distribución gaussiana, luego a partir de un muestreo de esta distribución se toma un dato Z , y luego haciendo un proceso inverso se trata de recuperar este dato (ver Figura 2-19)

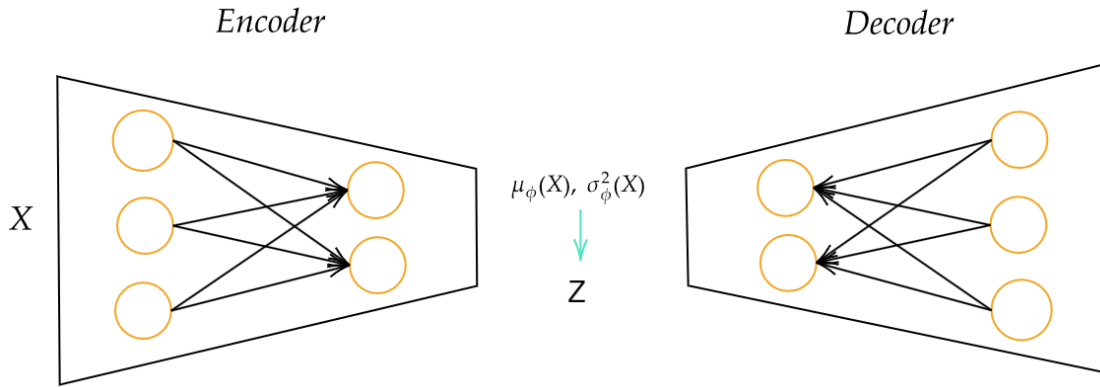


Figura 2-19: Esquema de un VAE, en este caso el encoder mapea un dato X , a los parámetros de una distribución normal multivariada ($\mu_\phi(X)$ y $\sigma_\phi^2(X)$), y el decoder a partir de un muestreo de esta distribución intenta decodificar el dato. Imagen inspirada de [9]

Lo que se hace en la parte del encoder es transformar un dato $x^{(i)}$ en dos vectores μ y σ que definen los parámetros de una distribución en el espacio latente (espacio cuya dimensión escogemos). La distribución que se asume es una distribución normal multivariada.

Veamos esto un poco más en detalle, recordemos que el ELBO para un conjunto de datos fijo, estaba dado por la fórmula

$$\mathcal{L}(q_\phi) = -D_{KL}(q_\phi(z)|p(z|D)) + \mathbb{E}_{z \sim q_\phi}[\log p(D)].$$

En este caso ϕ denotarían los parámetros correspondientes al encoder, y θ corresponderá a los parámetros del decoder, además en vez de utilizar $q_\phi(z)$ para aproximar $p(z|D)$, utilizaremos $q_\phi(z|x^{(i)})$ y aproximaremos $p(z)$ (posterior aproximará la distribución de las variables latentes). Otro cambio que haremos es en la parte del logaritmo de $p_\theta(D)$, ahora como el conjunto de datos es móvil, esta evidencia dependerá de los datos latentes que tomemos, así esta parte cambiará por $\log p_\theta(x^{(i)}|z)$, por tanto el objetivo, a maximizar, es

$$\mathcal{L}(\phi, \theta, x^{(i)}) = -D_{KL}(q_\phi(z|x^{(i)})|p(z)) + \mathbb{E}_{z \sim q_\phi(z|x^{(i)})}[\log p_\theta(x^{(i)}|z)]. \quad (2-35)$$

Notamos que no todo es inferencia variacional, como la habíamos planteado, pues la evidencia es también móvil respecto a θ , y respecto al conjunto de variables latentes, estamos ajustando tanto el posterior como la verosimilitud al mismo tiempo [19]. Haremos las siguientes suposiciones sobre $p(z)$, $q_\phi(z|x^{(i)})$, $p_\theta(x^{(i)}|z)$

- i) Diremos que $p(z) = \mathcal{N}(z; 0, I)$, I es la identidad de tamaño k , donde k es la dimensión del espacio latente.
- ii) Diremos que $q_\phi(z|x^{(i)}) = \mathcal{N}(z; \mu_\phi(x^{(i)}), \sigma_\phi^2(x^{(i)})I)$, I es la identidad de tamaño k , donde k es la dimensión del espacio latente, y $\mu_\phi(x)$, es la media de tamaño k .
- iii) Diremos que $p_\phi(x|z) = \mathcal{N}(x; \mu_\phi(z), \Sigma_\phi(z))$, donde $\mu_\phi(z)$ es la media de tamaño n , n es la dimensión de los datos y $\Sigma_\phi(z)$, es la matriz de covarianza de tamaño n

De la ecuación (2-35), y de las anteriores suposiciones, vemos que lo que intentamos hacer es acomodar las variables latentes y la distribución de estas, de tal manera que estas características latentes se centren en 0 (con desviación estándar I) [19], en la Figura 2-20 se ve como los datos se tratan de distribuir conforme $\mathcal{N}(0, I)$, esto por parte del primer sumando de (2-35), y por parte del segundo sumando lo que intentamos es ver que tan verosímiles son nuestro datos a la distribución de las datos latentes.

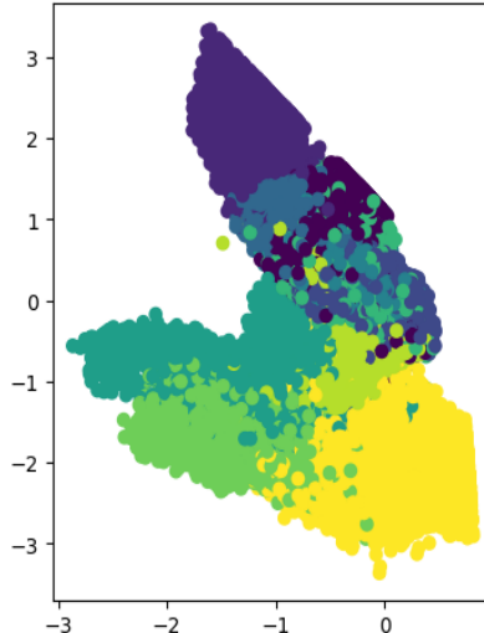


Figura 2-20: Representaciones latentes de los datos de Fashion-MNIST [10], los colores representan una prenda en particular, por ejemplo el color amarillo son las representaciones latentes de imágenes de botas, Así es como se verían distribuidos los datos de todas las fotos de las prendas (imágenes de tamaño 784) luego de haberlas pasado por el encoder a un espacio latente de dimensión 2. Como se ve los datos parecen estar distribuidos cerca a $(0,0)$ y para ciertas direcciones se acomodan representaciones de los mismos tipos de datos, Imagen inspirada de [10]

Ahora que ya tenemos la idea de como funciona el VAE, procederemos a transformar la

ecuación (2-35), en algo un poco mas trabajable, para poder optimizarlo

$$\mathcal{L}(\phi, \theta, x^{(i)}) = -D_{KL}(q_\phi(z|x^{(i)})|p(z)) + \mathbb{E}_{z \sim q_\phi(z|x^{(i)})}[\log p_\theta(x^{(i)}|z)] \quad (2-36)$$

$$\approx -D_{KL}(q_\phi(z|x^{(i)})|p(z)) + \frac{1}{L} \sum_{l=1}^L \log p_\theta(x^{(i)}|\epsilon_l * \sigma_\phi(x^{(i)}) + \mu_\phi(x^{(i)})), \quad (2-37)$$

para la ultima ecuación podemos notar que se utilizaron 2 cosas, la primera es una aproximación de la media, y la segunda es el truco de reparametrización de la Subsección 2.2.4, en este caso en vez de muestrear directamente de $z^{(i,l)} \sim q_\phi(z|x^{(i)})$, utilizamos la transformación diferenciable, $g_\phi(\epsilon^{(l)}, x^{(i)}) = \mu_\phi(x^{(i)}) + \sigma_\phi(x^{(i)}) * \epsilon^{(l)}$, donde $\epsilon^{(l)} \sim \mathcal{N}(0, I)$. Por otro lado recordemos que de la tercera propiedad del Teorema 2.2.2 (ecuación (2-23)), tenemos los siguiente

$$-D_{KL}(q_\phi(z|x^{(i)})|p(z)) = \frac{1}{2}(k + \sum_{j=1}^k \sigma_{\phi j}(x^{(i)})^2 + \mu_{\phi j}^2(x^{(i)}) - \ln(\sigma_{\phi i}^2(x^{(i)}))). \quad (2-38)$$

Ahora notemos que maximizar la segunda parte de la ecuación (2-32), se puede hacer de la siguiente manera

$$\begin{aligned} & \arg \max_{\theta, \phi} \mathbb{E}_{q_\phi}[\log p_\theta(x^{(i)}|z)] \\ &= \arg \max_{\theta, \phi} \frac{1}{L} \sum_{l=1}^L \left[-\frac{d}{2} \ln(2\pi) - \frac{1}{2} \log |\Sigma_\theta(z_\phi^{(l)})| - \frac{1}{2} (x^{(i)} - \mu_\theta(z_\phi^{(l)}))^T \Sigma_\theta(z_\phi^{(l)})^{-1} (x^{(i)} - \mu_\theta(z_\phi^{(l)})) \right] \\ &= \arg \max_{\theta, \phi} \frac{1}{L} \sum_{l=1}^L \left[-\frac{1}{2} \log |\Sigma_\theta(z_\phi^{(l)})| - \frac{1}{2} (x^{(i)} - \mu_\theta(z_\phi^{(l)}))^T \Sigma_\theta(z_\phi^{(l)})^{-1} (x^{(i)} - \mu_\theta(z_\phi^{(l)})) \right]. \end{aligned}$$

Si asumimos que $\Sigma_\theta = 1$, entonces tenemos que lo anterior se puede escribir como

$$= \arg \max_{\theta, \phi} \frac{1}{L} \sum_{l=1}^L \left[-\frac{1}{2} (x - \mu_\theta(z_\phi^{(l)}))^T (x - \mu_\theta(z_\phi^{(l)})) \right] \quad (2-39)$$

$$= \arg \min_{\phi, \theta} \sum_{l=1}^L (x^{(i)} - \mu_\theta(z_\phi^{(l)}))^T (x^{(i)} - \mu_\theta(z_\phi^{(l)})) \quad (2-40)$$

$$= \arg \min_{\phi, \theta} \sum_{l=1}^L (x^{(i)} - \mu_\theta(z_\phi^{(l)}))^T (x^{(i)} - \mu_\theta(z_\phi^{(l)})). \quad (2-41)$$

Generalmente la optimización de las ecuaciones (2-38) y (2-39), se hacen computacionalmente utilizando variantes del descenso del gradiente como Adam optimizer (ver [20]). Un ejemplo de como se distribuyen los datos, puede ser apreciado en la Figura **2-21**, en donde se muestra como en las representaciones latentes pueden codificar las imágenes de personas o dígitos luego de haber aprendido la distribución.



Figura 2-21: *Espacio latente (de dimensión 2) de un codificador automático variacional, entrenado con un imágenes de caras de la universidad de Chicago [11], se asigno a algunos puntos cercanos a $(0,0)$ su respectiva decodificación por medio del decodificador y se graficaron sobre ellos. En la imagen se aprecia como la red neuronal aprende determinadas características, por ejemplo en la imagen se ve que subiendo en el eje y las personas tiene cabello y bajando no. Imagen propia [12]*

El Codificador Automático Variacional (VAE) nos proporciona una metodología robusta para la compresión y reconstrucción de datos. como vimos el VAE mapea los datos de entrada X a una distribución latente mediante los parámetros $\mu_\phi(X)$ y $\sigma_\phi^2(X)$ esta representación latente es crucial, ya que nos permite generar nuevas muestras desde una distribución normal multivariada. Además dado que la función de perdida esta dada por la ecuación (2-35) este enfoque no solo ayuda en la reducción dimensional, sino que también preserva la estructura estadística de los datos originales, facilitando aplicaciones en generación de datos sintéticos y análisis de variabilidad [4]. Otro punto a tener en cuenta es que el espacio latente aprendido por el VAE ofrece una representación compacta y significativa de los datos de entrada que utilizaremos en el Capítulo 5 para proyectar sobre la imagen de estos. En los experimentos con conjuntos de datos como Fashion-MNIST o imágenes de caras, se observa que las características similares se agrupan en regiones específicas del espacio latente (como se ve en las Figuras 2-20 y 2-21). Esto se ilustra mediante visualizaciones donde puntos en el espacio latente corresponden a reconstrucciones específicas, mostrando la capacidad del modelo para capturar variaciones clave en los datos.

3 Métodos Clásicos

En esta sección, veremos algunos métodos clásicos para calcular valores propios de matrices grandes, compararemos su velocidad para llegar a un umbral de precisión. Estos métodos han sido utilizados como base para algoritmos mas sofisticados como se ve en [1] y [2]. El material presentado en capítulo es tomado de [1].

3.1. Jacobi

El Método de Jacobi (Sección 8.5 de [1]) para la obtención de valores y vectores propios de una matriz simétrica consiste en lo siguiente : Dada una matriz simétrica A de tamaño n , la idea es llevar a A lo más cerca posible a una forma diagonal. Si denotamos por $D^{(0)}$ la diagonal de A y $N^{(0)}$ la parte no diagonal, entonces

$$A = D^{(0)} + N^{(0)}$$

y

$$\|A\|_F^2 = \|D^{(0)}\|_F^2 + \|N^{(0)}\|_F^2.$$

Se quiere un algoritmo que en cada iteración disminuya la norma de Frobenius de $N^{(i)}$, es decir

$$\|N^{(i+1)}\|_F < \|N^{(i)}\|_F$$

Definimos

$$\text{off}(A) = \|N^{(0)}\|_F = \sqrt{\sum_{i=1}^n \sum_{j=i, j \neq i}^n a_{ij}^2}$$

Para esto se utilizan matrices de Givens para eliminar selectivamente componentes de $N^{(i)}$ (Sección 8.5 de [1]). Se escogen índices p, q con $p < q$ y se busca eliminar dos entradas de la matriz por fuera de la diagonal, (generalmente se toma q de tal manera que se elimine la entrada en valor absoluto mas grande fuera de la diagonal), es decir, queremos escoger c y s adecuados para que la siguiente matriz sea diagonal.

$$\begin{aligned} \begin{bmatrix} b_{pp} & b_{pq} \\ b_{qp} & b_{qq} \end{bmatrix} &= \begin{bmatrix} c & -s \\ s & c \end{bmatrix} \begin{bmatrix} a_{pp} & a_{pq} \\ a_{qp} & a_{qq} \end{bmatrix} \begin{bmatrix} c & -s \\ s & c \end{bmatrix}^T \\ &= \begin{bmatrix} b_{pp} & 0 \\ 0 & b_{qq} \end{bmatrix}. \end{aligned}$$

Recordando que las transformaciones ortogonales preservan la norma de Frobenius, tenemos de la anterior igualdad que $\|A\|_F = \|G(p, q, \theta)AG(p, q, \theta)^T\|_F$, o equivalentemente

$$a_{pp}^2 + a_{qq}^2 + 2a_{pq}^2 = b_{pp}^2 + b_{qq}^2.$$

Notando $B = G(p, q, \theta)AG(p, q, \theta)^T$, dado que las diagonales de B y A solo difieren en las componentes con índices pp y qq , tenemos que

$$\text{off}(B)^2 = \|B\|_F^2 - \sum_{i=1}^n b_{ii}^2 \quad (3-1)$$

$$= \|A\|_F^2 - \sum_{i=1}^n a_{ii}^2 - (a_{pp}^2 + a_{qq}^2 - b_{pp}^2 - b_{qq}^2) \quad (3-2)$$

$$= \text{off}(A)^2 - 2a_{pq}^2, \quad (3-3)$$

es decir

$$\text{off}(B) \leq \text{off}(A),$$

por lo que B es “más diagonal” que A .

Teorema 3.1.1. *Considere el procedimiento anterior, aplicado a una matriz simétrica A , de tamaño $n \times n$. Entonces, si $\{A_k\}_{k \in \mathbb{N}}$ denota las matrices obtenidas al hacer la transformación ortogonal de Givens sucesivamente a $A^{(k)}$, entonces*

$$\lim_{k \rightarrow \infty} \text{off}(A^k) = 0. \quad (3-4)$$

Demostración. Suponga que en cada iteración escogemos a_{pq} de tal forma que sea el número de mayor magnitud de los elementos fuera de la diagonal (si hay varios escoja uno), luego tenemos que

$$\text{off}(A^{(k)})^2 \leq N((a_{pq}^{(k)})^2 + (a_{qp}^{(k)})^2) = 2N(a_{pq}^{(k)})^2$$

Donde $N = \frac{n(n-1)}{2}$, pero note que lo anterior es equivalentemente a:

$$\frac{\text{off}(A_k)^2}{N} \leq 2(a_{pq}^{(k)})^2. \quad (3-5)$$

Luego por (3-5), vemos que (3-1), implica que

$$\text{off}(B)^2 \leq \text{off}(A)^2 - \frac{1}{N} \text{off}(A)^2 = \left(1 - \frac{1}{N}\right) \text{off}(A)^2.$$

Por inducción si $A^{(k)}$, denota la matriz A , después de k iteraciones, del método de Jacobi, entonces

$$\text{off}(A^{(k)})^2 \leq \left(1 - \frac{1}{N}\right)^k \text{off}(A^{(0)})^2.$$

Como $1 - \frac{1}{N} < 1$, esto implica que el método converge a una matriz cuyas entradas fuera de la diagonal son nulas, o equivalentemente hemos probado (3-4). $\square$

Esto quiere decir que el método clásico de jacobi converge de manera lineal [1]. Ahora computacionalmente, el calculo de c y s se hacen de la siguiente manera, para minimizar los errores de redondeo, considere

$$0 = a_{pq}(c^2 - s^2) + (a_{pp} - a_{qq})cs.$$

Asumimos que $a_{pq} \neq 0$, o de lo contrario, escogemos la matriz de Givens igual a la identidad. Tenemos

$$\frac{a_{qq} - a_{pp}}{2a_{pq}} = \frac{(c^2 - s^2)}{2cs} = \frac{1}{2} \left(\frac{c}{s} - \frac{s}{c} \right).$$

Escogiendo $\tau = \frac{a_{qq} - a_{pp}}{2a_{pq}}$ y $t = \frac{s}{c}$. Lo anterior se escribe

$$\tau = \frac{1}{2} \left(\frac{1}{t} - t \right)$$

osea

$$2t\tau = 1 - t^2$$

y por tanto

$$t^2 + 2t\tau - 1 = 0.$$

Las raíces son

$$t = -\tau \pm \sqrt{1 + \tau^2}$$

y se despeja c y s en términos de t así,

$$c = \frac{1}{\sqrt{1 + t^2}}, \quad s = tc.$$

Se escoge la menor de las raíces, note que este procedimiento nos ahorra el calculo explícito del ángulo de rotación, además que no implica restas que puedan hacer cancelaciones catastróficas, por lo que es muy estable (ver [1]).

3.2. Iteración de potencias

El método de potencias es un algoritmo utilizado para encontrar el vector propio dominante de una matriz (Sección 8.2 de [1]). Dada una matriz A y un vector inicial no nulo q_0 , el método itera utilizando la siguiente fórmula

$$q_{k+1} = \frac{Aq_k}{\|Aq_k\|}, \tag{3-6}$$

donde k es el índice de la iteración. La sucesión de vectores resultante $\{q_k\}$ tiende a converger hacia el vector propio de A del valor propio dominante si este valor propio es único. El

método de potencias se detiene cuando se alcanza un criterio de convergencia, que generalmente involucra comparar la magnitud del cambio en q_k con un umbral predefinido [1]. A continuación mostramos que en efecto el procedimiento, mostrado en (3-6), converge a un vector propio de la matriz A , de hecho converge al vector propio con valor propio en valor absoluto mas grande.

Algorithm 5 Método de potencias

```

1: Escoja  $q^{(0)} \in \mathbb{R}^n$ , y  $A$  una matriz a de  $\mathbb{R}^{n \times n}$ 
2: for  $k = 1, 2, \dots$  do
3:    $z^{(k)} = Aq^{(k-1)}$ 
4:    $q^{(k)} = z^{(k)} / \|z^{(k)}\|_2$ 
5:    $\lambda^{(k)} = [q^{(k)}]^T Aq^{(k)}$ 
6: end for

```

A Continuación presentamos una prueba, de la convergencia de este método para matrices simétricas con vector propio dominante de multiplicidad algebraica igual a 1 (tomada de [1]). Cabe aclarar las condiciones de que sea simétrica y de que la multiplicidad algebraica del mayor valor propio sea 1, se puede relajar, de hecho sirve para cualquier matriz cuadrada siempre y cuando el vector inicial no sea ortogonal al vector propio del mayor valor propio, si quiere consultar una prueba de esto que utiliza bloques de Jordán para probarlo puede consultar [21]

Teorema 3.2.1. . Suponga que $A \in \mathbb{R}^{n \times n}$, es simétrica y que

$$Q^T A Q = \text{diag}(\lambda_1, \dots, \lambda_n), \quad (3-7)$$

donde $Q = [q_1 | \dots | q_n]$ es ortogonal y $|\lambda_1| > |\lambda_2| \geq \dots |\lambda_n|$. Sean los vectores $q^{(k)}$ como se especifican en el algoritmo (7), y sea $\theta_k \in [0, \pi/2]$, tal que

$$\cos(\theta_k) = |q_1^T q^{(k)}|.$$

Si $\cos(\theta_0) \neq 0$, entonces para $k = 0, 1, \dots$, tenemos que

$$|\lambda^{(k)} - \lambda_1| \leq \max_{2 \leq i \leq n} |\lambda_1 - \lambda_i| \tan(\theta_0)^2 \left| \frac{\lambda_2}{\lambda_1} \right|^{2k}. \quad (3-8)$$

Demostración. Por la definición de la iteración, se sigue que $q^{(k)}$ es un multiplo de $A^k q^{(0)}$, y por lo tanto

$$|\sin(\theta_k)|^2 = 1 - (q_1^T q^{(k)})^2 = 1 - \left(\frac{q_1^T A^k q^{(0)}}{\|A^k q^{(0)}\|_2} \right)^2.$$

Si $q^{(0)}$ tiene la expansión en vectores propios: $q^{(0)} = a_1 q_1 + \dots + a_n q_n$, entonces

$$|a_1| = |q_1^T q^{(0)}| = \cos(\theta_0) \neq 0,$$

con

$$a_1^2 + \cdots + a_n^2 = 1$$

y

$$A^{(k)} q^{(0)} = a_1 \lambda_1^k q_1 + a_2 \lambda_2^k q_2 + \cdots + a_n \lambda_n^k q_n.$$

Así, tenemos lo siguiente

$$\begin{aligned} |\sin(\theta_k)|^2 &= 1 - \frac{a_1^2 \lambda_1^{2k}}{\sum_{i=1}^n a_i^2 \lambda_i^{2k}} = \frac{\sum_{i=2}^n a_i^2 \lambda_i^{2k}}{\sum_{i=1}^n a_i^2 \lambda_i^{2k}} \leq \frac{\sum_{i=2}^n a_i^2 \lambda_i^{2k}}{a_1^2 \lambda_1^{2k}} \\ &= \frac{1}{a_1^2} \sum_{i=2}^n a_i^2 \left(\frac{\lambda_i}{\lambda_1} \right)^{2k} \leq \frac{1}{a_1^2} \left(\sum_{i=2}^n a_i^2 \right) \left(\frac{\lambda_2}{\lambda_1} \right)^{2k} \\ &= \frac{1 - a_1^2}{a_1^2} \left(\frac{\lambda_2}{\lambda_1} \right)^{2k} = \tan(\theta_0)^2 \left(\frac{\lambda_2}{\lambda_1} \right)^{2k}. \end{aligned}$$

Por otro lado note que

$$\lambda^{(k)} = [q^{(k)}]^T A q^{(k)} = \frac{[q^{(0)}]^T A^{2k+1} q^{(0)}}{[q^{(0)}]^T A^{2k} q^{(0)}} = \frac{\sum_{i=1}^n a_i^2 \lambda_i^{2k+1}}{\sum_{i=1}^n a_i^2 \lambda_i^{2k}}. \quad (3-9)$$

Así, tenemos lo siguiente

$$|\lambda^{(k)} - \lambda_1| = \left| \frac{\sum_{i=2}^n a_i^2 \lambda_i^{2k} (\lambda_i - \lambda_1)}{\sum_{i=1}^n a_i^2 \lambda_i^{2k}} \right| \leq \max_{2 \leq i \leq n} |\lambda_1 - \lambda_i| \cdot \frac{1}{a_1^2} \cdot \sum_{i=2}^n a_i^2 \left(\frac{\lambda_i}{\lambda_1} \right)^{2k} \quad (3-10)$$

$$\leq \max_{2 \leq i \leq n} |\lambda_1 - \lambda_i| \cdot \frac{1 - a_1^2}{a_1^2} \cdot \left(\frac{\lambda_2}{\lambda_1} \right)^{2k} \quad (3-11)$$

$$= \max_{2 \leq i \leq n} |\lambda_1 - \lambda_i| \cdot \tan(\theta_0)^2 \cdot \left(\frac{\lambda_2}{\lambda_1} \right)^{2k}. \quad (3-12)$$

Luego cuando $k \rightarrow \infty$, vemos que el método converge, al mayor valor propio y a su vector asociado, segun las ecuaciones (3-13) y (3-9).

□

Para hallar un conjunto de valores y vectores propios de la matriz se puede realizar de dos maneras diferentes; Método por matriz ortogonal que consiste en seleccionar un matriz ortogonal aleatoria para realizar la iteración anterior, pero esta vez utilizando descomposición QR y deflación, que consiste en ir eliminando los valores propios de mayor magnitud de la matriz (ver [1]).

3.2.1. Deflación de matrices

Si se desean encontrar varios vectores propios, el método puede ser extendido utilizando la técnica de deflación [22]. Después de encontrar un vector propio dominante v_{dom} , se deflaciona

la matriz original A restando una cantidad proporcional de $v_{dom}v_{dom}^T$ de A (En el caso de que la multiplicidad algebraica del valor propio dominante sea 1, ver [22]). El proceso se repite con la matriz deflacionada para encontrar el siguiente vector propio, y así sucesivamente. El algoritmo de deflación de una matriz se justifica teóricamente por el siguiente teorema de [22],

Teorema 3.2.2. *Sea A una matriz diagonalizable de tamaño $n \times n$ con valores propios $|\lambda_1| > |\lambda_2| > \dots > |\lambda_k|$, y sea P_1 la matriz de proyección ortogonal sobre el espacio propio asociado a λ_1 . Sea q_1 un vector propio unitario asociado a λ_1 . Entonces la matriz $A - \lambda_1 P_1$ tiene como valores propios a $0, \lambda_2, \dots, \lambda_k$.*

Demostración. Dado que A es diagonalizable podemos descomponerla como combinación lineal de matrices de proyección ortogonal (ver [22]), lo anterior lo podemos realizar de la siguiente manera; Sean $V_i = \{v \in \mathbb{R}^n | Av = \lambda_i v\}$ los espacios propios asociados a A y P_i la matriz de proyección en el espacio de V_i , luego por teorema de descomposición ortogonal de operadores diagonalizables de [22],

$$A = \lambda_1 P_1 + \lambda_2 P_2 + \dots + \lambda_k P_k. \quad (3-13)$$

Donde las proyecciones P_i son únicas y las podemos calcular encontrando una base ortonormal de V_i , colocando los vectores de esta como columnas de una matriz B_i y haciendo el producto $B_i B_i^T$. Dado que la descomposición de (3-13) es única (ver [22]) basta pasar a restar el primer sumando, para ver que $A - \lambda_1 P_1$ tiene como valores propios a $0, \lambda_2, \dots, \lambda_k$. $\square$

3.2.2. Método por iteración ortogonal

Por otra parte, el método de potencias iteradas lo podemos realizar considerando una matriz $n \times r$ Q_0 con r columnas ortogonales ($1 \leq r \leq n$), en lugar de un vector unitario q_0 para hacer la iteración [1]. Esta vez la iteración consistirá en hacer repetidamente facturación QR , como se ve en el pseudocódigo 6

Algorithm 6 Método por matriz Ortogonal

- 1: Escoja una matriz Ortogonal $Q_{(0)}$
 - 2: **for** $k = 1, 2, \dots$ **do**
 - 3: $Z_k = A Q_{k-1}$
 - 4: $Q_k R_k = Z_k$
 - 5: **end for**
-

Notemos que si $r = 1$, entonces lo anterior es solamente el método de potencias tradicional. de hecho, la secuencia $\{Q_k e_1\}$ es precisamente la secuencia de vectores producidas por la iteración de potencias, empezando con el vector $q^{(0)} = Q_0 e_1$.

Veamos porque este procedimiento converge a la matriz ortogonal de los vectores propios que de la matriz que estamos buscando. para ello nos guiaremos de la prueba mostrada en la Sección 8.3 de [1]. Para analizar el comportamiento del algoritmo 6 asuma que A es una matriz $n \times n$ simétrica y que

$$Q^T Q = D = \text{diag}(\lambda_i), \quad |\lambda_1| \geq |\lambda_2| \geq \cdots \geq |\lambda_n|. \quad (3-14)$$

Particionemos Q y D como sigue,

$$[Q_\alpha \mid Q_\beta] \quad \begin{bmatrix} D_1 & 0 \\ 0 & D_2 \end{bmatrix}. \quad (3-15)$$

Donde Q_α es de tamaño $n \times r$, Q_β es de tamaño $n \times n - r$, D_1 es de tamaño $r \times r$, D_2 es de tamaño $n - r \times n - r$. Si $|\lambda_r| > |\lambda_{r+1}|$, entonces

$$D_r(A) = \text{ran}(Q_0)$$

es el subespacio dominante invariante de dimensión r . Es el único subespacio invariante asociado a los valores propios $\lambda_1, \lambda_2, \dots, \lambda_r$ (ver [1]). El siguiente teorema (Teorema 8.2.2 de [1]) muestra condiciones razonables para que los subespacios $\text{ran}(Q_k)$ generados por 6, converjan a $D_r(A)$, a una rzon proporcional a $|\lambda_{r+1}/\lambda_r|^k$.

Teorema 3.2.3. *Sea A un matriz simétrica de tamaño $n \times n$ y considere la descomposición dada en (3-14) y (3-15), con $n \geq 2$. Asuma $|\lambda_r| > |\lambda_{r+1}|$ y que d_k es definido por*

$$d_k = \text{dist}(D_r(A), \text{ran}(Q_k)), \quad k \geq 0.$$

Si $d_0 < 1$, entonces las matrices Q_k , generadas por 7, satisfacen que

$$d_k \leq \left| \frac{\lambda_{r+1}}{\lambda_r} \right|^k \frac{d_0}{\sqrt{1 - d_0^2}}. \quad (3-16)$$

Demostración. Empezamos mencionando que la distancia entre dos subespacios S_1 y S_2 se define como

$$\text{dist}(S_1, S_2) = \|P_1 - P_2\|$$

donde P_1 y P_2 son las proyecciones ortogonales sobre los subespacios (ver [1]), además de esto se puede deducir que la condición $d_0 < 1$ significa que ninguno de los vectores de las columnas de Q_0 son perpendiculares a $D_r(A)$. Usando inducción podemos ver que de la iteración utilizada en 6 se cumple que

$$A^k Q_0 = Q_k (R_k \cdots R_1)$$

Esta es un factorización QR de $A^k Q_0$ y utilizando las ecuaciones (3-14) y (3-15), obtenemos lo siguiente

$$\begin{bmatrix} D_1^k & 0 \\ 0 & D_2^k \end{bmatrix} \begin{bmatrix} Q_\alpha^T Q_0 \\ Q_\beta^T Q_0 \end{bmatrix} = \begin{bmatrix} Q_\alpha^T Q_k \\ Q_\beta^T Q_k \end{bmatrix} (R_k \cdots R_1). \quad (3-17)$$

Si las matrices V_k y W_k son definidas por

$$\begin{aligned} V_k &= Q_\alpha^T Q_0, \\ W_k &= Q_\beta^T Q_0, \end{aligned}$$

entonces

$$D_1^k V_0 = V_k (R_k \cdots R_1) \quad (3-18)$$

y

$$D_2^k V_0 = V_k (R_k \cdots R_1). \quad (3-19)$$

Ya que

$$\begin{bmatrix} V_k \\ W_k \end{bmatrix} = \begin{bmatrix} Q_\alpha^T Q_k \\ Q_\beta^T Q_k \end{bmatrix} = [Q_\alpha | Q_\beta] Q_k = Q^T Q_k,$$

La ultima ecuación es la pequeña descomposición *CS* (Teorema 2.5.2 de [1]), de esto se sigue que

$$1 = \sigma_{\min}(V_k)^2 + \sigma_{\max}(W_k)^2 = \sigma_{\min}(V_k)^2 + d_k.$$

(Recordar que $\sigma_{\max}(A)$ y $\sigma_{\min}(A)$ hacen referencia al mayor y menor valor singular de la matriz A) Una consecuencia de esto es que

$$\sigma_{\min}(V_0)^2 = 1 - \sigma_{\max}(W_0)^2 = 1 - d_0^2 > 0.$$

Se sigue de la ecuación (3-18) que las matrices V_k y $(R_k \cdots R_1)$ son no singulares. Usando ambas y la ecuación (3-19) obtenemos que

$$W_k = D_2^k W_0 (R_k \cdots R_1)^{-1} = D_2^k W_0 (D_1^k V_0)^{-1} V_k = D_2^k (W_0 V_0^{-1}) D_1^{-k} V_k$$

y por lo tanto

$$\begin{aligned} d_k &= \|W_k\|_2 \leq \|D_2^k\|_2 \cdot \|W_0\|_2 \cdot \|V_0^{-1}\|_2 \cdot \|D_1^{-k}\|_2 \cdot \|V_k\|_2 \\ &= |\lambda_{r+1}|^k \cdot d_0 \cdot \frac{1}{1 - d_0^2} \cdot \frac{1}{|\lambda_r|^k}, \end{aligned}$$

y por tanto la ecuación (3-16) se tiene, como queríamos. $\square$

Tanto la iteración de potencia como el método de Jacobi tienen algunas ventajas atractivas para ciertas aplicaciones y situaciones, como su facilidad de implementación y comprensión [1]. los algoritmos solo requieren de operaciones matriciales básicas y no implican cálculos complejos. En el caso de la iteración de potencias es rápido y eficiente para matrices grandes y dispersas, ya que solo requiere un vector para almacenar y no hay memoria o almacenamiento adicional para obtener resultados intermedios. La multiplicación matricial-vectorial se puede hacer de manera eficiente utilizando técnicas de matriz dispersa o computación paralela [16]. Además, el algoritmo puede manejar matrices no simétricas y no normales, sin asumir ninguna propiedad específica de la matriz como simetría u ortogonalidad, siempre que tenga

un valor propio dominante único. En el caso del método de Jacobi el hacer las multiplicaciones por las matrices de Givens hace que este método sea extremadamente ligero y que no requiera de cálculos muy grandes dado que solo afecta a dos filas o columnas de la matriz que estemos trabajando, en general estos algoritmos son bastante eficientes y son usados como ideas para algoritmos mas complejos [1], compararemos el tiempo de computo para cierta matriz especial en el Capitulo 6 junto con su precisión.

4 Algoritmo bidiagonal de Golub-Kahan para la descomposición SVD.

Ahora procedemos a mostrar uno de los algoritmos que utilizan librerías de numpy para calcular la descomposición SVD, mas específicamente abordaremos la rutina de LAPACK GESVD (ver [23]), para calcular la descomposición SVD de una matriz de covarianza de imágenes de caras, y por tanto encontrar las caras propias. Cabe aclarar que en este proyecto no consideraremos cálculos en paralelo, ni tampoco utilizaremos optimización de la memoria cache, como si lo hacen las librerías de LAPACK (ver [2]), tan solo estudiaremos el algoritmo, desde el foco del álgebra lineal.

El algoritmo detrás de como se hace la descomposición SVD, cuando utilizamos alguna librería de numpy es en esencia *El algoritmo bidiagonal de Golub-Kahan para la descomposición SVD* [1]. Estas librerías tienen optimizado el uso de la memoria y procesadores en los computadores, además hay diferentes variantes de este mismo que se pueden utilizar, dependiendo si la matriz es esparza, o si cumple ciertas condiciones (ver [16]). En nuestro caso la matriz que utilizaremos no tiene alguna de estas propiedades especiales como para colocar alguna de estas variantes, por lo que nos iremos por el algoritmo tradicional.

La idea principal es reducir la matriz $A \in \mathbb{R}^{m \times n}$, a una matriz bidiagonal, bajo una transformación ortogonal, es decir descomponemos A en la forma

$$A = UB'V^T$$

donde $U \in \mathbb{R}^{m \times m}$ es ortogonal, $B' \in \mathbb{R}^{m \times n}$ es bidiagonal superior, $V \in \mathbb{R}^{n \times n}$ es ortogonal. Luego de la bidiagonalización, procedemos hacer descomposición SVD, a la matriz bidiagonal

$$B = U'_B \Sigma' V_B'^T.$$

Donde $U'_B \in \mathbb{R}^{m \times m}$ es ortogonal, $\Sigma' \in \mathbb{R}^{m \times n}$ es diagonal superior, $V' \in \mathbb{R}^{n \times n}$ es ortogonal, la notación prima en la matrices es porque en general no vamos a trabajar con la matriz completa si no con parte de ella, por ejemplo, no vamos a trabajar con la matriz bidiagonal superior B' , si no solo con la parte cuadrada $n \times n$ bidiagonal superior B .

El resultado final de la descomposición es entonces colocado como

$$A = UU'_B \Sigma' (VV'_B)^T.$$

4.1. Primera parte (bidiagonalización)

El algoritmo de bidiagonalización de Householder se basa en pre y post multiplicaciones con reflectores de Householder, de modo que al final del proceso se logra la siguiente relación

$$B' = \begin{bmatrix} B \\ 0 \end{bmatrix} = U_{n-1} \cdots U_1 A V_1 \cdots V_{n-2}.$$

Llamaremos $U = U_1 \cdots U_{n-1}$, y $V = V_1 \cdots V_{n-2}$, donde U_k, V_k son reflexiones de Householder, esta descripción fue hecha por Golub y Kahan en [1], recordemos que un reflector de Householder es definido como

$$H_v = I_n - \frac{2vv^T}{v^T v}. \quad (4-1)$$

Lo que hace esta matriz cuando se aplica a un vector x , es reflejarlo respecto al plano normal a v , como se ilustra en la Figura 4-1. Generalmente se intenta escoger v , de tal manera que

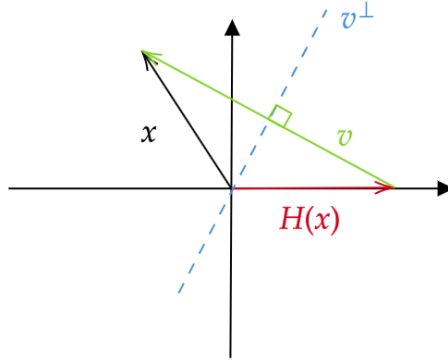


Figura 4-1: Reflexión de Householder en $\mathbb{R}^2$, escogiendo un vector v apropiado de tal manera que la reflexión de x respecto al normal de v sea un múltiplo escalar del vector canónico $(1, 0)$ (Idea de [1]).

la reflexión caiga en un múltiplo escalar de algún vector de la base canónica, por ejemplo para reflejar un vector x , sobre e_1 , se puede escoger v como

$$v = \frac{\|x\|e_1 - x}{\|x\| - x_1}$$

o como el negativo de este. Utilizando reflectores Householder, aplicados alternativamente desde la izquierda y desde la derecha, las entradas de la matriz se transforman a cero dejando una bidiagonal de valores distintos de cero. La matriz bidiagonal resultante se puede diagonalizar para resolver los valores singulares. Todo lo anteriormente dicho puede ser reducido en el Algoritmo 7, tomado de [1], en este caso la función **house** que aparece en el algoritmo, toma un vector y devuelve los parámetros para construir la matriz que lo refleja sobre el vector canónico e_1 , en este caso el parámetro β que se nombra en el Algoritmo 7, es el $2/vv^T$ que aparece en la ecuación (4-1).

Algorithm 7 Bidiagonalización de Householder

Require: $A \in \mathbb{R}^{m \times n}$, con $m \geq n$

- 1: **for** $j = 1 : n$ **do**
- 2: $[v, \beta] = \text{house}(A(j : m, j))$
- 3: $A(j : m, j : n) = (I_{m-j+1} - \beta vv^T)A(j : m, j : n)$
- 4: $A(j + 1 : m, j) = v(2 : m - j + 1)$
- 5: **if** $j \leq n - 2$ **then**
- 6: $[v, \beta] = \text{house}(A(j, j + 1 : n)^T)$
- 7: $A(j : m, j + 1 : n) = A(j : m, j + 1 : n)(I_{n-j} - \beta vv^T)$
- 8: $A(j, j + 2 : n) = v(2 : n - j)^T$
- 9: **end if**
- 10: **end for**
- 11:

4.2. Segunda parte (SVD de la parte bidiagonal)

Para esta parte necesitaremos algunos conceptos y procedimientos previos, que utilizaremos para describir el algoritmo general en la Sección 4.2.1 (consultar Sección 8.6 de [1] para más detalles de estos conceptos). En esta Sección explicaremos la forma en la que podemos calcular la descomposición SVD de una matriz bidiagonal de forma computacionalmente eficiente [1], ayudándonos principalmente de multiplicaciones por matrices de Givens.

Recordemos que dada una matriz A de tamaño $m \times n$, con $m \geq n$ (esta última condición se puede relajar para matrices $m \times n$ en general ver Sección 5.2 de [1], pero para nuestros intereses trabajaremos con $m \geq n$) siempre se puede encontrar una descomposición de ella como un producto de una matriz ortogonal Q , y una matriz triangular superior R , en particular la factorización QR para una matriz tridiagonal puede ser computada mediante matrices de Givens, de la siguiente manera (Sección 8.3 de [1]):

Algorithm 8 QR para matrices tridiagonales

- 1: $T = [t_{ij}]$ matriz tridiagonal
- 2: **for** $k = 1 : n - 1$ **do**
- 3: $[c, s] = \text{givens}(t_{kk}, t_{k-1,k})$
- 4: $m = \min(k + 2, n)$
- 5: $T(k : k + 1, k : m) = \begin{pmatrix} c & s \\ -s & c \end{pmatrix} T(k : k + 1, k : m)$
- 6: **end for**

Como las matrices de Givens son ortogonales y su producto es ortogonal, entonces el anterior proceso nos da la descomposición deseada ($Q =$ Producto de matrices de Givens, $R =$ Matriz

T' resultante del algoritmo anterior).

Suponga que T es una matriz simétrica tridiagonal, entonces podemos encontrar una aproximación a una descomposición de la forma $T = QDQ^T$, utilizando el procedimiento 9:

Algorithm 9 Iteración QR

- 1: $T_0 = T$
 - 2: Seleccione la cantidad de iteraciones N
 - 3: **for** $k = 1 : N$ **do**
 - 4: Factorize utilizando 8 $T_{k-1} = Q_{k-1}R_{k-1}$
 - 5: Construya $T_k = R_{k-1}Q_{k-1}$.
 - 6: **end for**
-

Para k , tenemos que $T_k = (Q_0 \cdot Q_1 \cdots Q_k)^T T_0 (Q_0 \cdot Q_1 \cdots Q_k)$, de esta forma cuando $k \rightarrow \infty$, obtenemos la descomposición deseada. Ahora bien hay formas en las que podemos acelerar la convergencia de la iteración QR, y esto es por medio de lo que llamamos “Shifts” (o cambios en español), para ver mas en detalle porque este ayuda a la convergencia del método puede consultar [1], ahora vamos a explicar en que consisten estos “Shifts”. Suponga que T es una matriz simétrica tridiagonal de tamaño $n \times n$, como se muestra a continuación,

$$T = \begin{pmatrix} a_1 & b_1 & & \cdots & 0 \\ b_1 & a_2 & \ddots & & \vdots \\ & \ddots & \ddots & \ddots & \\ \vdots & & \ddots & \ddots & b_{n-1} \\ 0 & \cdots & & b_{n-1} & a_n \end{pmatrix} \quad (4-2)$$

con

$$T(n-1:n, n-1:n) = \begin{pmatrix} a_{n-1} & b_{n-1} \\ b_{n-1} & a_n \end{pmatrix}, \quad (4-3)$$

si $s \in \mathbb{R}$, y $T - sI = QR$ es la factorización QR, entonces $T_+ = RQ + sI = Q^T T Q$, es también tridigonal, de esta manera se puede efectuar la iteración QR, a esta nueva matriz T_+ , como caso particular cuando este s , se escoge como el valor propio de (4-3) mas cercano a a_n , se llama Wilkinson shift, y este valor propios s , viene dado por

$$s = a_n + d - \text{sign}(d) \sqrt{d^2 + b_{n-1}^2},$$

donde $d = (a_{n-1} - a_n)/2$, por [3] se sabe que este método converge de forma cubica a la matriz D . Como caso particular para el método que vamos a ejecutar tenemos que $a_{n-1} = d_{n-1}^2 + f_{n-2}^2$, $b_{n-1} = d_{n-1}f_{n-1}$, $a_n = d_n^2 + f_{n-1}^2$.

4.2.1. Algoritmo

Ahora nos disponemos a exponer el algoritmo para calcular la SVD de la parte bidiagonal, para ello considere $T = B^T B$, aplicando el algoritmo QR con cambio implícito de Wilkinson (Algoritmo 8.3.2 de [3]) a esta matriz obtendremos una nueva matriz, tridiagonal $T_1 = Q_1^T T Q_1$, si lo volvemos hacer, obtendremos una matriz nuevamente tridiagonal $T_2 = Q_2^T T_1 Q_2$, y así sucesivamente obtenemos $T_n = Q_n T_{n-1} Q_n$, lo interesante de esto, es que si T es una matriz simétrica (como en nuestro caso), entonces este algoritmo converge a una matriz Q , y D , tales que $D = Q^T T Q$, donde D es una matriz diagonal, y Q es una matriz ortogonal. Con lo anterior una vez tengamos una buena aproximación de Q , podemos hacer lo mismo, esta vez aplicado a $\hat{T} = B B^T$, obteniendo otra matriz $\hat{Q}$, tal que $D = \hat{Q}^T \hat{T} \hat{Q}$, de esta manera

$$T = Q D Q^T = Q D^{1/2} \hat{Q}^T \hat{Q} D^{1/2} Q^T = (\hat{Q} D^{1/2} Q^T)^T (\hat{Q} D^{1/2} Q^T) = B^T B.$$

La última igualdad se tiene por el teorema Q implícito de [1] que nos asegura que lo que está en parentesis es efectivamente B , es decir, con el proceso anteriormente descrito hemos encontrado la factorización SVD de B .

Ahora todo este proceso se puede hacer simultáneamente, y sin tener que llamar explícitamente a la matriz T y $\hat{T}$: de la siguiente manera:

- i) Como se mostró en el “Paso QR simétrico implícito con cambio de Wilkinson”, para la primera iteración en el cálculo de la matriz Q , se computa matrices de Givens $G_1, G_2, \dots, G_{n-1}$, tales que $Q_1 = G_1 G_2 \dots G_{n-1}$, y $T_1 = Q_1^T T Q_1$ era tridiagonal, con $\text{off}(T_1) \leq \text{off}(T)$.
- ii) En vez de hacer lo anterior, suponga que aplicamos, estas matrices de rotación de Givens, directamente a la matriz B , nosotros necesitamos determinar matrices de rotación $U_1, V_2, U_2, \dots, V_{n-1}$ y U_{n-1} , para perseguir el elemento no deseado distinto de cero por la bidiagonal, es decir por cada aplicación de la rotación de givens aparece un elemento que es distinto de 0 fuera de la bidiagonal, la aplicación siguiente, borrará este elemento pero pondrá otro elemento distinto de 0 en otra parte (como se muestra en la parte de abajo), este proceso de perseguir el elemento distinto de cero, se detendrá cuando el elemento llegue a la posición $n, n-1$, en ese momento multiplicaremos por U_{n-1}^T , y la matriz nuevamente será bidiagonal. Esquemáticamente tenemos,

$$B \leftarrow B G_1 = \begin{bmatrix} \times & \times & 0 & 0 & 0 & 0 \\ + & \times & \times & 0 & 0 & 0 \\ 0 & 0 & \times & \times & 0 & 0 \\ 0 & 0 & 0 & \times & \times & 0 \\ 0 & 0 & 0 & 0 & \times & \times \\ 0 & 0 & 0 & 0 & 0 & \times \end{bmatrix}$$

$$\begin{aligned}
 B \leftarrow U_1^T B &= \begin{bmatrix} \times & \times & + & 0 & 0 & 0 \\ 0 & \times & \times & 0 & 0 & 0 \\ 0 & 0 & \times & \times & 0 & 0 \\ 0 & 0 & 0 & \times & \times & 0 \\ 0 & 0 & 0 & 0 & \times & \times \\ 0 & 0 & 0 & 0 & 0 & \times \end{bmatrix} & B \leftarrow BV_2 &= \begin{bmatrix} \times & \times & 0 & 0 & 0 & 0 \\ 0 & \times & \times & 0 & 0 & 0 \\ 0 & + & \times & \times & 0 & 0 \\ 0 & 0 & 0 & \times & \times & 0 \\ 0 & 0 & 0 & 0 & \times & \times \\ 0 & 0 & 0 & 0 & 0 & \times \end{bmatrix} \\
 B \leftarrow U_2^T B &= \begin{bmatrix} \times & \times & 0 & 0 & 0 & 0 \\ 0 & \times & \times & + & 0 & 0 \\ 0 & 0 & \times & \times & 0 & 0 \\ 0 & 0 & 0 & \times & \times & 0 \\ 0 & 0 & 0 & 0 & \times & \times \\ 0 & 0 & 0 & 0 & 0 & \times \end{bmatrix} & B \leftarrow BV_3 &= \begin{bmatrix} \times & \times & 0 & 0 & 0 & 0 \\ 0 & \times & \times & 0 & 0 & 0 \\ 0 & 0 & \times & \times & 0 & 0 \\ 0 & 0 & + & \times & \times & 0 \\ 0 & 0 & 0 & 0 & \times & \times \\ 0 & 0 & 0 & 0 & 0 & \times \end{bmatrix} \\
 B \leftarrow U_3^T B &= \begin{bmatrix} \times & \times & 0 & 0 & 0 & 0 \\ 0 & \times & \times & 0 & 0 & 0 \\ 0 & 0 & \times & \times & + & 0 \\ 0 & 0 & 0 & \times & \times & 0 \\ 0 & 0 & 0 & 0 & \times & \times \\ 0 & 0 & 0 & 0 & 0 & \times \end{bmatrix} & B \leftarrow BV_4 &= \begin{bmatrix} \times & \times & 0 & 0 & 0 & 0 \\ 0 & \times & \times & 0 & 0 & 0 \\ 0 & 0 & \times & \times & 0 & 0 \\ 0 & 0 & 0 & \times & \times & 0 \\ 0 & 0 & 0 & + & \times & \times \\ 0 & 0 & 0 & 0 & 0 & \times \end{bmatrix} \\
 B \leftarrow U_4^T B &= \begin{bmatrix} \times & \times & 0 & 0 & 0 & 0 \\ 0 & \times & \times & 0 & 0 & 0 \\ 0 & 0 & \times & \times & 0 & 0 \\ 0 & 0 & 0 & \times & \times & + \\ 0 & 0 & 0 & 0 & \times & \times \\ 0 & 0 & 0 & 0 & 0 & \times \end{bmatrix} & B \leftarrow BV_5 &= \begin{bmatrix} \times & \times & 0 & 0 & 0 & 0 \\ 0 & \times & \times & 0 & 0 & 0 \\ 0 & 0 & \times & \times & 0 & 0 \\ 0 & 0 & 0 & \times & \times & 0 \\ 0 & 0 & 0 & 0 & \times & \times \\ 0 & 0 & 0 & 0 & + & \times \end{bmatrix} \\
 B \leftarrow U_5^T B &= \begin{bmatrix} \times & \times & 0 & 0 & 0 & 0 \\ 0 & \times & \times & 0 & 0 & 0 \\ 0 & 0 & \times & \times & 0 & 0 \\ 0 & 0 & 0 & \times & \times & 0 \\ 0 & 0 & 0 & 0 & \times & \times \\ 0 & 0 & 0 & 0 & 0 & \times \end{bmatrix}
 \end{aligned}$$

iii) Una vez tengamos la matrices del item anterior, podemos ver que la nueva matriz bidiagonal se escribe como

$$B_1 = (U_{n-1}^T \cdots U_1^T) B (G_1 V_2 \cdots V_{n-1}) = U^T B V.$$

Por el Teorema Q implícito (Teorema 7.4.2 de [1]), podemos saber que justamente V se corresponde con el Q_1 de la factorización $T_1 = Q_1^T T Q_1$, al igual que U se corresponder con el $\hat{Q}_1$, por lo tanto podemos efectuar implícitamente la transición de T a $T_1 = B_1^T B_1$, trabajando directamente en la matriz bidiagonal B .

- iv) Ahora para el paso anterior hay problemas al seguir el elemento no nulo que se genera con las matrices de givens si alguno de los elementos de la bidiagonal es 0, puesto que cada elemento de la bidiagonal elimina el nuevo elemento distinto de 0 que aparece en el paso anterior y genera uno nuevo elemento distinto de 0 para que el algoritmo pueda seguir. De esta manera tenemos que tener cuidado a la hora de afrontar estos 0's.

Consideremos las matrices

$$B = \begin{bmatrix} d_1 & f_1 & \cdots & 0 \\ 0 & d_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & f_{n-1} \\ 0 & 0 & \cdots & d_n \end{bmatrix}, \quad T = B^T B = \begin{bmatrix} d_1^2 & d_1 f_1 & \cdots & 0 \\ d_1 f_1 & d_2^2 + f_1^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & d_{n-1} f_{n-1} \\ 0 & 0 & \cdots & d_n^2 + f_{n-1}^2 \end{bmatrix}.$$

Dado que las entradas de la subdiagonal de T son de la forma $d_i f_i$, es claro que debemos buscar ceros en la banda bidiagonal. Si $f_k = 0$ para algún k , entonces,

$$B = \begin{bmatrix} B_1 & 0 \\ 0 & B_2 \end{bmatrix}.$$

Donde B_1 es una matriz $k \times k$, y B_2 es una matriz $n-k \times n-k$, y por tanto el problema se reduce a dos problemas mas pequeños involucrando las matrices B_1 y B_2 . Si $d_k = 0$, para algún $k < n$, entonces se multiplica por una secuencia de transformaciones de Givens a izquierda para hacer 0 a f_k , como el ejemplo que se muestra a continuación

$$\begin{aligned} B &= \begin{bmatrix} \times & \times & 0 & 0 & 0 & 0 \\ 0 & \times & \times & 0 & 0 & 0 \\ 0 & 0 & 0 & \times & 0 & 0 \\ 0 & 0 & 0 & \times & \times & 0 \\ 0 & 0 & 0 & 0 & \times & \times \\ 0 & 0 & 0 & 0 & 0 & \times \end{bmatrix} & \xrightarrow{(3,4)} & \begin{bmatrix} \times & \times & 0 & 0 & 0 & 0 \\ 0 & \times & \times & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \textcolor{red}{+} & 0 \\ 0 & 0 & 0 & \times & \times & 0 \\ 0 & 0 & 0 & 0 & \times & \times \\ 0 & 0 & 0 & 0 & 0 & \times \end{bmatrix} \\ & \xrightarrow{(3,5)} & \begin{bmatrix} \times & \times & 0 & 0 & 0 & 0 \\ 0 & \times & \times & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \textcolor{red}{+} \\ 0 & 0 & 0 & \times & \times & 0 \\ 0 & 0 & 0 & 0 & \times & \times \\ 0 & 0 & 0 & 0 & 0 & \times \end{bmatrix} & \xrightarrow{(3,6)} & \begin{bmatrix} \times & \times & 0 & 0 & 0 & 0 \\ 0 & \times & \times & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \times & \times & 0 \\ 0 & 0 & 0 & 0 & \times & \times \\ 0 & 0 & 0 & 0 & 0 & \times \end{bmatrix} \end{aligned}$$

si $d_n = 0$, entonces la última columna se puede cerrar con una serie de multiplicaciones a derecha de las matrices de givens (se va suviendo el elemento distinto de cero por las posiciones $(n-1, n), (n-2, n), \dots, (1, n)$)

Todo lo anterior corresponde al Algoritmo bidiagonal de Golub-Kahan para la descomposición SVD de matrices con $m \geq n$, notemos que todos estos pasos se pueden efectuar aun cuando $n > m$, pero considerando algunos cambios pequeños en las multiplicaciones por las matrices ortogonales que vamos calculando en los sucesivos pasos. Como comentarios finales podemos destacar que Aunque la primera parte del algoritmo puede tener multiplicaciones grandes, esta es una parte mínima del algoritmo, pues solo se realizan $2n - 2$ multiplicaciones de estas, y luego se aprovecha el enfoque en pequeñas secciones de las matrices que tienen las rotaciones de Givens, haciendo que la convergencia sea cúbica [1], además a diferencia de los otros métodos expuestos, este sirve para cualquier tipo de matriz $m \times n$. Este tipo de algoritmos que consiste en bidiagonalizar, son bastante efectivos y estudiados (ver [16] para mas profundización), tanto es así que librerías como Numpy de Python, utilizan algoritmos de Intel [2], con este procedimiento para efectuar la descomposición SVD.

5 Componentes principales, utilizando el rango de un operador

5.1. Cuando el rango es el de una matriz.

Una idea bastante recurrente en matemáticas es utilizar información adicional para hacer que algoritmos o procedimientos sean mas eficientes [1], una de estas es utilizar el rango de un operador para poder reducir el espacio donde se esta buscando como se expone en [3], por ejemplo para nuestro caso quisiéramos buscar los vectores propios de una matriz V sabiendo estos están cerca al rango de una matriz G , para ello consideremos el proceso que se hizo en la iteración de potencias (3-6), pero esta vez, pero esta vez proyectando en cada iteración sobre el rango de G , mediante

$$P_G(x_0) := \arg \min_{w \in Im(G)} ||w - x_0||_2.$$

Estudiemos un poco mas en detalle quien es esta proyección, y tratemos de comparar como es la eficiencia de esta iteración con la de la iteración de potencias tradicional (3-6). Todo este procesos esta encaminado a entender el proceso pero cuando G es un modelo generativo, aun así intentemos ver como funciona para matrices y que garantías podemos dar de convergencia para este método.

Veamos esto un poco mas en detalle para una matriz G de tamaño 3×3 de rango 2. Es claro que con estas condiciones la imagen de G es un plano, es decir, existen constantes A_G, B_G, C_G , tales que

$$Im(G) = \{(x, y, z) | A_G x + B_G y + C_G z = 0\},$$

donde A_G, B_G y C_G , se pueden calcular por ejemplo como solución al sistema de ecuaciones (Suponiendo que las dos primeras columnas son l.i) como

$$G_{11}A_G + G_{21}B_G + G_{31}C_G = 0$$

$$G_{12}A_G + G_{22}B_G + G_{32}C_G = 0$$

$$A_G^2 + B_G^2 + C_G^2 = 1$$

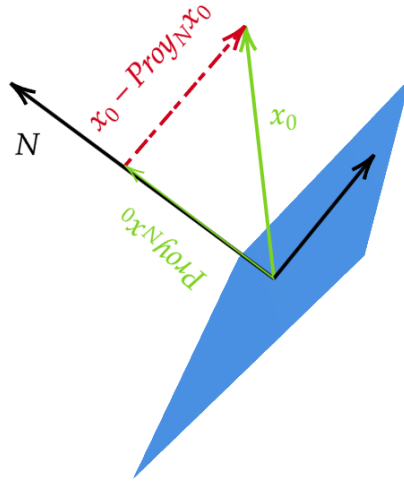


Figura 5-1: Proyección en el rango de G una matriz. Imagen propia

Ayudandonos de la Figura 5-1, podemos ver que

$$\begin{aligned}\overline{P}_G(x_0) &= \arg \min_{w \in \text{Im}(G)} \|w - x_0\|_2 \\ &= x_0 - \text{Proy}_N x_0 \\ &= x_0 - (N^T \cdot x_0)N\end{aligned}$$

Donde $\overline{P}_G(x_0)$ es la proyección sobre la imagen, no necesariamente normalizada, así la proyección normalizada que estamos buscando sería:

$$P_G(x_0) = \frac{\overline{P}_G(x_0)}{\|\overline{P}_G(x_0)\|}.$$

Note que este mismo procedimiento lo podemos generalizar, para una matriz G de tamaño $n \times k$, de rango $r \leq n$, basta encontrar una base para el espacio $\text{Im}(A)^\perp = \ker(A^T)$, (que la podemos tomar normalizada), y lo que haremos es simplemente:

$$\begin{aligned}\overline{P}_G(x_0) &= \arg \min_{w \in \text{Im}(G)} \|w - x_0\|_2 \\ &= x_0 - \sum_{i=1}^{n-r} \text{Proy}_{N_i} x_0 \\ &= x_0 - \sum_{i=1}^{n-r} (N_i^T \cdot x_0) N_i.\end{aligned}$$

Donde los vectores N_i , $i = 1, 2, 3, \dots, n - r$ forman una base para el espacio $\text{Im}(A)^\perp$. Note además que en este caso, primero proyectamos sobre la imagen de G , y luego normalizamos, para hallar la proyección sobre el círculo unidad. Este procedimiento es equivalente a

proyectar directamente sobre el círculo unidad contenido en el plano, puesto que

$$\|x_0 - y\|^2 = \|x_0 - \text{Proy}_G x_0\|^2 + \|y - \text{Proy}_G x_0\|^2. \quad (5-1)$$

Luego minimizar la distancia de x_0 a y que pertenecen al círculo unitario, es equivalente a minimizar la distancia de y que pertenecen al círculo con la proyección de x_0 sobre la imagen de G , de esta manera, nuestro proceso es equivalente al que se expondrá cuando tengamos la imagen de un VAE (ver Sección 4.2).

Con esto podemos hacer el método de potencias para alguna matriz V , pero en cada iteración proyectamos sobre el rango de la matriz G . Es decir hacemos,

$$w^{(t+1)} = P_G(Vw^{(t)}). \quad (5-2)$$

A Continuación mostramos un ejemplo en donde tenemos que la imagen de G , contiene un vector propio de V , y comparamos su eficiencia con el método de potencias tradicional.

$$G = \begin{bmatrix} 1 & 4 & 1 \\ 0 & 6 & 0 \\ 0 & 5 & 0 \end{bmatrix}, \quad V = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad (5-3)$$

en este caso es claro que el valor propio de V de mayor magnitud es 3, y esta asociado al vector propio $v = [1 \ 0 \ 0]^T$ que se encuentra en el espacio columna de G . Aplicando el método (5-2), a las matrices (5-3), vemos que, después de 11 iteraciones, empezando con un vector aleatorio de componentes enteras $[9 \ 5 \ 10]^T$, el método de potencia proyectado, ya alcanza 5 cifras significativas. y es un poco mas rapido que el metodo de potencia tradicional:

Potencias proyectado	Método de potencias
0.0218	0.0488
0.0116	0.0325
0.0061	0.0217
0.0033	0.0145
0.0017	0.0096
0.0009	0.0064
0.0005	0.0043
0.0003	0.0029
0.0001	0.0019
0.0001	0.0013
0.0000	0.0008

Tabla 5-1: Comparación entre Potencias proyectado y potencias, cuando G contiene en su rango al vector de mayor valor propio de V , con la métrica $\|w^{(t)} - v\|_2$.

Lo anterior, inspira la idea de utilizar otro tipo de rangos para buscar los valores propios de una matriz, dado que como vemos, en la Tabla 5.1. se gana bastante precisión únicamente utilizando un conocimiento adicional sobre los datos. La siguiente sección utiliza lo aprendido en el Capítulo 2 sobre redes neuronales y codificadores automáticos variacionales para estudiar la convergencia del método de potencias, con información adicional guardada en el entrenamiento de una red neuronal, si se quiere profundizar mas en esta idea puede consulta [?], en donde bajo ciertas Hipótesis sobre la red se puede dar una estimativa cuadrática del error y la cantidad de iteraciones para llegar a él.

5.2. Cuando el rango es el decoder de un VAE

Antes entrar al algoritmo, daremos algunas notaciones que utilizaremos, durante esta sección, que nos servirán para justificar la convergencia del algoritmo, se recomienda ver el Capítulo 2 de preliminares antes de entrar a leer la presente sección, allí se exponen algunos conceptos de probabilidad (Sección 2.2 y la referencia [9]), de redes neuronales (Sección 2.1 y las referencias [7, 6, 4]), de conceptos relacionado con inferencia variacional y el codificador variacional (VAE) (Subsección 2.2.3, la Sección 2.2.5 y las referencias [8, 3]). En esta sección intentaremos dar un acercamiento teórico al como trabajar con una red neuronal ya entrenada para poder obtener los valores propios de una matriz [3], esta idea principalmente se basa en el hecho de que las redes neuronales aprenden ciertos patrones fundamentales de los datos que son difíciles de ver para nosotros los humanos dada la cantidad de no linealidades que se introducen cuando se trabaja con ellas [7], el objetivo de esto es tomar algo de información de estos patrones y ver que tanta conexión tienen con los patrones que se aprende cuando hacemos descomposición SVD de la matriz. Comenzaremos dando algunas definiciones que plantean una base teórica sólida para poder tener herramientas que demostraran la convergencia del Método sobre el cual se basa esta sección (para mas detalles de los teoremas y definiciones puede consultar [3]).

Recordemos la Sección 2.2.5, en donde vimos un tipo de red neuronas especial la cual es conocida como VAE, esquemáticamente podemos reconocerla por la Figura 2-18, guiándonos por este gráfico y por la definición que dimos de VAE tenemos que n va a denotar la dimensión de los datos, por ejemplo en nuestro caso que trabajaremos con imágenes así n denotara la cantidad de píxeles, m denotara la cantidad de datos, k denotar la dimensión del espacio latente, $G : \mathbb{R}^k \rightarrow \mathbb{R}^n$ va a denotar el decoder del VAE, además nos referiremos a d como la cantidad de capas neuronales y n_i denotara la cantidad de neuronas en la capa i del decoder.

Ahora procedamos a dar algunas definiciones sobre el espacio en el cual va a trabajar nuestro VAE, todas estas también pueden ser consultadas en [3].

Definición 5.2.1. Sean n, k números enteros positivos, q y $L, r > 0$ números reales, entonces diremos lo siguiente:

- i) Llamaremos $\mathcal{S}^{n-1}$ a la esfera hueca de radio 1 de $\mathbb{R}^n$, es decir, $\mathcal{S}^{n-1} := \{x \in \mathbb{R}^n : \|x\|_2 = 1\}$.
- ii) Llamaremos $\mathcal{B}_q^k(r)$ a la esfera llena de radio r en la norma $\|\cdot\|_q$ de $\mathbb{R}^k$, es decir, $\mathcal{B}_q^k(r) = \{z \in \mathbb{R}^k : \|z\|_q = \sqrt[q]{|z_1|^q + \dots + |z_k|^q} \leq r\}$.
- iii) Diremos que una función $f : \mathbb{R}^k \rightarrow \mathbb{R}^n$ es L -lipchitz, si para todo x, y en su dominio, se cumple que $\|f(x) - f(y)\|_2 \leq L\|x - y\|$.

Ahora introduciremos algunas notaciones para la complejidad de una red neuronal, estas las utilizaremos para entender el Lema 5.2.2 y el Teorema 5-8, ambos también están enunciados en [1] y allí también podemos consultar su demostración, estos nos dan una cota para el error en el algoritmo que introduciremos en esta sección.

Definición 5.2.2. Sean $f : \mathbb{N} \rightarrow \mathbb{R}$ y $g : \mathbb{N} \rightarrow \mathbb{R}$, dos sucesiones de números reales.

- i) Diremos que $f(i)$ es $\Omega(g(i))$ o que $f(i) \geq \Omega(g(i))$, si existen constantes $c > 0$ y $i_0 \geq 0$ tales que $f(i) \geq c \cdot g(i)$, para todo $i \geq i_0$.
- ii) Diremos que $f(i)$ es $O(g(i))$, o que $f(i) = O(g(i))$, si existen constantes $c > 0$ y $i_0 \geq 0$ tales que $0 \leq f(i) \leq c \cdot g(i)$.
- iii) Diremos que $f(i)$ es $\Theta(g(i))$ o que $f(i) = \Theta(g(i))$, si es $\Omega(g(i))$ y $O(g(i))$, es decir, si existen constantes $c_1, c_2 > 0$ y $i_0 \geq 0$, tales que $0 \leq c_1 \cdot g(i) \leq f(i) \leq c_2 \cdot g(i)$, para todo $i \geq i_0$.

Estas notaciones son generalmente utilizados en el ámbito de algoritmos y nos dan limites superiores o inferiores del tiempo en que se demora en ejecutar un algoritmo dependiendo de que tan grande sea la entrada [10]. Por ejemplo considere la sucesión $f(i) = \sqrt{i}$, en este caso tenemos que $f(i) \geq \Omega(\ln i)$. Otro ejemplo seria tomar $f(i) = \arctan(i)$, en este caso tenemos que esta sucesión escala como $\Theta(1)$, dado que para n suficientemente grande esta esta acotada por $\pi/2$ y 0. Para nuestro caso particular, se asumirá que nuestro generador es una función L -lipchitz, con L que cumple $L = n^{\Theta(d)}$ (d el número de capas neuronales, y n la dimensión de los datos). Otro ejemplo sencillo es tomar $f(i) = 2i^2 + i$, y tendremos que para i suficientemente grande existirá $c = 4$, que cumple $|f(i)| \leq 4i^2$, de esta manera $f(i) = O(4i^2)$. Si se quiere consultar mas acerca de estas notaciones, puede leerse [10].

Ya con la notación clara procedemos a entrar al método, primero notemos que encontrar el vector propio asociado al mayor valor propio de una matriz simétrica definida positiva V , es equivalente a resolver el problema de maximización:

$$u_1 = \arg \max_{w \in \mathbb{R}^n} w^T V w. \quad (5-4)$$

Sujeto a por ejemplo que $\|w\|_2 = 1$ (ver [22] para justificación de esta afirmación). Ahora notemos que la restricción que colocamos puede ser bastante grande para este problema si tenemos algún conocimiento previo de la matriz, es por ello que en la referencia [3], se inspiran en este hecho y tratan de resolver el siguiente problema,

$$u_1 = \arg \max_{w \in \mathbb{R}^n} w^T V w \quad \text{s.t.} \quad w \in \text{Range}(G), \quad (5-5)$$

Donde G es un modelo generativo (Decoder de un VAE), pre-entrenado, el cual se asume que tiene el rango contenido en $\text{Range}(G) \subseteq \mathcal{S}^{n-1}$, y que tiene información acerca de como se distribuyen las imágenes.

En general si tenemos un modelo generativo G , este se puede normalizar, simplemente tomando la función $\hat{G}(z) = \frac{G(z)}{\|G(z)\|_2}$. Además, para propósitos teóricos el dominio de esta función se va a reducir a $\mathcal{D} := \{z \in B_2^k(r) : \|G(z)\|_2 > R_{\min}\}$, Para algún $R_{\min} > 0$, Así nuestro modelo generativo sera en realidad la función $\hat{G} : \mathcal{D} \rightarrow \mathcal{S}^{n-1}$, descrita como arriba. Ahora si L era la constante de Lipchitz de G , entonces la constante de Lipchitz de $\hat{G}$ será $L/R_{\min}$, además para una red neuronal con d capas se va a tener tipicamente que $L = n^{\Theta(d)}$ [3]. De esta manera podemos tomar $R_{\min}$ siendo mas pequeño que $1/n^{\Theta(d)}$.

Para aproximar una solución a (5-5), la propuesta que se da en [3], es utilizar el método de potencias, pero esta vez en cada iteración proyectamos sobre el rango de G . Mas específicamente si V es una matriz simétrica definida positiva. Entonces, el algortimo se basa en hacer la iteración

$$w^{(t+1)} = \mathcal{P}_G(Vw^{(t)}), \quad (5-6)$$

donde

$$\mathcal{P}_G(x) := \arg \min_{w \in \text{Range}(G)} \|w - x\|_2. \quad (5-7)$$

Al igual que en el método de potencias el vector inicial w^0 se escoge de manera aleatoria, o para algunas aplicaciones especificadas en [3], se escoge como la columna que tiene la entrada en la diagonal en valor absoluto mas grande de V .

Algorithm 10 Ppower

Require: V una Matriz simétrica definida positiva, T número de iteraciones, G un modelo pre-entrenado, y $w^{(0)}$ un vector inicial

- 1: $t = 0$
- 2: **for** $t = 0, 1, \dots, T - 1$ **do**
- 3: $w^{(t+1)} = \mathcal{P}_G(Vw^{(t)})$
- 4: **end for**
- 5: Devolver $w^{(T)}$

Ahora vamos a enunciar algunos Lemas, que nos aseguran la convergencia de 5-7. De aquí en los que sigue V va a ser una matriz simétrica definida positiva, con valores propios $\lambda_1 > \lambda_2 \geq \dots \lambda_n > 0$, y $\hat{x}$ va a ser un vector propio de norma 1, asociado a λ_1

Lema 5.2.1. *para cualquier $x \in \mathbb{R}^n$, sea $Q(x) = x^T V x$, entonces si V simétrica y definida positiva la secuencia $\{Q(w^{(t)})\}_{t>0}$, para $w^{(t)}$ definido en (5-7), en monótonamente no decreciente.*

Lema 5.2.2. *Sea V una matriz simétrica definida positiva, y asuma que existe un vector propio unitario $\hat{x}$ asociado al mayor valor propio λ_1 de V , entonces para todo $s \in \text{Range}(G)$, que satisface $S^T \hat{x} > 0$. Tenemos que:*

$$\|\mathcal{P}_G(Vs) - \hat{x}\|_2 \leq \frac{2\lambda_2 \|s - \hat{x}\|_2}{\lambda_1 s^T \hat{x}} + \frac{C}{s^T \hat{x}} \sqrt{\frac{k \log(nLr)}{m}}. \quad (5-8)$$

Teorema 5.2.3. *Sea V y $\hat{x}$ como en los anteriores teoremas, asuma que existe $t_0 \in \mathbb{N}$ tal que $\hat{x}^T w^{(t_0)} = 2\gamma + \nu < 1 - \tau$, donde $\gamma = \frac{\lambda_2}{\lambda_1}$, y ν, τ , ambos son positivos y escalan como $\Theta(1)$. llamemos a $\mu_0 = \frac{2\gamma}{\hat{x}^T w^{(t_0)}}$, y en adición suponga que $m \geq C_{\nu, \tau} \cdot \log(nLr)$, con $C_{\nu, \tau} > 0$, entonces tenemos que después de $\Delta_0 = O(\log(\frac{m}{k \log(nLr)}))$ iteraciones del método de Ppower:*

$$\|w^{(t)} - \hat{x}\|_2 \leq \frac{C_{\nu, \tau}}{(1 - \mu_0)\nu} \sqrt{\frac{k \log(nLr)}{m}}. \quad (5-9)$$

El Lema 5.2.1 y el Lema 5.2.2 son parte importante de la demostración del Teorema 5.2.3, en estos básicamente se demuestra que la sucesión definida por la ecuación (5-7) es decreciente y llega a cierta cota cuando se toma la proyección. El Teorema 5.2.3 es demostrado en [3] y nos da una estimación del error de reconstrucción del vector propio dominante, en términos de la dimensión latente k , la cantidad de datos m , la dimensión de estos n y la constante de lipchitz del decoder. También impone ciertas condiciones sobre la inicialización del método de potencias ($\hat{x}^T w^{(t_0)} = 2\gamma + \nu < 1 - \tau$). En general la constante de Lipchitz no es tan fácil de calcular y resulta de un estudio profundo de los datos y la estructura de la red neuronal, además cabe aclarar que esta es un estimador estadístico, pues la optimización de la proyección puede escoger entre diferentes mínimos [3]

En el siguiente capítulo y en el código anexo a este trabajo (ver [12]) se estudia la eficiencia computacional de este método y se modifican ciertas condiciones (como que $n \gg m$ y la condición de inicio) para ver su resultado, y compáralo con los resultados de los métodos estudiados en capítulos anteriores.

6 Comparación

Ahora nos disponemos a hacer las respectivas comparaciones en tiempo y precisión de cada uno de los métodos expuestos anteriormente, para ello, lo que haremos es ver como se comportan calculado los valores propios de matrices de tamaño 821×821 que aparecen cuando se intenta hacer reconocimiento facial. Mas específicamente, suponga que tenemos un conjunto de m fotos de caras, y que cada cara tiene n píxeles, nosotros podemos armar una tensor M , de tamaño $n \times m \times 3$, intuitivamente por cada canal esta matriz contiene en cada columna la información de una cara como se muestra en la Figura 6-1,

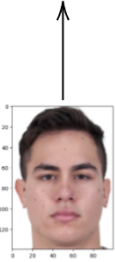
$$M = \begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & \dots & a_{1m} \\ a_{21} & a_{22} & a_{23} & \dots & \dots & a_{2m} \\ a_{31} & a_{32} & a_{33} & \dots & \dots & a_{3m} \\ \vdots & \vdots & \vdots & \ddots & \ddots & \vdots \\ \vdots & \vdots & \vdots & \ddots & \ddots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \dots & \dots & a_{nm} \end{bmatrix}$$


Figura 6-1: Tensor de datos M , en donde la columna j representa la información contenida en una imagen j del data set que hallamos tomado (en nuestro caso es el de las caras de la Universidad de Chicago [11]), y la fila i -ésima representa la información contenida en el pixel i -ésimo de todas las imágenes. Imagen inspirada en [11]

Notemos ahora la fila i de un canal fijo en el anterior tensor, corresponde al pixel número i de todas las imagen en cierto orden (por ejemplo colocando cada fila de pixeles de la imagen concatenada una debajo de la otra). Para lo que sigue se va a tomar a M en un canal fijo por ejemplo el rojo (M_r) para hacer mas entendible el procedimiento de caras propias. En general se toma esta matriz M_r , como aquella en la que cada columna j es la cara j -ésima pero restándole la “cara promedio”, donde la cara promedio se calcula sumando todas la caras y dividiendo entre el número total. De esta manera, $M_r M_r^T$ corresponde a una

realización de la matriz de covarianza, en donde cada entrada nos da la relación o covarianza entre los píxeles, de esta manera si calculamos los vectores y valores propios de esta matriz, estamos viendo la direcciones en donde mas se correlacionan ciertos píxeles. Si lo pensamos en términos de imágenes esto es bastante útil, pues es entendible que hallan ciertos lugares en donde los píxeles siempre tengan colores muy parecidos, por ejemplo el contorno de todas las caras es muy similar, y por tanto a priori podemos ver que se tiene que correlacionar de alguna forma, o por ejemplo si las imágenes tienen fondo, es de saber que los píxeles que hallan en las esquinas siempre sean del mismo color, o aquellas zonas de la cara que la mayoría comparte como los ojos, nariz y boca, también van a concentrar píxeles que tengan alta correlación, en la Figura 6-2 se ve el primer valor propio de la matriz MM^T para un conjunto de datos de caras (ver [11]), en donde apreciamos las características mas comunes a las imágenes, en términos de sombras y color.

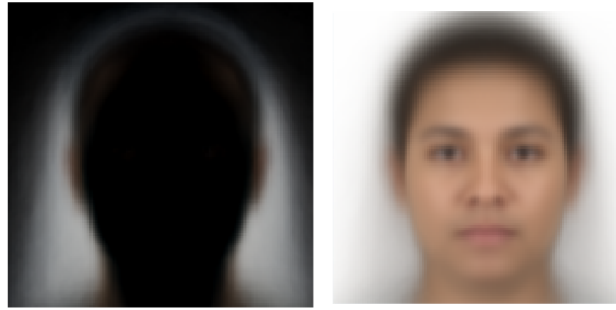


Figura 6-2: *A la izquierda se encuentra la primer cara propia del conjunto de datos de caras de chicago tomado de [11], esta cara es la representación del vector propio de mayor magnitud asociado a MM^T , a la derecha se encuentra la cara promedia de este conjunto de datos, que se calcula sumando todas la caras y dividiendo por el total en cada canal. Imagen propia*

Una vez hallamos calculado los valores y vectores propios de la matriz simétrica $M_r M_r^T$, basta encontrar los vectores, cuyos valores propios en magnitud sean mas grandes, y estos nos darán las direcciones de mayor correlación, y por tanto si tomamos estas direcciones como una nueva base, estamos reduciendo la cantidad de información que tenemos que guardar.

Por ejemplo si queremos utilizar lo anterior para hacer reconocimiento facial de manera óptima y teniendo un base de datos previa, lo que podríamos hacer es los siguiente: Construir la matriz $M_r M_r^T$, calcular los vectores propios de esta matriz, seleccionar aquellos vectores propios que tengan asociados un valor propio de magnitud grande (generalmente basta escoger menos del 10 % del total de los vectores propios), una vez tengamos estos vectores propios, cuando llegue una cara nueva, lo que haremos es proyectarla en estos, y ver si sus coordenadas coinciden con las de alguna otra cara en el dataset.

Ya que tenemos la idea del problema sobre el cual vamos a trabajar para comparar nuestros algoritmos, nos disponemos a mostrar, el dataset sobre el cual trabajaremos, y como compararemos la eficiencia de los métodos que discutimos en capítulos anteriores.

La base de datos de rostros de chicao, desarrollada en la Universidad de Chicago (ver [11]) es una herramienta diseñada para la investigación científica. Esta base de datos ofrece fotografías de alta resolución y estandarizadas de rostros masculinos y femeninos de diversas etnias, con edades comprendidas entre 17 y 65 años. El conjunto de datos está compuesto por 821 fotografías de caras tomadas de personas de diferentes etnias y razas reclutadas en los Estados Unidos y en India. En algunas fotografías se muestran diferentes expresiones de un mismo individuo.

Para nuestro propósitos, trabajaremos con imágenes de tamaño 140×100 , de esta manera la matriz M_r de datos que trabajaremos luego de haberla centrado es de tamaño 14000×821 . En principio según lo expuesto anteriormente debemos calcular los valores propios de la matriz $M_r M_r^T$, pero dado que esta matriz es de tamaño 14000×14000 , y necesitamos pocos valores propios, se hace ineficiente trabajar con una matriz tan grande. Es por ello que haremos todo el proceso con la matriz $\Sigma_r = M_r^T M_r$, ya que si $M_r = U D^{1/2} V^T$, es la descomposición SVD de M_r , Entonces $\Sigma_r = M_r^T M_r = V (D^{1/2})^T D^{1/2} V^T = V D V^T$ luego calcular los valores y vectores propios de $M_r M_r^T = U D^{1/2} (D^{1/2})^T U^T$, es equivalente a calcular los valores y vectores propios de Σ_r , y luego pasarlos por M_r , como se ve a continuación.

$$M_r V = U D^{1/2}. \quad (6-1)$$

Así trabajaremos con la matriz Σ_r de tamaño 821×821 . A los elementos de la diagonal de $D^{1/2}$ de (6-1) los denotaremos por σ_i para $i = 1, 2, \dots, 821$, luego todos los valores propios de Σ_r son de la forma σ_i^2 dado que esta matriz es simétrica y definida positiva.

Ahora lo que haremos es una tabla del tiempo que se demoran cada uno de los métodos, en obtener todos los vectores propios de Σ_r , y miraremos con que precisión los obtienen, para ello hemos disidido optar por la métrica de frobenius, puesto que codifica la idea de que todos los valores propios se encuentren cerca .

$$\sum_{i=1}^{821} \|M_r^T M_r v_i - v_i \sigma_i^2\|_2^2 = \|M_r^T M_r V - V (D^{1/2})^T D^{1/2}\|_F^2 \quad (6-2)$$

$$= \|\Sigma_r V - V D\|_F^2. \quad (6-3)$$

En la Tabla **6-1**, encontramos la comparación entre métodos, esta se hizo tomando un máximo de 1000 iteraciones y un error umbral fuera de la diagonal de D de 10^{-5} , se tomo el promedio de correr el programa con cada método 5 veces.

	Tiempo en minutos	Precisión $\ \Sigma_r V - VD\ _F^2$
Potencias con Deflación	11.13	$1,8765 \cdot 10^{-4}$
Potencias matriz Ortogonal	13.44	$1,3527 \cdot 10^{-6}$
Jacobi	0.66	$5,1705 \cdot 10^{-27}$
Golub and Kahan	1.96	$9,9853 \cdot 10^{-10}$

Tabla 6-1: Comparación en tiempo y presición ($\|\Sigma_r V - VD\|_F^2$) entre los algoritmos expuestos en los anteriores capítulos, para el calculo de los valores propios de una matriz de tamaño 821×821

Como vemos en la tabla el algoritmo con mejor tiempo y presición fue el de Jacobi, esto es explicable puesto que su convergencia es cubica (mayor a la convergencia lineal de las iteraciones de potencias) y un poco mayor a la del algoritmo bidiagonal de Golub-Kahan, como se ve en [1]. Aunque para la aplicación particular que presentamos el algoritmo mas rápido y preciso fue el de Jacobi, no quiere decir que los otros no tengan sus ventajas en algunos otros ámbitos, por ejemplo el algoritmo bidiagonal de Golub-Kahan, sirve para todo tipo de matrices, a diferencia del de Jacobi que solo computa la descomposición de valores propios de una matriz simétrica.

Para nuestro caso encontramos que el método proyectado de potencias no se puede comparar en estos ámbito con los anteriores métodos, dado que computacionalmente la optimización de una red neuronal no es tan rápida como la multiplicación de matrices (ver [4]) que realizan los otros métodos, aún así decidimos estudiar computacionalmente su compartimiento, a continuación se muestra parte de la implementación y resultados obtenidos. La implementación de este método se utilizo la librería de python llamada *Keras*, que da facilidad para la implementación de los VAE, dado que se puede definir explícitamente el encoder, decoder, bottleneck y los métodos para el entrenamiento del mismo, para mas detalle sobre la esta librería puede consultar [24] y para detalles sobre la teoría puede consultar [25] y anexos. Los datos fueron modificados para poder obtener tiempos de computo que no fueran extremadamente demorados para nuestros propósitos, de esta manera se trabajo con 821 imágenes de tamaño $28 \times 28 \times 3$ (restándoles la cara promedio en cada canal), que se guardaron en un tensor M de tamaño $821 \times 784 \times 3$, y con el cual se entreno la red neuronal. La arquitectura del VAE, fue de $2352 - 512 - 256 - 30$ neuronas por parte del encoder y $30 - 256 - 512 - 2352$ neuronas por parte del decoder (donde las activaciones se hicieron con ReLU para las capas intermedia y sigmoide para la capa final), la dimensión latente que se decidió utilizar fue 30, la tasa de aprendizaje de Adam (descenso del gradiente modificado) fue de 0.001, el tamaño de lote fue de 32 y las épocas fueron 100, estos hiperparámetros se ajustaron de acuerdo a mediciones como las hechas en la Tabla 6-2 y por las selecciones de los mismos hechas en [3].

La implementación de la proyección $\mathcal{P}_G$ definida en el Capítulo 5 requiere de una optimización para encontrar el mínimo de las distancias a la imagen del encoder normalizado, esta parte se hizo utilizando también Adam, con tasa de aprendizaje de 0,6, para seleccionarla se tomo un promedio de los errores de reconstrucciones de puntos que están en la imagen del decoder para diferentes tasas, los resultados se presentan en la Figura 6-3 a).

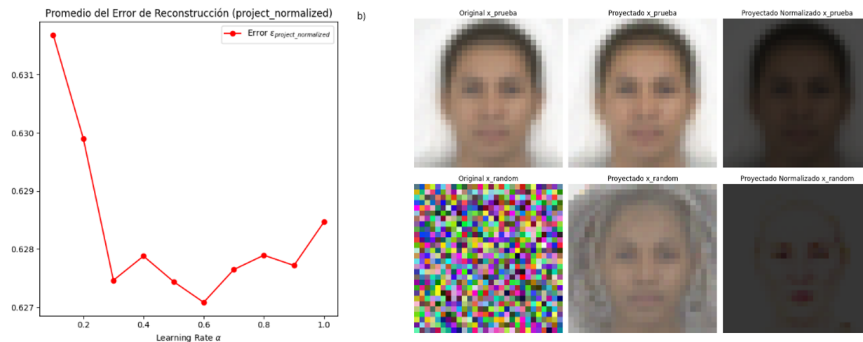


Figura 6-3: a) En esta gráfica se ve el promedio del error de reconstrucción para diferentes valores de tasa de aprendizaje, para una imagen en el rango del decoder (la imagen correspondiente a la decodificación del 0 en dimensión 30) normalizado fija, se ve como alrededor de 0.6 obtenemos un mínimo de 0.625. b) En esta parte se ve la proyección de una imagen que pertenece al decoder sobre el mismo y sobre su imagen normalizada, también se visualiza la proyección de un imagen aleatoria tanto en la imagen del decoder, como en la imagen normalizada de este. Recordar que las imágenes son de caras y fueron tomadas de [11]. Imagen propia

Luego de haber ajustado los parámetros tanto de nuestra arquitectura para el conjunto de datos de las caras (restándole la cara promedio), como de nuestra proyección, se procedió a implementar Ppower definido por la ecuación (5-9) y tomado de [3], los resultados de esta implementación se tomaron para el calculo de los vectores propios dominantes de cada uno de los canales del tensor Σ^T , donde Σ^T es de tamaño $728 \times 784 \times 3$ (recordar que Σ^T es la matriz de covarianza de los datos en los 3 canales y se calcula multiplicando a M^T por M en cada canal). En Figura 6-4 se muestra el error en norma 2 entre el valor propio real dominante y el calculado por Ppower en cada uno de los canales, junto con el error entre los valores propios real y calculado. También se muestra una tabla en donde se ve el mínimo error en norma 2 entre los vectores propios en los 3 canales para diferentes espacios latentes (parece no ser determinante a la hora de hacer proyección sobre la imagen del decoder normalizado)

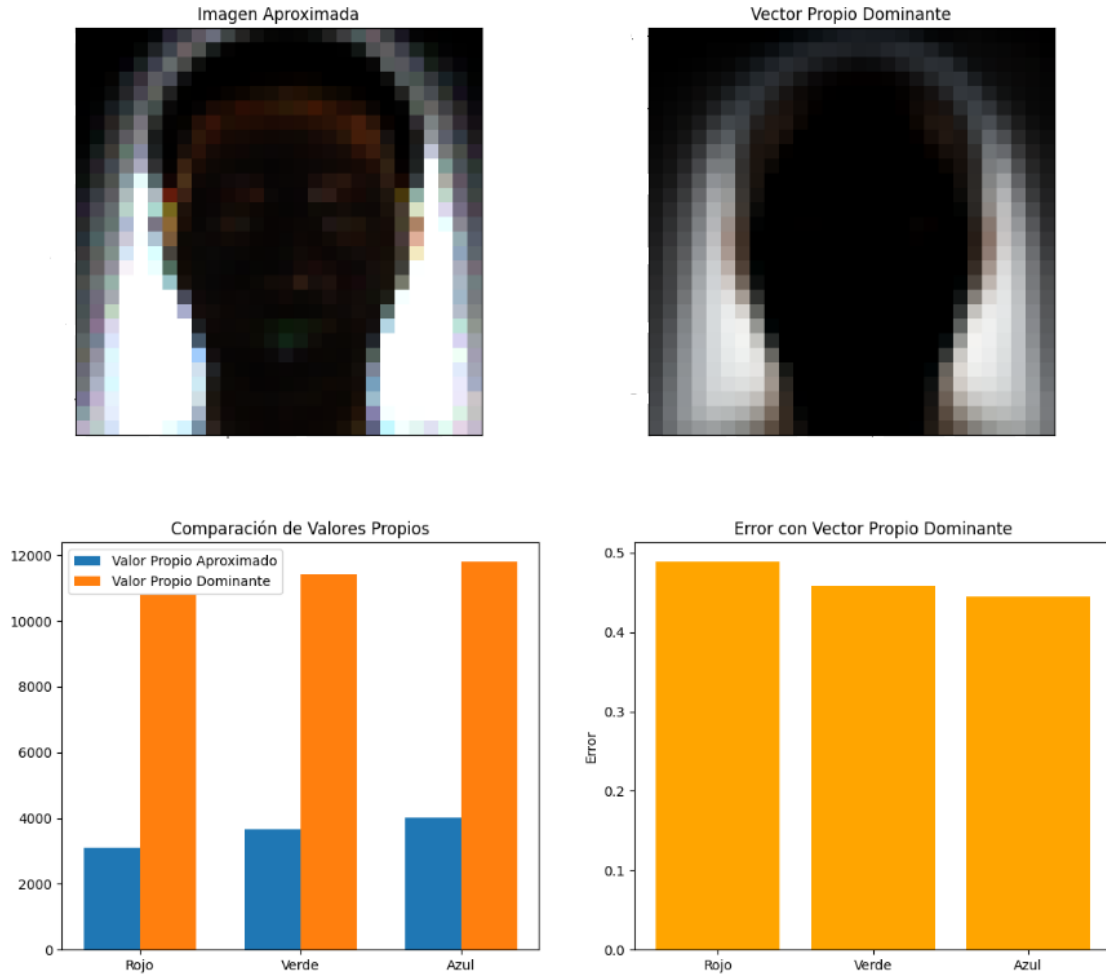


Figura 6-4: a) En la parte superior izquierda se ve el vector propio dominante de Σ^T (superponiendo los 3 canales) que aproxima Ppower con 10 iteraciones (se multiplico por un factor de 30 para poder visualizarla y no fuera totalmente oscura). En la parte superior derecha se ve el vector propio dominante de Σ^T real calculado con numpy. b) La gráfica de la parte inferior izquierda muestra la diferencia entre los valores propios dominantes en cada canal, en azul los calculados por Ppower y en naranja los calculados por la librería numpy de python. La gráfica de la parte inferior derecha muestra el error en norma 2 de los vectores propios dominantes en cada uno de los 3 canales. Puede consultar el código donde se hace esta implementación en [12]. Y del cual se inspiraron las imagenes de este documento

Dimensión espacio latente	30	60	90	120	150	180	210	800
$\min_{c \in \text{channel}} \ x_c - w_c^{(T)}\ $	0,45	0,44	0,434	0,437	0,434	0,435	0,434	0,436
Mejor tasa de Tasa de aprendizaje para P_G	0,6	0,5	0,5	0,4	0,5	0,3	0,4	0,6
Pérdida del VAE	13,17	13,12	13,97	14,22	14,82	16,01	16,16	16,49

Tabla 6-2: Resultados de reconstrucción del valor propio dominante de Σ^T con Ppower con 10 iteraciones, utilizando un VAE entrenado con M y diferentes dimensiones del espacio latente, en esta tabla también se miro el mejor ajuste de la tasa de aprendizaje para la optimización interna que hace P_G , y la perdida de entrenamiento del VAE, para verificar si había alguna relación entre esta y la reconstrucción del valor propio dominante calculado con Ppower.

Los resultados presentados por la Figura 6-4 y la Tabla 6-2, pueden ser verificados en el código anexo a este trabajo [12]. Estos resultados muestran que la representación de la cara propia dominante calculada con el método de Ppower captura otras características del conjunto de datos que la cara propia real no hace, esto puede ser debido en gran parte a la forma en la que se redujo el espacio a solo 30 dimensiones, los mínimos que se hallan en el entrenamiento del VAE o al conjunto de datos utilizado para el entrenamiento de la red nueronal. Como se ve en las gráficas de la parte inferior de 6-4, los resultados simplemente no se puede comparar con los obtenidos por los otros métodos en la Tabla 6-1, esto es debido a que como se mostró en el capítulo anterior existe una brecha bastante amplia para la convergencia del método proyectado de potencias (para mas información ver [3]), y por tanto nunca vamos a poder recuperar completamente el vector propio, el tiempo que tarda el método en calcular el vector propio dominante es de 27 segundos (sin contar la optimización del modelo generativo), lo que no lo hace viable tampoco temporalmente. Otro resultado que vemos en la Tabla 6-2, es que el espacio latente no influyo en que el error de reconstrucción del vector propio dominante fuera mejor, lo que hace pensar que el conjunto de datos en el entrenamiento tiene mayor influencia en que tanto se llena la bola unitaria a partir de la imagen del decoder (normalizado). Otro resultado que se observa es que a mayor dimensión latente (y a una cantidad de épocas fija) el error de entrenamiento del VAE se hace mas grande pero el error de reconstrucción del vector propio parece no cambiar.

A partir de los resultados anteriores y de los vacíos teóricos en torno a los hiperparámetros, datos y estructura computacional del VAE entorno a la optimización de la reconstrucción de caras propias se plantean los siguientes trabajos futuros: Estudiar la influencia del método de optimización en la función de perdida del VAE en la reconstrucción de los valores propios de una matriz con Ppower, en nuestro caso utilizamos Adam [20], entonces se espera dar garantías teóricas, sobre la tasa de aprendizaje de este método. Estudiar la influencia del método de optimización en la función de perdida de la proyección $\mathcal{P}_G$, en nuestro caso encontramos aproximaciones a mínimos de la reconstrucción del valor propio dominante en función de la variación de la tasa de aprendizaje, pero queremos encontrar una relación

entre estos métodos computacionales y una teoría que nos muestre esto en función de las dimensiones del conjunto de datos (o si depende de la estructura de los mismos). Estudiar la influencia en el error de reconstrucción de la cantidad de capas neuronales y funciones de activación que se utilizan en las mismas (saber que tipo de funciones aseguran una mejor convergencia o foliación de la bola unitaria, es decir, mejores estimadores que los obtenidos en [3]). Estudiar otro tipo de operadores en la búsqueda de valores propios de una matriz de datos, o encontrar espacios en donde se pueda codificar la información de manera óptima, como por ejemplo modelos de difusión que están siendo muy famosos últimamente en la generación de imagen [4].

Bibliografía

- [1] G. H. Golub, *Matrix computations*, vol. 1. Baltimore, Maryland 21218-4363: The Johns Hopkins University Press, four ed., january 2013.
- [2] Intel, “Singular value decomposition: Lapack computational routines.” <https://www.intel.com/content/www/us/en/docs/onemkl/developer-reference-c/2023-1/singular-value-decomposition-lapack-computation.html>, august 1980.
- [3] Z. Li, J. Liu, S. Ghosh, J. Han, and J. Scarlett, “Generative principal component analysis,” *Applied Numerical Mathematics*, vol. 169, p. 33, september 2021.
- [4] S. J. Prince, *Understanding Deep Learning*. The MIT Press, 2023.
- [5] vlarobbyk, “Desenso gradiente con acutlización de tasa de aprendizeje.” <https://github.com/vlarobbyk/descenso-por-gradiente/blob/master/Descenso-por-Gradiente.png>, september 2010.
- [6] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, “Automatic differentiation in machine learning: a survey.” <https://www.jmlr.org/papers/volume18/17-468/17-468.pdf>, april 2018.
- [7] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. “<http://www.deeplearningbook.org>.”
- [8] Ceyrón, “Variational inference: Symply explained.” <https://www.youtube.com/playlist?list=PLISXH-iEM4JloWnKysIEPPysGVg4v3PaP>, december 2022.
- [9] V. K. Stefano Ermon, “Notas de clase, probailidad.” <https://ermongroup.github.io/cs228-notes/>, december 2018.
- [10] GeeksforGeeks, “Codificadores automáticos variacionales.” <https://www.geeksforgeeks.org/variational-autoencoders/>, December 2023.
- [11] J. C. y. B. W. Debbie S. Ma, “Cfd base de datos de caras de chicago.” <https://www.chicagofaces.org/>, april 2015.

- [12] B. A. R. Castro and C. E. N. Guzman, “Códigos relacionados con el estudio de conceptos asociados al cálculo de componentes principales.” [“https://github.com/brromeroc/Codigos-del-proyecto-Revision-de-conceptos-asociados-al-c-alcuulo-de-componentes-Principales](https://github.com/brromeroc/Codigos-del-proyecto-Revision-de-conceptos-asociados-al-c-alcuulo-de-componentes-Principales), december 2023.
- [13] M. Nielsen, “How the backpropagation algorithm works.” <http://neuralnetworksanddeeplearning.com/chap2.html>, december 2019.
- [14] M. Stojiljković, “Stochastic gradient descent algorithm with python and numpy.” <https://realpython.com/gradient-descent-algorithm-python/>, january 2019.
- [15] L. B. Castañeda, *Probabilidad*, vol. 1. Bogota D.C: Universidad Nacional de Colombia - Facultad de Ciencias, 1 ed., june 2010.
- [16] N. Bosner, “Fast methods for large scale singular value decomposition,” *Department of Mathematics, University of Zagreb Bijenička cesta 30, 10000 Zagreb*, vol. 1, pp. 45–48, Zagreb 2006.
- [17] N. Abramson, *Teoría de la Información y la Codificación*, vol. 1. Boston, Massachusetts, Estados Unidos: McGRAW-HILL Book Company, Inc, 5 ed., june 1981.
- [18] J. Duchi, *Derivations for Linear Algebra and Optimization*, vol. 1. California: University of California, Berkeley, first ed., december 2014.
- [19] D. P. Kingma and M. Welling, “Auto-encoding variational bayes,” 2022.
- [20] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” 2017.
- [21] H. P.-G. Richard von Mises, *Praktische Verfahren der Gleichungsauflösung*, vol. 1. Berlin: ZAMM - Zeitschrift für Angewandte Mathematik und Mechanik, second ed., June 1971.
- [22] R. K. Kenneth Hoffman, *Linear Algebra*, vol. 1. Englewood Cliffs, New Jersey: Prentice-Hall, Inc., second ed., June 1971.
- [23] N. Bosner, “Accuracy and efficiency of one-sided bidiagonalization algorithm,” *Applied Numerical Mathematics*, vol. 169, p. 33, september 2004.
- [24] F. Chollet, “Keras page.” <https://keras.io/>, june 2015.
- [25] Z. Liu, J. Liu, S. Ghosh, J. Han, and J. Scarlett, “Implementación análisis generativo de componentes principales.” <https://github.com/liuzq09/GenerativePCA>, september 2022.
- [26] A. Bora, A. Jalal, E. Price, and A. G. Dimakis, “Compressed sensing using generative models,” 2017.

-
- [27] Ceyrón, “Interactive elbo.” <https://englishprobabilistic-machine-learningelbo-interactive-or5u7m.streamlit.app/>, april 2022.